

Automation using Python

Complete Course Book (Modules 1 - 19)

Instructor: Amol Chawathe

Platform: Quantbots.co

Generated: June 2026

Table of Contents

Module 1: Python Basics

[Introduction to Python](#)

[Data Types](#)

[Variables](#)

[Data Structure](#)

[Operations on Data Structure](#)

[Inbuilt methods](#)

Module 2: Data Types and Variables

[Data Types](#)

[Understanding Variables](#)

[Getting User Input and Print statements](#)

[Use quotes and escape character](#)

[Lists](#)

[Tuples](#)

[Dictionaries](#)

[Sets](#)

[Different Methods on data structures](#)

[String Indexing and Slicing Strings](#)

[Concatenation and Repetition](#)

[Common String Methods](#)

[String Formatting](#)

[Namespaces](#)

[Formatted String Literals \(f-strings and .format\(\) method\)](#)

[Built-in String Functions](#)

Module 3: Flow Control

[Python operators](#)

[If, else, elif clauses](#)

[Iteration](#)

[Loops - for and while loops](#)

[Module 4: Functions and Modules](#)

[Defining Functions](#)

[Using Parameters and Return Values](#)

[Using Arguments and Default Parameters](#)

[Scope of Variables](#)

[Modules and Packages](#)

[Writing and Importing Modules](#)

[Module 5: SQL](#)

[Introduction to SQL](#)

[Types of databases relational vs non-relational](#)

[Installing and setting up database \(sqlite / postgresql / mysql\)](#)

[Basic queries and filtering conditions](#)

[CRUD operations](#)

[Basic queries with aggregate functions](#)

[SQL joins](#)

[Advanced queries](#)

[Module 6: Introduction to Python Automation](#)

[Overview of Automation and Its Benefits](#)

[Introduction to Python Libraries for Automation](#)

[Setting up the development environment](#)

[Module 7: File Handling and Manipulation](#)

[Working with files and directories](#)

[Automating file operations](#)

[File searching and filtering](#)

[Opening Files](#)

[The os and os.path modules](#)

[Reading files](#)

[Writing into a file](#)

[Appending data to a file](#)

[Introduction to web scraping](#)

[Using BeautifulSoup and Requests libraries for web scraping](#)

Module 8: GUI Automation

[Introduction to GUI automation](#)

[Automating mouse and keyboard actions](#)

[Interacting with desktop applications](#)

Module 9: Database Automation

[Introduction to database automation](#)

[Connecting to databases using Python](#)

[Automating database queries and data manipulation tasks](#)

Module 10: Task Scheduling and Automation

[Introduction to task scheduling](#)

[Using cron jobs and task scheduler libraries](#)

[Automating recurring tasks and processes](#)

Module 11: Error Handling and Logging

[Handling errors and exceptions in automation scripts](#)

[Implementing logging for debugging and monitoring](#)

Module 12: Parallel Execution in Python

[Threading](#)

[Multiprocessing for parallel execution](#)

[Implementing automation best practices](#)

[Performance optimization techniques](#)

Module 13: Excel Automation

[Introduction to Excel automation](#)

[Reading and writing Excel files using pandas library](#)

[Handling different Excel formats \(e.g., .xls, .xlsx\)](#)

[Extracting data from Excel worksheets and ranges](#)

[Automating data manipulation and analysis tasks with Excel files](#)

Module 14: PDF Automation

[Reading and extracting data from PDF files using PyPDF2 or pdfplumber library](#)

[Automating PDF generation and manipulation tasks](#)

[Converting PDF files to other formats \(e.g., text, images\)](#)

Module 15: Image Processing with Python

[Automating image manipulation tasks \(e.g., resizing, cropping, filtering\)](#)

[Working with image recognition and OCR \(Optical Character Recognition\) for automation](#)

Module 16: Pandas and DataFrames

[Introduction to NumPy and Arrays](#)

[NumPy Operations and Math](#)

[Introduction to Pandas and DataFrames](#)

[Data Selection and Filtering](#)

[Data Manipulation and Cleaning](#)

[File I/O and Real-World Case Study](#)

Module 17: Plotly and Matplotlib

[Introduction to Matplotlib and Line Charts](#)

[Matplotlib Basic Chart Types](#)

[Matplotlib Subplots and Pandas Integration](#)

[Introduction to Plotly and Basic Charts](#)

[Plotly Histograms, Box Plots, and 3D Charts](#)

[Plotly Graph Objects, Layouts, and Dashboards](#)

Module 18: Building Web Apps with Flask

[Introduction to Flask and Setup](#)

[Routing and Request Handling](#)

[Rendering Templates and Static Files](#)

[Introduction to REST APIs and HTTP Methods](#)

[Integrating CSV Data with Pandas and SQLite](#)

[Building and Running REST APIs](#)

[Testing APIs with Postman](#)

[Module 19: Building Web Apps with Django](#)

[Introduction to Django and Setup](#)

[Designing Views, Routing, and Templates](#)

[CSV Import and Database CRUD](#)

[Building APIs with DRF and APIView](#)

[Pagination, Filtering, and Ordering](#)

[Displaying Graphs in Templates](#)

Module 1: Python Basics

Introduction to Python

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what Python is and why it is widely used for automation
- Install Python and run code in the interactive shell and from a script file
- Write a simple program using `print()`, comments, and correct indentation
- Describe how Python source code is executed (interpreter, bytecode)
- Identify common tools used in automation workflows (terminal, editor, virtual environments)

Introduction

Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. Today it powers web applications, data science, DevOps tooling, and—most relevant to this course—**automation**: repeating tedious tasks such as renaming files, pulling data from APIs, filling spreadsheets, or scheduling scripts to run overnight.

Python reads like English more than many languages. That clarity, combined with a huge standard library and ecosystem (`requests`, `pandas`, `Selenium`, and more), makes it a practical first language for automators who are not full-time software engineers.

In this chapter you will run your first lines of Python and learn the habits—comments, indentation, running scripts—that every later module builds on.

What Is Python?

Characteristic	What it means for you
----------------	-----------------------

Interpreted	Code runs line-by-line through the Python interpreter; no separate compile step before testing
Dynamically typed	You do not declare variable types upfront; types are determined at runtime
Multi-paradigm	Supports procedural, object-oriented, and functional styles
Batteries included	Rich standard library (files, dates, networking, JSON, etc.) without extra installs
Cross-platform	Same scripts run on Windows, macOS, and Linux with minimal changes

Why Python for automation?

- **Fast to write** — A 10-line script can replace an hour of manual clicks.
- **Readable** — Scripts you write today are understandable six months later.
- **Integrations** — Libraries talk to Excel, PDFs, browsers, databases, and cloud APIs.
- **Community** — Solutions to common automation problems are easy to find.

Installing Python

1. Download the latest **Python 3** installer from python.org/downloads.
2. On Windows, check “**Add python.exe to PATH**” during installation.
3. Verify in a terminal (Command Prompt, PowerShell, or Terminal):

```
python3 --version # Example output: Python 3.12.3
```

On some systems the command is `python` instead of `python3`.

Virtual environments (recommended)

For real projects, create an isolated environment so package versions do not clash:

```
cd /path/to/your/project python3 -m venv venv source venv/bin/activate # macOS/Linux
# venv\Scripts\activate # Windows
```

You will use virtual environments again in Module 6 when setting up automation tooling.

Running Python Code

1. Interactive mode (REPL)

Type `python3` (or `python`) with no filename. You get a `>>>` prompt—**Read-Eval-Print Loop**:

```
>>> 2 + 2 4 >>> print("Hello from the REPL") Hello from the REPL
```

Press `Ctrl+D` (macOS/Linux) or `Ctrl+Z` then `Enter` (Windows) to exit.

Use the REPL for quick experiments. For repeatable automation, use **script files**.

2. Script files

Save code in a file ending with `.py`, then run:

```
python3 hello.py
```

Example `hello.py`:

```
# My first automation-ready script print("Automation using Python – Part 1")
print("Module 1: Python Basics")
```

Expected output:

```
Automation using Python – Part 1 Module 1: Python Basics
```

Your First Program

```
# greet.py – demonstrates print and comments # Single-line comment: ignored by
Python, read by humans print("Hello, world!") # print() can show multiple values
separated by spaces name = "Alex" print("Welcome,", name)
```

Output:

```
Hello, World! Welcome, Alex
```

print() essentials

```
print("Line 1") print("Line 2", "Line 3") # default separator is a space print("A",
"B", "C", sep="-") # custom separator: A-B-C print("Loading", end="...") # custom end
(default is newline) print(" done")
```

Output:

```
Line 1 Line 2 Line 3 A-B-C Loading... done
```

Comments and Documentation

Style	Syntax	Use case
Single-line	<code># comment</code>	Explain <i>why</i> a line exists
Multi-line (informal)	Multiple <code>#</code> lines	Short notes
Docstring	<code>"""..."""</code> under a function	Official description (covered in Module 4)

```
# Bad: restates the obvious x = x + 1 # add one to x # Good: explains business logic
retry_count = retry_count + 1 # API allows max 3 retries per run
```

Comments are not executed. Over-commenting clutters code; under-commenting makes automation scripts hard to maintain.

Indentation Matters

Python uses **indentation** (spaces, typically 4 per level) to define blocks—not curly braces `{}` like C or Java.

```
status = "success" if status == "success": print("Job finished OK") # indented block
belongs to if print("Sending notification") else: print("Job failed")
```

Wrong indentation causes `IndentationError`. Never mix tabs and spaces in one file; use spaces only.

How Python Executes Your Code

1. You run `python3 script.py`.
2. The interpreter **reads** the source file.
3. It **compiles** to bytecode (`.pyc` cached internally).
4. The **Python Virtual Machine** executes bytecode.
5. Output appears in the terminal (or logs, files, etc.).

For automation, the important idea is: **change the script** → **run again** → **same steps repeat reliably**.

Tools You Will Use

Tool	Role
Terminal	Run scripts, <code>pip install</code> , cron jobs
Editor / IDE	VS Code, PyCharm, Cursor—syntax highlighting, debugging
Version control (Git)	Track changes to automation scripts
Package manager (<code>pip</code>)	Install third-party libraries

Notes and Best Practices

- Always use **Python 3**; Python 2 is obsolete.
- Name script files with **lowercase** and **underscores**: `backup_files.py`, not `Backup Files.py`.
- Start scripts with a **shebang** only if you need direct execution on Unix: `#!/usr/bin/env python3`.
- Keep scripts in a project folder with a `venv` and a `requirements.txt` (later modules).

- Run small snippets in the REPL; commit working logic to `.py` files.
-

Summary

- Python is an interpreted, readable language with strong support for automation and scripting.
 - Install Python 3, verify with `python3 --version`, and prefer virtual environments for projects.
 - Run code interactively (`>>>`) or via `python3 your_script.py`.
 - Use `print()` for output, `#` for comments, and consistent **4-space indentation** for blocks.
 - Automation success depends on repeatable scripts—not one-off REPL experiments.
-

Practice Exercises

Exercise 1 — Hello, Automator

Write a script `hello_automator.py` that prints exactly three lines:

1. Your name
2. The text: `I am learning Python automation`
3. Today's date as you would write it manually (e.g. `May 21, 2026`)

Run it from the terminal and confirm output.

Exercise 2 — Formatted status report

Using only `print()` (no variables yet if you prefer), display a small “job report” like:

```
=== Daily Backup Report === Status: STARTED Files scanned: 0 Status: COMPLETE
```

Use `sep` or multiple `print()` calls so the header line is visually distinct.

Exercise 3 — REPL exploration

In the interactive shell:

1. Evaluate `10 * 5 + 2`
-

2. Evaluate `"auto" + "mation"`
3. Use `print("Test", end="!")` twice in a row and observe how `end` changes line breaks

Write one sentence in a comment (in a `.py` file) describing what `end=` does.

Exercise 4 — Fix the broken script

The script below has errors. Copy it into `broken.py`, fix it, and run successfully.

```
if True print("Indentation matters") print("This line should run only when True")
```

Exercise Solutions (Check After You Try)

Click to reveal solutions **Exercise 1** (example):

```
print("Alex Morgan") print("I am learning Python automation") print("May 21, 2026")
```

Exercise 2 (example):

```
print("=== Daily Backup Report ===") print("Status:", "STARTED", sep=" ")  
print("Files scanned:", 0) print("Status:", "COMPLETE", sep=" ")
```

Exercise 3: `"automation"` is printed; first `print` with `end="!"` does not add a newline, so the next output continues on the same line unless you reset with default `end="\n"`. **Exercise 4:**

```
if True: print("Indentation matters") print("This line should run only when True")
```

Further Reading

- [Python documentation — Tutorial](#)
- [Python documentation — Using Python on your platform](#)
- PEP 8 — [Style Guide for Python Code](#) (naming and formatting conventions)

Data Types

Learning Objectives

By the end of this chapter, you will be able to:

- Name and use Python's core built-in types: `int`, `float`, `str`, `bool`, and `None`
- Inspect types with `type()` and `isinstance()`
- Convert between types safely for automation tasks (parsing numbers from text, etc.)
- Predict the result of mixed-type operations
- Choose appropriate types when representing real-world automation data

Introduction

Every value in Python has a **data type**. The type tells the interpreter what operations are valid: you can divide two numbers, but dividing two sentences is meaningless.

Automation scripts constantly juggle types: file paths are **strings**, retry counts are **integers**, success flags are **booleans**, and missing configuration might be **None**. Understanding types prevents subtle bugs—especially when reading CSV rows, API JSON, or user input.

Built-in Types at a Glance

Type	Example	Typical automation use
<code>int</code>	42, -7, 0	Counts, IDs, exit codes
<code>float</code>	3.14, -0.5	Percentages, timings, coordinates
<code>str</code>	"hello", 'C:\\logs'	Paths, messages, JSON text

<code>bool</code>	True, False	Feature flags, success/failure
<code>NoneType</code>	<code>None</code>	“No value yet” / optional setting

Python also has collection types (`list`, `dict`, etc.) covered in the **Data Structure** chapter.

Integers (`int`)

Whole numbers with unlimited size (only memory limits apply).

```
files_processed = 128 exit_code = 0 year = 2026 # Underscores improve readability
(PEP 515) batch_size = 10_000 print(batch_size) # 10000
```

Arithmetic

```
print(10 + 3) # 13 print(10 - 3) # 7 print(10 * 3) # 30 print(10 / 3) # 3.333...
(always float division) print(10 // 3) # 3 (floor division) print(10 % 3) # 1
(remainder) print(2 ** 8) # 256 (power)
```

Floating-Point Numbers (`float`)

Approximate real numbers—fine for metrics, risky for exact money without `decimal`.

```
cpu_usage = 87.5 duration_seconds = 0.1 + 0.2 print(duration_seconds) #
0.30000000000000004 (binary float quirk) # Round for display
print(round(duration_seconds, 2)) # 0.3
```

Strings (`str`)

Immutable sequences of Unicode characters. Single or double quotes are equivalent.

```
path = "C:/data/report.csv" message = 'Process finished' # Triple quotes for
multiline templates email_body = """Hello, Your nightly backup completed
successfully. """
```

Escape sequences

```
print("Line one\nLine two") # newline print("Tab\tthere") # tab print("Quote:
\"inside\") # escaped quote print(r"C:\new\folder") # raw string – backslashes
literal
```

Booleans (bool)

Only two values: `True` and `False`. Subclass of `int` (`True == 1`, `False == 0`), but use them for logic—not math.

```
job_ok = True retry_enabled = False print(job_ok and retry_enabled) # False
print(job_ok or retry_enabled) # True print(not job_ok) # False
```

Truthiness

Many values act as “true” or “false” in conditions:

Falsy	Truthy (examples)
<code>False</code> , <code>None</code> , <code>0</code> , <code>0.0</code> , <code>""</code> , <code>[]</code> , <code>{}</code>	Non-zero numbers, non-empty strings/collections

```
name = "" if name: print("Has name") else: print("Name missing") # runs – empty
string is falsy
```

None — Absence of Value

`None` is a singleton meaning “no value assigned.”

```
result = None if result is None: print("No result from API yet")
```

Use `is None` / `is not None`—not `== None` (style and correctness with custom objects).

Checking Types

```
value = "100" print(type(value)) # <class 'str'> print(type(value) == str) # True #
isinstance() is preferred (handles inheritance) print(isinstance(value, str)) # True
print(isinstance(100, (int, float))) # True if numeric
```

In automation, log types when debugging bad API data:

```
payload = {"count": "42"} # string, not int! print(type(payload["count"]))
```

Type Conversion (Casting)

Convert explicitly when reading files or user input:

```
age_text = "30" age = int(age_text) print(age + 1) # 31 price = float("19.99") count
= int(3.9) # truncates toward zero → 3 label = str(404) # "404" flag = bool(1) #
True flag = bool(0) # False flag = bool("") # False
```

Safe conversion pattern

```
raw = "not_a_number" try: num = int(raw) except ValueError: num = 0 print(f"Invalid
integer '{raw}', using default {num}")
```

You will formalize this with `try/except` in Module 11.

Mixed-Type Operations

```
print(3 + 5) # 8 (int + int) print(3 + 5.0) # 8.0 (int + float → float) # print("3"
+ 5) # TypeError – must convert print("3" + str(5)) # "35" (string concatenation)
print("repeat " * 3) # "repeat repeat repeat "
```

Type Hints (Preview)

Optional annotations document intent (Module 4 goes deeper):

```
def process_rows(count: int, path: str) -> bool: ...
```

They do not change runtime behavior in standard Python but help editors and teammates.

Notes and Best Practices

- Prefer `isinstance(x, int)` over `type(x) == int` when checking types.
- JSON numbers often arrive as `int` or `float`; JSON `null` becomes `None`.
- File paths on Windows may contain backslashes—use raw strings `r"..."` or forward slashes.
- Do not use `float` for currency; use `decimal.Decimal` when precision matters.
- Name booleans clearly: `is_active`, `has_error`, `should_retry`.

Summary

- Core scalar types: `int`, `float`, `str`, `bool`, and `None`.
 - Use `type()` and `isinstance()` to inspect values when debugging pipelines.
 - **Cast** with `int()`, `float()`, `str()`, `bool()` when moving between text and numbers.
 - `/` always returns a float; use `//` for integer division.
 - Empty or zero values are often **falsey**—useful in automation conditionals.
-

Practice Exercises

Exercise 1 — Inventory counters

Create variables for:

- `items_in_stock` (integer): 150
- `unit_price` (float): 12.5
- `product_name` (string): "USB-C Hub"

Print the **total inventory value** (`items_in_stock * unit_price`) rounded to 2 decimal places using `round()`.

Exercise 2 — Parse configuration strings

Simulate reading config from a text file (values are strings):

```
max_retries = "3" timeout_sec = "2.5" debug = "false"
```

Convert each to the correct type (`int`, `float`, `bool`). For `bool`, treat `"true"` (case-insensitive) as `True`, anything else as `False`.

Print all three converted values and their types.

Exercise 3 — Truthiness audit

Without running `if` blocks mentally only—then verify in Python—predict whether each prints “yes” or “no”:

```
def check(label, value): if value: print(label, "yes") else: print(label, "no")
check("A", 0) check("B", "0") check("C", []) check("D", None) check("E", "False")
```

Write one paragraph explaining why "False" is truthy.

Exercise 4 — Automation log line

Build a single string `log_line` from:

- `timestamp = "2026-05-21T08:00:00"`
- `level = "INFO"`
- `message = "Copied 12 files"`

Format: `[INFO] 2026-05-21T08:00:00 — Copied 12 files` using concatenation or an f-string (introduced fully in Module 2).

Exercise Solutions

Click to reveal solutions **Exercise 1:**

```
items_in_stock = 150 unit_price = 12.5 product_name = "USB-C Hub" total =
items_in_stock * unit_price print(round(total, 2)) # 1875.0
```

Exercise 2:

```
max_retries = "3" timeout_sec = "2.5" debug = "false" max_retries = int(max_retries)
timeout_sec = float(timeout_sec) debug = debug.lower() == "true" print(max_retries,
type(max_retries)) print(timeout_sec, type(timeout_sec)) print(debug, type(debug))
```

Exercise 3: ``B`` and ``E`` print "yes" because non-empty strings are truthy—even ``0`` and ``False``. **Exercise 4:**

```
timestamp = "2026-05-21T08:00:00" level = "INFO" message = "Copied 12 files" log_line
= f"[{level}] {timestamp} — {message}" print(log_line)
```

Further Reading

- [Built-in Types](#)
- [PEP 257 — Docstring Conventions](#)

Variables

Learning Objectives

By the end of this chapter, you will be able to:

- Create variables and assign values using `=`
 - Follow Python naming rules and PEP 8 conventions
 - Reassign variables and understand that names refer to objects
 - Use multiple assignment and swapping idioms
 - Distinguish between variables and constants by convention
 - Explain dynamic typing in practical automation scenarios
-

Introduction

A **variable** is a name that refers to a value stored in memory. In automation scripts, variables hold file paths, counters, API tokens (from environment variables, not hard-coded secrets), and status messages.

Unlike some languages, Python does not require `int x = 5`;—you simply assign and Python infers the type dynamically.

Creating Variables

Assignment uses a single equals sign `=`. It is **not** a mathematical equality test.

```
host = "localhost" port = 8080 is_running = True
```

Read assignment as: “**Attach the name on the left to the object on the right.**”

```
x = 10 x = 20 # name x now refers to 20; 10 may be garbage-collected if unused x = x + 5 # 25 print(x)
```

Naming Rules and Conventions

Hard rules (syntax)

Rule	Valid	Invalid
Letters, digits, underscores	<code>retry_count</code>	<code>2fast</code> (starts with digit)
Case-sensitive	<code>Count</code> $\neq$ <code>count</code>	—
No reserved keywords	<code>status</code>	<code>class</code> , <code>for</code> , <code>if</code>

```
# SyntaxError examples (do not run as a block): # class = "economy" # my-var = 1
```

PEP 8 conventions (style)

Style	Example	Use for
<code>snake_case</code>	<code>max_retries</code> , <code>log_file</code>	Variables, functions
<code>UPPER_SNAKE</code>	<code>MAX_SIZE</code> , <code>DEFAULT_PATH</code>	Module-level constants
Descriptive names	<code>elapsed_seconds</code>	Not es or t

```
# Constants by convention (still reassignable unless you enforce otherwise)
MAX_RETRIES = 3
DEFAULT_OUTPUT_DIR = "/var/automation/output"
```

Dynamic Typing

The same name can refer to different types over time:

```
status = 200 # int – HTTP OK code
status = "success" # str – human-readable
status = True # bool – simplified flag
```

For maintainability in production automation, **avoid reusing one name for unrelated meanings**. Prefer separate names: `http_code`, `status_message`, `job_succeeded`.

Multiple Assignment

```
# Same value to several names x = y = z = 0 # Unpack a sequence into names width,
height = 1920, 1080 print(width, height) # Swap without a temporary variable a, b =
10, 20 a, b = b, a print(a, b) # 20 10
```

Unpacking is common when parsing CSV rows or `split()` results:

```
line = "error,Connection timeout,3" level, message, code = line.split(",")
print(level, message, code)
```

Variable Scope (Preview)

Inside a function, assignments create **local** names unless declared `global` (rare in good automation code):

```
retries = 1 # module-level def run_job(): retries = 3 # local – does not change
module-level retries print("inside", retries) run_job() print("outside", retries) #
still 1
```

Module 4 covers scope in depth.

Deleting Names

```
temp_data = [1, 2, 3] del temp_data # print(temp_data) # NameError
```

Use `del` sparingly; usually reassignment or letting a function end is enough.

Variables and Objects

Python variables are **references** (labels on objects):

```
list_a = [1, 2, 3] list_b = list_a # both names refer to the SAME list
list_b.append(4) print(list_a) # [1, 2, 3, 4] # Copy if you need independence list_c
= list_a.copy() list_c.append(99) print(list_a) # unchanged by list_c
```

This matters when passing lists or dicts into functions that mutate them.

Environment Variables in Automation

Store secrets in the environment, not in source code:

```
import os api_key = os.environ.get("API_KEY") if not api_key: raise SystemExit("Set API_KEY before running this script")
```

You will use `os` heavily in Module 7.

Notes and Best Practices

- Initialize counters and flags before loops: `processed = 0, errors = []`.
- Use `UPPER_SNAKE` for values that should not change during a run.
- Never name variables `list`, `dict`, `str`—they shadow built-ins.
- Prefer `snake_case` over `camelCase` in Python projects.
- Group related settings in a dict or config file (later modules).

Summary

- Variables are created with `name = value`; types are inferred dynamically.
- Follow `snake_case` for variables and `UPPER_SNAKE` for constants.
- **Unpacking** (`a, b = 1, 2`) simplifies working with pairs and split strings.
- Names refer to **objects**; aliasing (`b = a`) shares mutable data unless you **copy**.
- Use environment variables for secrets in real automation deployments.

Practice Exercises

Exercise 1 — Batch rename counter

Simulate renaming files `photo_001.jpg ... photo_005.jpg`:

```
prefix = "photo" extension = ".jpg" start = 1 count = 5
```

Use a loop (or manual prints) to build and print each filename. Store the **last filename** in `last_file` and print it after the loop.

Exercise 2 — Swap and unpack

Start with `source = "archive"`, `destination = "live"`. Swap them using tuple unpacking. Print both names to prove the swap.

Then unpack `metrics = "98.5,42,1200"` into `cpu`, `memory`, `disk` (as strings first, then convert `cpu` to float).

Exercise 3 — Naming audit

Which names violate PEP 8 or shadow built-ins? Rewrite them.

```
List = ["a", "b"] fileName = "report.csv" MAX_retries = 5
```

Exercise 4 — Shared reference bug

Predict the output, then run:

```
defaults = {"retries": 1} run_a = defaults run_b = defaults run_b["retries"] = 5
print(run_a["retries"])
```

Fix the pattern so `run_a` and `run_b` have independent dicts with the same initial values.

Exercise Solutions

Click to reveal solutions **Exercise 1** (loop version):

```
prefix = "photo" extension = ".jpg" start = 1 count = 5 last_file = None
for i in range(start, start + count): last_file = f"{prefix}_{i:03d}{extension}"
print(last_file) print("Last:", last_file)
```

Exercise 2:

```
source = "archive" destination = "live" source, destination = destination, source
metrics = "98.5,42,1200" cpu, memory, disk = metrics.split(",") cpu = float(cpu)
```

Exercise 3: Use `file_list`, `file_name`, `MAX_RETRIES`. **Exercise 4:** Use `.copy()`:

```
run_a = defaults.copy() run_b = defaults.copy() run_b["retries"] = 5
```

Further Reading

- [PEP 8 — Naming conventions](#)
- [Python assignment statements](#)

Data Structure

Learning Objectives

By the end of this chapter, you will be able to:

- Describe lists, tuples, dictionaries, and sets and when to use each
- Create and access elements in each structure
- Explain mutability vs immutability
- Model simple automation data (records, unique tags, ordered queues) with the right structure
- Combine structures (e.g., list of dicts) for realistic datasets

Introduction

Data structures organize multiple values. Automation rarely deals with a single number—you work with **lists of files**, **dictionaries of settings**, **unique error codes**, or **fixed coordinate pairs**.

Python's four essential built-in collections are:

Structure	Ordered	Mutable	Duplicates	Syntax
list	Yes	Yes	Allowed	<code>[1, 2, 3]</code>
tuple	Yes	No	Allowed	<code>(1, 2, 3)</code>
dict	Yes*	Yes (keys unique)	Keys unique	<code>{"a": 1}</code>
set	No	Yes	No	<code>{1, 2, 3}</code>

*Python 3.7+ preserves insertion order in dicts.

Lists

Ordered, mutable sequences—your default “collection of items.”

```
files = ["report.csv", "summary.csv", "archive.zip"] files.append("notes.txt")
files[0] = "report_v2.csv" print(len(files)) # 4 print(files[-1]) # last item:
notes.txt
```

Creating lists

```
empty = [] numbers = list(range(5)) # [0, 1, 2, 3, 4] same = [0] * 3 # [0, 0, 0] –
careful with mutable items
```

Nested lists

```
matrix = [[1, 2], [3, 4]] print(matrix[1][0]) # 3
```

Tuples

Like lists but **immutable**—good for fixed records you should not accidentally change.

```
point = (10, 20) rgb = (255, 128, 0) # Tuple unpacking x, y = point print(x, y) #
Single-element tuple needs a comma singleton = (42,)
```

Use tuples for:

- Coordinates, date parts that stay together
- Returning multiple values from a function
- Dictionary keys (lists cannot be keys)

```
config = { ("prod", "db"): "postgres://...", ("dev", "db"): "sqlite:///dev.db", }
```

Dictionaries

Key–value mappings—ideal for JSON-like records and configuration.

```
job = { "id": "job-1042", "status": "running", "retries": 0, } job["status"] =
"completed" job["duration_sec"] = 45.2 print(job.get("owner", "system")) # default if
key missing
```

Keys must be hashable

Strings, numbers, tuples (of immutables) work; lists do not.

```
# invalid = {[1, 2]: "value"} # TypeError
```

Iterating

```
for key in job: print(key, job[key]) for key, value in job.items(): print(f"{key} => {value}")
```

Sets

Unordered collections of **unique** items—fast membership tests and deduplication.

```
tags = {"urgent", "billing", "urgent"} # duplicate dropped print(tags) # {'urgent', 'billing'} tags.add("ops") print("urgent" in tags) # True failed_ids = set([101, 102, 101, 103]) print(failed_ids) # {101, 102, 103}
```

Set operations

```
a = {1, 2, 3} b = {3, 4, 5} print(a | b) # union: {1, 2, 3, 4, 5} print(a & b) # intersection: {3} print(a - b) # difference: {1, 2}
```

Strings as Sequences

Strings behave like ordered sequences of characters (but are immutable):

```
path = "logs/app.log" print(path[0]) # l print(path[-3:]) # log (slicing – see next chapter)
```

Choosing the Right Structure

Scenario	Structure
Ordered queue of files to process	<code>list</code>
Fixed API version + host pair	<code>tuple</code>

User profile by field name	<code>dict</code>
Unique visitor IDs seen today	<code>set</code>
Tabular data (many rows)	<code>list of dict</code> (or pandas later)

Example: list of dicts (common in automation)

```
runs = [ {"id": 1, "ok": True, "files": 12}, {"id": 2, "ok": False, "files": 0},
{"id": 3, "ok": True, "files": 7}, ] success_count = sum(1 for r in runs if r["ok"])
print(f"Successful runs: {success_count}")
```

Mutability Summary

```
# Mutable – in-place changes affect all references row = [1, 2, 3] alias = row
alias.append(4) print(row) # [1, 2, 3, 4] # Immutable – “change” creates new object t
= (1, 2) # t.append(3) # AttributeError
```

Notes and Best Practices

- Use `dict.get(key, default)` instead of risking `KeyError`.
- Do not use a list as a default argument in functions (Module 4 pitfall)—use `None` and create inside.
- For config, prefer dicts or dedicated config files over dozens of separate variables.
- Serialize dicts/lists to JSON for APIs and logs (`json.dumps` in later modules).
- Sets are ideal for “already processed” IDs to avoid duplicate work.

Summary

- **Lists** — ordered, mutable sequences for workflows and queues.
- **Tuples** — fixed, immutable records; useful for unpacking and dict keys.
- **Dicts** — labeled fields and configuration; mirror JSON objects.
- **Sets** — unique members and fast “seen before” tracking.

- Real automation data often uses **nested structures**, especially lists of dictionaries.

Practice Exercises

Exercise 1 — File processing queue

Create a list `queue` with paths:

```
"/data/in/a.csv", "/data/in/b.csv", "/data/in/c.csv"
```

Pop the first item (use `pop(0)` or slicing), append `"/data/in/d.csv"`, and print the final queue.

Exercise 2 — Server record

Create a dict `server` with keys: `hostname`, `port`, `ssl`, `tags` (where `tags` is a set of at least two strings). Add a new tag `"monitored"` and change `port` to `443`.

Print each key-value pair on its own line.

Exercise 3 — Deduplicate event IDs

Given:

```
events = [1001, 1002, 1001, 1003, 1002, 1004]
```

Build a **set** `unique_events` and a **sorted list** `sorted_events` from it. Print both.

Exercise 4 — Mini report table

Build `reports` as a list of three dicts with keys `name`, `size_kb`, `ok` (bool). Write a loop that prints only reports where `ok` is `True`, one per line:

```
name: size_kb KB
```

Exercise Solutions

Click to reveal solutions ****Exercise 1:****

```
queue = ["/data/in/a.csv", "/data/in/b.csv", "/data/in/c.csv"] queue.pop(0)
queue.append("/data/in/d.csv") print(queue)
```

****Exercise 2:****

```
server = { "hostname": "api.internal", "port": 80, "ssl": False, "tags": {"prod",
"east"}, } server["tags"].add("monitored") server["port"] = 443 for k, v in
server.items(): print(k, v)
```

****Exercise 3:****

```
events = [1001, 1002, 1001, 1003, 1002, 1004] unique_events = set(events)
sorted_events = sorted(unique_events)
```

****Exercise 4:**** Example data and loop:

```
reports = [ {"name": "a.csv", "size_kb": 10, "ok": True}, {"name": "b.csv",
"size_kb": 0, "ok": False}, {"name": "c.csv", "size_kb": 5, "ok": True}, ] for r in
reports: if r["ok"]: print(f"{r['name']}: {r['size_kb']} KB")
```

Further Reading

- [Built-in Types — Data structures](#)
- [json — JSON encoder and decoder](#)

Operations on Data Structure

Learning Objectives

By the end of this chapter, you will be able to:

- Index and slice sequences (strings, lists, tuples)
- Use positive and negative indices correctly
- Concatenate and repeat sequences with `+` and `*`
- Test membership with `in` and `not in`
- Apply common operators on lists and dicts (`+=`, slicing assignment, merging dicts)
- Avoid off-by-one errors when extracting substrings and sublists

Introduction

Once data lives in lists, strings, or dicts, automation scripts **select**, **combine**, and **filter** it: take the file extension from a path, grab the first CSV row, or split a log line into fields.

This chapter covers the operations that apply across sequences and the most useful patterns for everyday scripting.

Indexing

Positions start at **0** for the first element.

```
items = ["alpha", "beta", "gamma"] print(items[0]) # alpha print(items[1]) # beta
print(items[2]) # gamma # print(items[3]) # IndexError – out of range
```

Negative indices

Count from the end: **-1** is the last item.

```
print(items[-1]) # gamma print(items[-2]) # beta
```

Index	-3	-2	-1
-------	----	----	----

Value	alpha	beta	gamma
-------	-------	------	-------

Slicing

Syntax: `sequence[start:stop:step]`

- **start** — included (default 0)
- **stop** — excluded (default end)
- **step** — stride (default 1)

```
text = "automation" print(text[0:4]) # auto print(text[:4]) # auto (from beginning)
print(text[4:]) # mation (through end) print(text[:]) # full copy of string
print(text[::2]) # atoain - every 2nd character print(text[::-1]) # noitamotua -
reversed
```

Slicing lists (returns a new list)

```
nums = [10, 20, 30, 40, 50] middle = nums[1:4] # [20, 30, 40] last_two = nums[-2:] #
[40, 50]
```

Assignment to slices (lists only)

```
data = [1, 2, 3, 4, 5] data[1:3] = [99, 88] print(data) # [1, 99, 88, 4, 5] data[:] =
[] # clear in place print(data) # []
```

Strings and tuples do **not** support item assignment (immutable).

Concatenation and Repetition

Operator	Meaning	Example
+	Concatenate	<code>"log/" + "app.txt"</code>
*	Repeat	<code>" - " * 40</code>

```
header = "ID" + "," + "Name" + "," + "Status" print(header) separator = "=" * 50
print(separator) paths = ["/in/"] * 3 # three references to SAME list – usually
unintended paths = ["/in/"] + ["/tmp/"] + ["/out/"] # better
```

Building paths safely

For real path work use `pathlib` (Module 7). For learning:

```
base = "/var/log" name = "app.log" full = base + "/" + name print(full)
```

Membership: `in` and `not in`

Works on strings, lists, tuples, sets, and dict keys.

```
allowed = {"csv", "json", "xml"} filename = "report.CSV" ext =
filename.split(".")[-1].lower() if ext in allowed: print("Extension OK") else:
print("Unsupported type")
```

```
message = "ERROR: disk full" if "ERROR" in message: print("Alert team")
```

```
user = {"id": 42, "name": "Sam"} print("email" in user) # False – key check
print("id" in user) # True
```

Comparing Sequences

```
print([1, 2] == [1, 2]) # True print([1, 2] < [1, 3]) # True – lexicographic order
print("ab" < "ac") # True
```

List Operations

```
a = [1, 2] b = [3, 4] combined = a + b # [1, 2, 3, 4] a += [5] # in-place: a is [1,
2, 5] repeated = [0] * 4 # [0, 0, 0, 0] print(len(a)) print(2 in a)
```

Shallow copy vs reference

```
original = [1, [2, 3]] shallow = original[:] shallow[0] = 99 shallow[1].append(4)
print(original) # [1, [2, 3, 4]] – nested list shared
```

Use `copy.deepcopy()` when nesting matters (stdlib `copy` module).

Dictionary Operations

```
defaults = {"retries": 1, "timeout": 30} overrides = {"timeout": 60} # Merge (Python 3.9+)
config = defaults | overrides print(config) # {'retries': 1, 'timeout': 60} #
Update in place defaults.update(overrides)
```

```
keys = list(config.keys()) values = list(config.values()) pairs =
list(config.items())
```

Set Operations (Recap)

```
seen = set() for job_id in [1, 2, 1, 3]: if job_id not in seen: print("Processing",
job_id) seen.add(job_id)
```

Practical Automation Examples

Extract file extension

```
path = "/archive/report.final.csv" parts = path.split(".") ext = parts[-1] if
len(parts) > 1 else "" print(ext) # csv
```

First and last log line

```
lines = ["start", "processing", "done"] first = lines[0] last = lines[-1]
```

Batch chunks (simple)

```
ids = list(range(10)) batch_size = 3 for i in range(0, len(ids), batch_size): batch =
ids[i : i + batch_size] print("Batch:", batch)
```

Common Pitfalls

Pitfall	Problem	Fix
Off-by-one	<code>text[0:len(text)]</code> vs last index	Remember stop is exclusive
Empty slice	<code>text[5:5]</code> → ""	Check lengths first

<code>[:-0]</code>	Returns empty (not full string)	Use <code>[:]</code> for full copy
Shared nested lists	Aliasing inner lists	<code>deepcopy</code> when needed
Case sensitivity	<code>"CSV" in {"csv"} is False</code>	Normalize with <code>.lower()</code>

Summary

- **Indexing** targets one element; **negative** indices count from the end.
- **Slicing** `start:stop:step` extracts sub-sequences; stop is **exclusive**.
- `+` and `*` concatenate and repeat; use carefully with mutable lists.
- `in` tests membership—essential for validation and filtering.
- **Dict merge** (`|`) and **update** combine configuration layers in automation scripts.

Practice Exercises

Exercise 1 — Path parsing

Given:

```
path = "/opt/automation/jobs/nightly/run_2026-05-21.log"
```

Using only slicing and `split` (no `pathlib`):

1. Print the **filename** (`run_2026-05-21.log`)
2. Print the **date portion** `2026-05-21` from the filename (hint: split on `_` and `.`)

Exercise 2 — Slice the queue

```
queue = ["a", "b", "c", "d", "e", "f"]
```

Without modifying the original list, create:

- `first_three` — first 3 items
- `last_two` — last 2 items
- `reversed_queue` — reversed order
- `every_other` — items at indices 0, 2, 4

Print all four.

Exercise 3 — Merge configs

```
base = {"env": "prod", "retries": 1, "notify": True} patch = {"retries": 3,
"timeout": 120}
```

Produce `effective` where patch wins on conflicts. Print `effective`. Then check if `"timeout"` in `effective`.

Exercise 4 — Filter extensions

```
files = ["a.csv", "b.txt", "c.csv", "d.JSON", "e.csv"] allowed = {"csv", "json"}
```

Build a new list `selected` containing only files whose extension (lowercase, after the last `.`) is in `allowed`. Print `selected`.

Expected: `['a.csv', 'c.csv', 'd.JSON']` if you lowercase extension before compare (`d.json` matches `json`).

Exercise Solutions

Click to reveal solutions ****Exercise 1:****

```
path = "/opt/automation/jobs/nightly/run_2026-05-21.log" filename =
path.split("/")[-1] print(filename) # run_2026-05-21 from run_2026-05-21.log middle =
filename.split("_", 1)[1] # 2026-05-21.log date_part = middle.split(".")[0]
print(date_part)
```

****Exercise 2:****

```
queue = ["a", "b", "c", "d", "e", "f"] first_three = queue[:3] last_two = queue[-2:]
reversed_queue = queue[::-1] every_other = queue[::2]
```

****Exercise 3:****

```
base = {"env": "prod", "retries": 1, "notify": True} patch = {"retries": 3,
"timeout": 120} effective = base | patch print("timeout" in effective) # True
```

```
**Exercise 4:**
```

```
files = ["a.csv", "b.txt", "c.csv", "d.JSON", "e.csv"] allowed = {"csv", "json"}
selected = [] for f in files: ext = f.split(".")[-1].lower() if ext in allowed:
selected.append(f) print(selected)
```

Further Reading

- [Common Sequence Operations](#)
- [pathlib](#) — preferred for paths in automation

Inbuilt Methods

Learning Objectives

By the end of this chapter, you will be able to:

- Use built-in functions such as `len()`, `min()`, `max()`, `sum()`, and `sorted()`
- Call common string methods for cleaning and parsing automation data
- Mutate lists with `append()`, `extend()`, `insert()`, `pop()`, and `remove()`
- Discover method documentation with `help()` and `dir()`
- Choose between in-place methods and functions that return new values

Introduction

Python objects expose **methods**—functions attached to a value: `" hi ".strip()`, `[1, 2].append(3)`.

The standard library also provides **built-in functions** that work on many types: `len(items)`, `sorted(names)`.

Automation scripts rely on these daily to normalize text, count rows, sort filenames, and manage in-memory queues before writing to disk or APIs.

Built-in Functions

Function	Purpose	Example
<code>len(x)</code>	Length / item count	<code>len("abc") → 3</code>
<code>min(x)</code>	Smallest item	<code>min([3, 1, 2]) → 1</code>
<code>max(x)</code>	Largest item	<code>max([3, 1, 2]) → 3</code>

<code>sum(iterable)</code>	Sum of numbers	<code>sum([10, 20]) → 30</code>
<code>sorted(x)</code>	New sorted list	<code>sorted("cba") → ['a', 'b', 'c']</code>
<code>reversed(x)</code>	Iterator over reversed sequence	<code>list(reversed([1,2])) → [2,1]</code>
<code>any(iterable)</code>	True if any truthy	<code>any([0, "", 5]) → True</code>
<code>all(iterable)</code>	True if all truthy	<code>all([1, 2, 3]) → True</code>
<code>enumerate(x)</code>	Index + value pairs	<code>for i, v in enumerate(["a","b"]):</code>
<code>zip(a, b)</code>	Pair elements	<code>list(zip([1,2], "ab"))</code>

```
sizes = [1024, 512, 2048, 256] print("Files:", len(sizes)) print("Largest:",
max(sizes)) print("Total bytes:", sum(sizes)) print("Ascending:", sorted(sizes))
```

sorted() VS .sort()

```
nums = [3, 1, 2] ordered = sorted(nums) # new list; nums unchanged print(nums,
ordered) nums.sort() # in-place; returns None print(nums) # [1, 2, 3]
```

Use `sorted()` when you need to keep the original order elsewhere.

String Methods (Essential for Automation)

Strings are immutable—methods return **new** strings.

Method	What it does
--------	--------------

<code>.lower() / .upper()</code>	Case conversion
<code>.strip() / .lstrip() / .rstrip()</code>	Remove whitespace (or chars)
<code>.replace(old, new)</code>	Substring replacement
<code>.split(sep)</code>	Break into list
<code>.join(iterable)</code>	Combine strings
<code>.startswith() / .endswith()</code>	Prefix/suffix test
<code>.find(sub)</code>	Index of substring (-1 if missing)
<code>.count(sub)</code>	Occurrence count

```
raw = " ERROR: Disk Full \n" clean = raw.strip().lower().replace(" ", " ")
print(clean) # error: disk full csv_line = "id,name,status" fields =
csv_line.split(",") print(fields) # ['id', 'name', 'status'] header = ",".join(["id",
"name", "status"]) print(header)
```

Chaining

```
email = " User@Example.COM " normalized = email.strip().lower() print(normalized) #
user@example.com
```

Checking content

```
filename = "backup.tar.gz" print(filename.endswith((".tar.gz", ".zip")))
print(filename.startswith("backup"))
```

List Methods

Method	Effect
<code>.append(x)</code>	Add one item to end
<code>.extend(iterable)</code>	Add all items from iterable
<code>.insert(i, x)</code>	Insert at index
<code>.pop()</code> / <code>.pop(i)</code>	Remove and return item
<code>.remove(x)</code>	Remove first matching value
<code>.clear()</code>	Remove all
<code>.index(x)</code>	First index of value
<code>.count(x)</code>	Count occurrences
<code>.reverse()</code>	In-place reverse
<code>.copy()</code>	Shallow copy

```
pending = [] pending.append("job-1") pending.extend(["job-2", "job-3"])
print(pending) done = pending.pop(0) print("Finished:", done) print("Remaining:",
pending)
```

append VS extend

```
a = [1, 2] a.append([3, 4]) # [1, 2, [3, 4]] – nested list b = [1, 2] b.extend([3,
4]) # [1, 2, 3, 4]
```

Dictionary Methods

```
settings = {"retries": 1, "timeout": 30} print(settings.keys())
print(settings.values()) print(settings.items()) settings.setdefault("retries", 5) #
set only if missing settings.setdefault("log_level", "INFO") removed =
settings.pop("timeout", None) print(removed, settings)
```

Set Methods

```
a = {1, 2, 3} b = {3, 4} a.update(b) # in-place union print(a) a.discard(99) # no
error if missing a.remove(1) # KeyError if missing
```

Discovering Methods: `help()` and `dir()`

```
help(str.split) print([m for m in dir(list) if not m.startswith("_")])
```

In the REPL, `help(list.append)` shows signature and docstring—useful when you forget parameter order.

Practical Example: Normalize a Log File

```
lines = [ " INFO: Started ", "ERROR: Timeout ", " info: done ", ] normalized = [] for
line in lines: text = line.strip().upper() if text.startswith("ERROR"):
normalized.append(text + " [ALERT]") else: normalized.append(text) for entry in
normalized: print(entry)
```

Method Chaining vs Readability

Prefer clarity over long chains:

```
# Dense # x = raw.strip().split(":")[1].lower().replace(" ", "_") # Clearer for
maintenance part = raw.strip().split(":", 1)[1] x = part.lower().replace(" ", "_")
```

Notes and Best Practices

- Methods that mutate (`list.sort`, `list.append`) return **None**—do not assign their result.
- Normalize user and file input with `.strip()` before validation.

- Use `.join()` to build delimited text—not repeated `+` in loops (efficiency and clarity).
- Prefer `in / startswith` over manual slicing when checking prefixes.
- Read library docs for timezone-aware dates, regex (`re` module), and paths (`pathlib`) in later modules.

Summary

- **Built-ins** (`len`, `min`, `max`, `sum`, `sorted`) work across types and keep code concise.
- **String methods** clean and parse text from logs, CSV, and APIs.
- **List methods** manage work queues; know **`append` vs `extend`**.
- **Dict/set methods** support defaults, safe removal, and set algebra.
- Use `help()` and `dir()` to explore unfamiliar objects interactively.

Practice Exercises

Exercise 1 — File stats

```
files = ["report.csv", "data.json", "image.png", "backup.csv"] sizes = [1200, 800, 5000, 1200]
```

Print:

1. Number of files
2. Largest size
3. Total size
4. Filenames sorted alphabetically (use `sorted()` without changing `files`)

Exercise 2 — Clean usernames

```
raw_users = [" Alice ", "BOB", " charlie", "bob", ""]
```

Produce `clean_users`:

- Strip whitespace
- Lowercase

- Skip empty strings
- Remove duplicates while preserving first-seen order (hint: loop + list, or dict.fromkeys)

Print `clean_users`. Expected order: alice, bob, charlie.

Exercise 3 — Work queue simulation

Start with `queue = ["email", "backup", "report"]`.

1. `append "cleanup"`
2. `pop` from the front until one item remains
3. Print each popped job and the final queue

Exercise 4 — Method discovery

Pick any string method from `dir(str)` that we did not cover in this chapter. In a short comment, describe what it does and demonstrate it on `"automation"`.

Exercise Solutions

Click to reveal solutions ****Exercise 1:****

```
files = ["report.csv", "data.json", "image.png", "backup.csv"] sizes = [1200, 800, 5000, 1200]
print(len(files)) print(max(sizes)) print(sum(sizes))
print(sorted(files))
```

****Exercise 2:****

```
raw_users = [" Alice ", "BOB", " charlie", "bob", ""] seen = set() clean_users = []
for u in raw_users: name = u.strip().lower() if not name or name in seen: continue
seen.add(name) clean_users.append(name) print(clean_users)
```

Alternative: ``list(dict.fromkeys(...))`` after filtering. ****Exercise 3:****

```
queue = ["email", "backup", "report"] queue.append("cleanup") while len(queue) > 1:
job = queue.pop(0) print("Popped:", job) print("Final queue:", queue)
```

****Exercise 4**** (example with `.title()`):

```
# .title() – title-cases words print("automation pipeline".title()) # Automation Pipeline
```

Further Reading

- [Built-in Functions](#)
- [Text Sequence Methods](#)
- [Mutable Sequence Types \(list methods\)](#)

Module 2: Data Types and Variables

Data Types

Learning Objectives

- Understand core Python data types
- Convert values safely
- Use the right type for insurance workflows

Introduction

Insurance automation scripts process policy numbers, premium amounts, claim counts, dates, and status flags. Using correct data types helps avoid bugs and incorrect reporting.

Core Types

Type	Example	Insurance Example
<code>int</code>	<code>3</code>	Number of open claims
<code>float</code>	<code>24500.75</code>	Annual premium amount
<code>str</code>	<code>"POL-2026-00129"</code>	Policy number
<code>bool</code>	<code>True</code>	Policy is active
<code>None</code>	<code>None</code>	Settlement date not available

```
policy_id = "POL-2026-00129" premium = 24500.75 open_claims = 3 is_active = True  
settled_on = None
```

Type Conversion

```
premium_text = "18000.50" premium_value = float(premium_text) claims_text = "2"  
claims_count = int(claims_text) print(type(premium_value), premium_value)  
print(type(claims_count), claims_count)
```

Always validate user input before converting.

Summary

- Data types control how values are stored and processed.
 - Use `int`, `float`, `str`, `bool`, and `None` intentionally.
 - Convert external data carefully.
-

Practice Exercises

1. Create variables for policy ID, premium, and policy status with correct types.
2. Convert `"25000"` into number and add 10%.
3. Represent missing nominee value using `None`.

Understanding Variables

Learning Objectives

- Create and update variables correctly
 - Follow good naming standards
 - Understand mutable vs immutable behavior
-

Variable Basics

A variable is a name that points to a value.

```
customer_name = "Anita Shah" sum_insured = 500000
```

Use business-meaningful names like `claim_status`, `renewal_due_date`, `branch_code`.

Reassignment

```
claim_status = "PENDING" claim_status = "APPROVED"
```

Mutable vs Immutable

```
# Immutable policy_type = "health" policy_type = policy_type.upper() # Mutable
pending_claims = ["CLM101", "CLM102"] queue = pending_claims queue.append("CLM103")
print(pending_claims)
```

Both `queue` and `pending_claims` point to the same list.

Summary

- Variables are labels for values.
 - Use clear names.
 - Be careful when multiple names reference the same mutable object.
-

Practice Exercises

1. Create 5 meaningful variables for policy processing.
2. Rename vague variables (a, b) into readable names.
3. Show list aliasing with two variables and one append operation.

Getting User Input and Print statements

Learning Objectives

- Capture input with `input()`
 - Convert user input to correct types
 - Print clean operational output
-

User Input

`input()` always returns a string.

```
policy_id = input("Enter policy ID: ") customer_age = int(input("Enter customer age: ")) base_premium = float(input("Enter base premium: "))
```

Insurance Example

```
policy_id = input("Policy ID: ").strip().upper() base = float(input("Base premium: ")) rider = float(input("Rider premium: ")) print("Policy", policy_id, "total premium:", base + rider)
```

Better Print Formatting

```
print("Policy", "Status", sep=" | ") print("POL-1001", "ACTIVE", sep=" | ") print("Processing", end="...") print("done")
```

Summary

- Convert input explicitly (`int`, `float`).
 - Use `.strip()` and case normalization for cleaner input.
 - Use `print()` options like `sep` and `end` for readable logs.
-

Practice Exercises

1. Ask for policy ID and premium, then print summary.
2. Ask claim amount and print GST-inclusive value.
3. Print a two-column status line with custom separator.

Use quotes and escape character

Learning Objectives

- Use single, double, and triple quotes properly
- Escape special characters safely
- Build clear messages for customer communication

Quote Styles

```
company = 'SafeLife Insurance' message = "Policy renewal reminder" email_body = ""Dear Customer, Your policy is due for renewal. Regards, Operations Team""
```

Escape Characters

Escape	Use
<code>\n</code>	New line
<code>\t</code>	Tab
<code>\\</code>	Backslash
<code>\"</code>	Double quote inside string

```
note = "Customer said: \"Please share claim status today.\"\" path =  
"C:\\insurance\\claims\\2026\\" print(note) print(path)
```

Raw Strings

```
pattern = r"^POL-\d{4}-\d{5}$"
```

Useful for regex and Windows paths.

Summary

- Choose quote style for readability.
 - Escape only what is required.
 - Use raw strings when backslashes are frequent.
-

Practice Exercises

1. Write a multi-line claim approval message using triple quotes.
2. Store He said "policy is active" correctly.
3. Define a Windows folder path string without errors.

Lists

Learning Objectives

- Create and update lists
 - Access list items with indexes/slices
 - Use list methods in insurance automation
-

Introduction

Lists are ideal for ordered and changing collections, such as claim IDs for a daily batch.

```
claim_ids = ["CLM1001", "CLM1002", "CLM1003"] print(claim_ids[0])
```

Common Methods

```
claim_ids.append("CLM1004") claim_ids.insert(1, "CLM1099")  
claim_ids.remove("CLM1002") last_item = claim_ids.pop() print(claim_ids)  
print("Removed last:", last_item)
```

Looping Through Lists

```
premiums = [12000, 18500, 9900] for premium in premiums: print("Premium:", premium)
```

Summary

- Lists are mutable and ordered.
 - Use methods like `append`, `remove`, and `pop`.
 - Looping over lists is common in batch processing.
-

Practice Exercises

1. Create a list of policy IDs and print the third item.
2. Add two new claim IDs and remove one.
3. Print only premiums greater than 10000.

Tuples

Learning Objectives

- Understand tuple behavior
 - Use tuples for fixed-format records
 - Apply tuple unpacking in clean code
-

Introduction

A tuple is ordered but immutable. It works well when values should not change accidentally.

```
policy_key = ("POL-2026-0001", "HEALTH")
```

Tuple Operations

```
record = ("CLM7781", "APPROVED", 25000) print(record[0]) print(len(record)) claim_id, status, amount = record print(claim_id, status, amount)
```

Summary

- Tuples are immutable sequences.
 - Use them for fixed records and returned values.
 - Tuple unpacking improves readability.
-

Practice Exercises

1. Create a tuple (`policy_id`, `customer_id`, `premium`) and unpack it.
 2. Try modifying one tuple item and observe the error.
 3. Store branch codes in a tuple and print the last item.
-

Dictionaries

Learning Objectives

- Create and update dictionaries
 - Access keys safely
 - Model policy and claim records
-

Introduction

Dictionaries map keys to values and are ideal for structured insurance data.

```
policy = { "policy_id": "POL-2026-0081", "holder_name": "Anita Shah", "premium":  
18500, "status": "ACTIVE" }
```

Access and Update

```
print(policy["holder_name"]) print(policy.get("nominee", "Not Provided"))  
policy["premium"] = 19250 policy["renewal_due"] = "2026-12-31"
```

Iterate

```
for key, value in policy.items(): print(key, ":", value)
```

Summary

- Dictionaries are key-value containers.
 - Use `.get()` for optional fields.
 - Use `.items()` for reporting and logs.
-

Practice Exercises

1. Create a claim dictionary with 5 fields.
2. Update claim status from `PENDING` to `SETTLED`.
3. Print all key-value pairs.

Sets

Learning Objectives

- Use sets for unique values
 - Apply set operations for reconciliation
 - Remove duplicates from uploaded data
-

Introduction

Sets are unordered collections of unique values.

```
policy_ids = {"POL001", "POL002", "POL003", "POL001"} print(policy_ids)
```

Set Operations

```
active_policies = {"POL001", "POL002", "POL005"} renewal_due = {"POL002", "POL004", "POL005"} print("Common:", active_policies & renewal_due) print("Only active:", active_policies - renewal_due) print("All outreach:", active_policies | renewal_due)
```

Summary

- Sets automatically remove duplicates.
 - Use intersection, union, and difference for data matching.
 - Best when order is not important.
-

Practice Exercises

1. Remove duplicate branch codes from a list.
 2. Find common policy IDs between two teams.
 3. Find missing IDs using set difference.
-

Different Methods on data structures

Learning Objectives

- Use built-in methods on lists, dicts, and sets
 - Select methods based on data shape
 - Reduce manual loop code
-

List Methods

```
claims = ["CLM5", "CLM1", "CLM9"] claims.sort() claims.reverse()  
print(claims.count("CLM5"))
```

Dictionary Methods

```
claim = {"id": "CLM5", "status": "PENDING"} print(claim.keys()) print(claim.values())  
claim.update({"status": "APPROVED"})
```

Set Methods

```
regions = {"north", "south"} regions.add("east") regions.discard("west")  
print(regions)
```

Summary

- Methods make code shorter and safer.
 - Use structure-specific methods intentionally.
 - Avoid reinventing built-in functionality.
-

Practice Exercises

1. Sort claim IDs in descending order.
 2. Update a policy dictionary with 3 new fields.
-

3. Add/remove items from a set of branch names.

String Indexing and Slicing Strings

Learning Objectives

- Access specific characters using index
 - Extract segments with slicing
 - Parse policy and claim references
-

Indexing

```
policy_id = "POL-2026-00981" print(policy_id[0]) # P print(policy_id[-1]) # 1
```

Slicing

```
print(policy_id[0:3]) # POL print(policy_id[4:8]) # 2026 print(policy_id[-5:]) #  
00981
```

Insurance Example

```
claim_ref = "CLM-MH-2026-7742" state = claim_ref[4:6] year = claim_ref[7:11]  
print(state, year)
```

Summary

- Indexing returns one character.
 - Slicing returns part of a string.
 - Useful for fixed-format IDs.
-

Practice Exercises

1. Extract branch from BR-DEL-0019.
 2. Print last 4 digits of policy number.
-

3. Reverse a claim string using slicing.

Concatenation and Repetition

Learning Objectives

- Join strings and sequences
 - Repeat sequence content
 - Build simple report layouts
-

Concatenation

```
title = "Policy" + " " + "Renewal Report" print(title)
```

Repetition

```
print("-" * 40) print("DAILY CLAIMS SUMMARY") print("-" * 40)
```

List Concatenation

```
batch_a = ["CLM001", "CLM002"] batch_b = ["CLM003"] all_claims = batch_a + batch_b  
print(all_claims)
```

Summary

- Use `+` for concatenation.
 - Use `*` for repetition.
 - Use carefully to keep output readable.
-

Practice Exercises

1. Build "May 2026 Premium Report" using concatenation.
 2. Print a 50-character separator.
 3. Merge two policy lists.
-

Common String Methods

Learning Objectives

- Clean incoming text data
 - Normalize case and spacing
 - Validate basic string conditions
-

Frequently Used Methods

```
name = " anita shah " print(name.strip().title()) email = "CUSTOMER@MAIL.COM"
print(email.lower()) status = "claim approved" print(status.upper())
```

Split and Join

```
line = "POL001,ACTIVE,19500" parts = line.split(",") print(parts) print(" |
".join(parts))
```

Validation

```
policy_id = "POL20260001" print(policy_id.startswith("POL"))
print(policy_id.isalnum())
```

Summary

- String methods are essential for data cleanup.
 - Use `strip`, `lower`, `upper`, `title`, `split`, `join` regularly.
 - Validate before processing.
-

Practice Exercises

1. Clean a mixed-case customer name.
-

2. Split a CSV string into fields.
3. Verify a policy ID starts with POL.

String Formatting

Learning Objectives

- Format data for logs and reports
 - Control alignment and decimal precision
 - Create readable premium summaries
-

Formatting Example

```
policy_id = "POL-7781" premium = 12345.6789 print(f"Policy: {policy_id:>12}")
print(f"Premium: {premium:,.2f}")
```

Output:

```
Policy: POL-7781 Premium: 12,345.68
```

Table Output Example

```
rows = [("POL001", "ACTIVE", 18000), ("POL002", "LAPSED", 9500)]
print(f"{'Policy':<10}{'Status':<10}{'Premium':>12}")
for policy, status, amount in rows: print(f"{policy:<10}{status:<10}{amount:>12,.2f}")
```

Summary

- Formatting improves readability and professionalism.
 - Use width specifiers and precision values.
 - Format money consistently.
-

Practice Exercises

1. Format claim amount with commas and 2 decimals.
 2. Left-align policy ID and right-align premium.
-

3. Print a 3-column report from tuples.

Namespaces

Learning Objectives

- Understand local and global namespaces
 - Avoid naming collisions
 - Prevent bugs by not shadowing built-ins
-

What Is a Namespace?

A namespace is a mapping of names to objects.

- **Local namespace:** inside a function
 - **Global namespace:** at script level
 - **Built-in namespace:** Python functions like `len`, `print`
-

Example

```
company = "SafeLife" # global
def show_company():
    company = "SafeLife - Mumbai" # local
    print(company)
show_company()
print(company)
```

Pitfall

```
# Avoid this
list = ["POL1", "POL2"]
```

This shadows Python's built-in `list()`.

Summary

- Python resolves names from local to global to built-in.
 - Keep names explicit and meaningful.
 - Do not use built-in names for variables.
-

Practice Exercises

1. Create local and global variables with same name and print both.
2. Shadow `sum` and observe the effect.
3. Refactor shadowed names.

Formatted String Literals (f-strings and .format() method)

Learning Objectives

- Use f-strings for modern formatting
 - Use `.format()` when templates are reusable
 - Build dynamic business messages safely
-

f-strings (Preferred)

```
policy_id = "POL-2026-1881" premium = 24500 status = "ACTIVE" message = f"Policy {policy_id} is {status}. Annual premium: ■{premium:,.2f}" print(message)
```

`.format()` Method

```
template = "Claim {0} for {1} is {2}." print(template.format("CLM-901", "Anita Shah", "SETTLED")) print("Policy {id} renewed on {date}".format(id="POL-01", date="2026-05-21"))
```

Summary

- Use f-strings in most current Python scripts.
 - Use `.format()` when string templates are externalized.
 - Both support number and alignment formatting.
-

Practice Exercises

1. Print claim amount with 2 decimal places using f-string.
 2. Recreate output using `.format()`.
 3. Build an SMS template for renewal reminder.
-

Built-in String Functions

Learning Objectives

- Use useful Python built-ins with strings
 - Combine methods and built-ins for fast cleaning
 - Validate data with `any()` and `all()`
-

Useful Built-ins

```
policy_id = "POL-2026-0009" print(len(policy_id)) print(min(policy_id))  
print(max(policy_id)) print(sorted(policy_id))
```

Practical Example

```
names = ["anita", "RAVI", "meera"] normalized = [n.title() for n in names]  
print("Count:", len(normalized)) print("Sorted:", sorted(normalized)) print("All  
non-empty:", all(n != "" for n in normalized))
```

Summary

- Built-ins like `len`, `sorted`, `min`, `max`, `any`, and `all` are powerful.
 - They reduce custom code.
 - Combine them with string methods for clean automation scripts.
-

Practice Exercises

1. Print length of each policy ID.
 2. Sort customer names after title-casing.
 3. Check if all policy IDs start with POL.
-

Module 3: Flow Control

Python operators

Learning Objectives

- Use arithmetic, comparison, and logical operators
- Write conditions used in policy and claim checks
- Combine operators for business rules

Operator Types

Type	Examples	Insurance use case
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Premium and tax calculation
Comparison	<code>==</code> , <code>!=</code> , <code>></code> , <code><</code>	Claim amount threshold checks
Logical	<code>and</code> , <code>or</code> , <code>not</code>	Multi-condition eligibility rules
Membership	<code>in</code> , <code>not in</code>	Check status in allowed list

```
claim_amount = 85000 is_fraud_flagged = False high_value = claim_amount > 50000
send_manual_review = high_value and not is_fraud_flagged print(send_manual_review)
```

Summary

- Operators are the foundation of flow control.
 - Use logical operators for real business decisions.
 - Keep conditions readable with clear variable names.
-

Practice Exercises

1. Check if premium is between 10000 and 30000.
2. Verify if claim status is in ["PENDING", "REVIEW"].
3. Write a condition for manual review based on amount and policy age.

If, else, elif clauses

Learning Objectives

- Use branching logic in automation scripts
 - Build multi-case business decision trees
 - Keep conditional code clean and testable
-

Basic Structure

```
if condition_1: # do this elif condition_2: # do this else: # fallback
```

Insurance Example: Claim Priority

```
claim_amount = 125000 fraud_score = 82 if fraud_score >= 80: priority = "INVESTIGATE"
elif claim_amount >= 100000: priority = "HIGH" elif claim_amount >= 25000: priority =
"MEDIUM" else: priority = "LOW" print("Claim priority:", priority)
```

Best Practices

- Order conditions from most specific to broad.
 - Avoid deeply nested `if` blocks when possible.
 - Move repeated logic into functions.
-

Summary

- `if/elif/else` controls decision paths.
 - Use it to implement policy and claims rules.
 - Prioritize readability in business logic.
-

Practice Exercises

1. Categorize customer age into `MINOR`, `ADULT`, `SENIOR`.
2. Set renewal discount slab using `if/elif/else`.
3. Route claim to `FAST_TRACK` or `MANUAL_REVIEW` based on amount and score.

Iteration

Learning Objectives

- Understand repeated execution with iteration
 - Process each record in a collection
 - Use `range()` and `enumerate()` effectively
-

What Is Iteration?

Iteration means repeating steps over items in a sequence.

```
policy_ids = ["POL001", "POL002", "POL003"]  
for pid in policy_ids:  
    print("Processing", pid)
```

`range()` and `enumerate()`

```
for i in range(3):  
    print("Batch", i + 1)  
claims = ["CLM101", "CLM102"]  
for index, claim_id in enumerate(claims, start=1):  
    print(index, claim_id)
```

Summary

- Iteration is essential for batch automation.
 - `for` loops are commonly used for collections.
 - `enumerate()` helps when index is also needed.
-

Practice Exercises

1. Iterate through 10 policy IDs and print each.
 2. Print index and claim ID using `enumerate()`.
 3. Use `range()` to simulate 5 daily reminder attempts.
-

Loops - for and while loops

Learning Objectives

- Use `for` and `while` loops appropriately
 - Control loop flow with `break` and `continue`
 - Avoid infinite loops in production scripts
-

`for` Loop Example

```
claim_amounts = [15000, 80000, 120000] for amount in claim_amounts: if amount > 100000: print("Escalate:", amount)
```

`while` Loop Example

```
attempts = 0 max_attempts = 3 while attempts < max_attempts: print("Sending renewal reminder", attempts + 1) attempts += 1
```

`break` and `continue`

```
claims = ["CLM001", "", "CLM003", "STOP", "CLM005"] for claim in claims: if claim == "": continue if claim == "STOP": break print("Processed", claim)
```

Summary

- Use `for` when iterating known collections.
 - Use `while` for condition-driven repeats.
 - Use `break/continue` carefully for control.
-

Practice Exercises

1. Print all active policy IDs from a list.
-

2. Use `while` to retry API call up to 5 times.
3. Skip invalid claim IDs and stop on sentinel value.

Module 4: Functions and Modules

Defining Functions

Learning Objectives

- Define reusable functions
- Keep logic DRY (Don't Repeat Yourself)
- Improve readability and maintainability

Why Functions?

Functions let you group logic once and reuse it many times.

```
def greet_policy_holder(name): print(f"Hello {name}, your policy update is ready.")
greet_policy_holder("Anita") greet_policy_holder("Rahul")
```

Function Structure

```
def function_name(parameters): # logic return value
```

Insurance Example

```
def calculate_total_premium(base_premium, rider_premium): return base_premium +
rider_premium print(calculate_total_premium(18000, 2500))
```

Summary

- Functions reduce duplication.
- Keep functions focused on one task.
- Use clear names that match business intent.

Practice Exercises

1. Write a function to print claim acknowledgment message.
2. Write a function that returns premium with GST.
3. Call each function with at least two sample inputs.

Using Parameters and Return Values

Learning Objectives

- Pass data into functions via parameters
 - Return processed results
 - Separate input, computation, and output
-

Parameters vs Return Values

- **Parameters:** inputs to a function
- **Return value:** output from a function

```
def apply_ncb_discount(premium, discount_percent):  
    discount = premium *  
    (discount_percent / 100)  
    return premium - discount  
final_premium =  
apply_ncb_discount(20000, 20)  
print(final_premium)
```

Multi-return Example

```
def claim_summary(claim_amount):  
    tax = claim_amount * 0.18  
    total = claim_amount + tax  
    return tax, total  
tax, total = claim_summary(50000)  
print(tax, total)
```

Summary

- Parameters make functions flexible.
 - Return values allow reuse of results in later logic.
 - Avoid printing inside every function unless needed.
-

Practice Exercises

1. Build a function that takes premium and rider and returns total.
 2. Build a function returning (approved_amount, rejected_amount).
-

3. Use returned values to print a final report line.

Using Arguments and Default Parameters

Learning Objectives

- Use positional and keyword arguments
 - Set practical default parameter values
 - Design functions for optional business rules
-

Positional and Keyword Arguments

```
def create_policy_notice(policy_id, status): return f"Policy {policy_id} is {status}." print(create_policy_notice("POL001", "ACTIVE")) print(create_policy_notice(status="LAPSED", policy_id="POL002"))
```

Default Parameters

```
def calculate_claim_settlement(claim_amount, deductible=1000): return max(0, claim_amount - deductible) print(calculate_claim_settlement(25000)) print(calculate_claim_settlement(25000, deductible=2500))
```

Summary

- Use defaults for common values.
 - Prefer keyword arguments when clarity matters.
 - Keep parameter order logical and simple.
-

Practice Exercises

1. Create a function with default tax rate `18%`.
 2. Call function using positional and keyword styles.
 3. Add an optional `is_senior_citizen` parameter for discount logic.
-

Scope of Variables

Learning Objectives

- Understand local and global scope
 - Prevent accidental side effects
 - Write predictable function logic
-

Local vs Global

```
company_name = "SafeLife Insurance" # global def print_branch_info(): branch = "Mumbai" # local print(company_name, branch) print_branch_info()
```

Common Mistake

```
counter = 0 def increment(): global counter counter += 1
```

Avoid heavy use of `global`; prefer returning updated values.

Better Pattern

```
def increment(counter): return counter + 1 counter = increment(0) print(counter)
```

Summary

- Local scope keeps functions safer.
 - Global variables can introduce hidden dependencies.
 - Prefer parameter/return based updates.
-

Practice Exercises

1. Create one local and one global variable and print both.
-

2. Rewrite a `global`-based counter function without `global`.
3. Explain why local scope is safer in automation scripts.

Modules and Packages

Learning Objectives

- Understand modules and packages
 - Organize related Python files logically
 - Improve reuse and team collaboration
-

Concepts

- **Package:** a folder containing related modules (often with `__init__.py`)

Example structure:

```
insurance_tools/ __init__.py premium.py claims.py reporting.py
```

Why It Matters in Insurance Automation

When scripts grow (renewals, claims, reminders, reconciliation), splitting logic into modules avoids monolithic files and simplifies maintenance.

Summary

- Use modules to split functionality.
 - Use packages to group related modules.
 - Better structure leads to easier testing and onboarding.
-

Practice Exercises

1. Create two modules: `premium.py` and `claims.py`.
 2. Move related functions to each module.
 3. Create package folder `insurance_tools` and organize files.
-

Writing and Importing Modules

Learning Objectives

- Write your own module files
 - Import modules and specific functions
 - Use aliases for clearer code
-

Create a Module

premium.py

```
def total_premium(base, rider): return base + rider
def apply_tax(amount, rate=0.18):
    return amount + amount * rate
```

Import and Use

main.py

```
import premium
gross = premium.total_premium(18000, 2500)
final = premium.apply_tax(gross)
print(final)
```

Alternative import style:

```
from premium import total_premium
print(total_premium(20000, 3000))
```

Summary

- Put reusable logic into module files.
 - Import using `import module` or `from module import name`.
 - Prefer explicit imports for readability.
-

Practice Exercises

1. Write a `claims.py` module with a `risk_level()` function.
2. Import and call it from `main.py`.
3. Add alias import and compare readability.

Module 5: SQL

Introduction to SQL

Learning Objectives

- Understand what SQL is
 - Query structured insurance data
 - Connect SQL work to automation use cases
-

What Is SQL?

SQL (Structured Query Language) is used to create, read, update, and delete data in relational databases.

In insurance, SQL is used for:

- Policy portfolio reporting
 - Claim status tracking
 - Renewal and lapse analytics
 - Regulatory and audit extracts
-

Basic Query Example

```
SELECT policy_id, holder_name, premium FROM policies;
```

Summary

- SQL is the standard language for relational data.
 - It is critical for reporting and operational automation.
 - Most Python automation workflows interact with SQL databases.
-

Practice Exercises

1. Write a query to select all rows from `claims`.
2. Select only `policy_id` and `status` from `policies`.
3. Explain one insurance process where SQL helps automate reporting.

Types of databases relational vs non-relational

Learning Objectives

- Compare relational and non-relational databases
- Choose the right database style for business scenarios
- Understand why SQL is central for structured insurance data

Relational vs Non-relational

Type	Structure	Example systems	Insurance fit
Relational	Tables + relationships	PostgreSQL, MySQL, SQLite	Core systems: policies, claims, billing
Non-relational	Documents/key-value/graph	MongoDB, Redis	Logs, event streams, flexible metadata

Why Relational Is Common in Insurance

Insurance operations rely on strongly structured data and referential consistency:

- One customer can have many policies
- One policy can have many claims
- Auditability and compliance require strict data rules

Summary

- Relational DBs are best for transactional insurance systems.
 - Non-relational DBs are useful for flexible/unstructured workloads.
 - SQL remains the primary language for core data operations.
-

Practice Exercises

1. List two relational and two non-relational databases.
2. Explain why claims data fits relational modeling.
3. Identify one scenario where document DB may help in insurance.

Installing and setting up database (sqlite / postgresql / mysql)

Learning Objectives

- Set up local SQL environments
- Understand where each database is useful
- Create a starter schema for practice

Quick Setup Choices

SQLite

- No server required
- Best for local training and small automation scripts

PostgreSQL

- Strong enterprise features
- Common in production insurance platforms

MySQL

- Widely used in web and operational systems
- Strong ecosystem and tooling

Starter Table Example

```
CREATE TABLE policies ( policy_id TEXT PRIMARY KEY, customer_id TEXT NOT NULL,
product_type TEXT NOT NULL, premium NUMERIC NOT NULL, status TEXT NOT NULL );
```

Summary

- Use SQLite for learning and lightweight automation.

- Use PostgreSQL/MySQL for larger multi-user systems.
 - Start with a clean schema and consistent naming.
-

Practice Exercises

1. Install SQLite and create a local DB file.
2. Create `customers` and `claims` tables.
3. Insert three rows in each table.

Basic queries and filtering conditions

Learning Objectives

- Retrieve specific columns and rows
 - Filter records using `WHERE`
 - Sort and limit outputs for reporting
-

Select and Filter

```
SELECT policy_id, holder_name, premium FROM policies WHERE status = 'ACTIVE';
```

Multiple Conditions

```
SELECT claim_id, policy_id, claim_amount FROM claims WHERE claim_amount > 50000 AND claim_status = 'PENDING';
```

Sort and Limit

```
SELECT policy_id, premium FROM policies ORDER BY premium DESC LIMIT 5;
```

Summary

- Use `SELECT` for retrieval and `WHERE` for filtering.
 - Combine conditions with `AND/OR`.
 - Use `ORDER BY` and `LIMIT` for ranked views.
-

Practice Exercises

1. Find all lapsed policies.
 2. List top 10 highest claim amounts.
-

3. Select policies with premium between 15000 and 30000.

CRUD operations

Learning Objectives

- Perform Create, Read, Update, Delete operations
 - Apply safe update and delete conditions
 - Understand CRUD in insurance operations
-

Create

```
INSERT INTO policies (policy_id, customer_id, product_type, premium, status) VALUES ('POL-1001', 'CUS-501', 'HEALTH', 18000, 'ACTIVE');
```

Read

```
SELECT * FROM policies WHERE policy_id = 'POL-1001';
```

Update

```
UPDATE policies SET premium = 19500 WHERE policy_id = 'POL-1001';
```

Delete

```
DELETE FROM policies WHERE policy_id = 'POL-1001';
```

Summary

- CRUD operations are daily database tasks.
 - Always use `WHERE` in update/delete statements.
 - Validate impact before running destructive queries.
-

Practice Exercises

1. Insert 3 claim records.
2. Update one claim status from `PENDING` to `APPROVED`.
3. Delete one test row using a safe filter.

Basic queries with aggregate functions

Learning Objectives

- Use aggregate functions for summary reporting
 - Group records by business dimensions
 - Build insurer dashboard metrics
-

Common Aggregates

- `COUNT()` → number of records
- `SUM()` → total amount
- `AVG()` → average value
- `MIN()` and `MAX()` → smallest/largest value

```
SELECT COUNT(*) AS total_claims, SUM(claim_amount) AS total_claim_amount,  
AVG(claim_amount) AS avg_claim_amount FROM claims;
```

Grouped Summary

```
SELECT product_type, COUNT(*) AS policy_count, SUM(premium) AS total_premium FROM  
policies GROUP BY product_type;
```

Summary

- Aggregates convert detailed rows into business insights.
 - Use `GROUP BY` for category-wise analysis.
 - Essential for operational and management reporting.
-

Practice Exercises

1. Count active vs lapsed policies by status.
-

2. Find maximum claim amount per branch.
3. Calculate average premium by product type.

SQL joins

Learning Objectives

- Combine data across related tables
 - Choose correct join type
 - Build customer-policy-claim reports
-

Common Join Types

- `INNER JOIN` → matching rows only
 - `LEFT JOIN` → all rows from left table + matches from right
 - `RIGHT JOIN` → all rows from right table + matches from left
-

Insurance Example

```
SELECT c.customer_id, c.customer_name, p.policy_id, p.status, cl.claim_id,
cl.claim_amount FROM customers c LEFT JOIN policies p ON c.customer_id =
p.customer_id LEFT JOIN claims cl ON p.policy_id = cl.policy_id;
```

This query helps identify customers with policies but no claims, and customers with no policies.

Summary

- Joins are essential for multi-table analysis.
 - Use meaningful aliases for readability.
 - Start with `LEFT JOIN` for complete customer views.
-

Practice Exercises

1. Join policies with customers and show holder names.
-

2. Find policies with no claim using join + IS NULL.
3. Build a claim summary with customer name and product type.

Advanced queries

Learning Objectives

- Use subqueries and common table expressions (CTEs)
 - Apply case-based logic in SQL
 - Build advanced operational reports
-

Subquery Example

Find claims above average amount:

```
SELECT claim_id, policy_id, claim_amount FROM claims WHERE claim_amount > ( SELECT  
AVG(claim_amount) FROM claims );
```

CTE Example

```
WITH high_value_claims AS ( SELECT claim_id, policy_id, claim_amount FROM claims  
WHERE claim_amount >= 100000 ) SELECT * FROM high_value_claims;
```

CASE Example

```
SELECT claim_id, claim_amount, CASE WHEN claim_amount >= 100000 THEN 'HIGH' WHEN  
claim_amount >= 25000 THEN 'MEDIUM' ELSE 'LOW' END AS risk_band FROM claims;
```

Summary

- Advanced SQL helps create intelligent business reports.
 - Subqueries and CTEs improve modularity.
 - `CASE` is useful for classification logic.
-

Practice Exercises

1. Find policies with premium above average premium.
2. Use CTE to list active policies due for renewal this month.
3. Categorize premium values into slabs with CASE.

Module 6: Introduction to Python Automation

Overview of Automation and Its Benefits

Learning Objectives

By the end of this chapter, you will be able to:

- Define automation in a practical business context
 - Identify tasks that are good candidates for automation
 - Explain the major benefits and limits of automation
 - Relate automation ideas to Python-based workflows
-

Introduction

Automation means using software to perform tasks with minimal manual effort. Instead of repeating the same steps by hand every day, we write instructions that a computer can follow reliably.

In business operations, automation is often used for:

- Copying or cleaning data
- Generating reports
- Sending emails or notifications
- Renaming, moving, or processing files
- Updating databases or spreadsheets
- Running scheduled background jobs

Python is a strong choice for automation because it is easy to read, has a huge ecosystem of libraries, and works well with files, APIs, spreadsheets, databases, and operating system tasks.

Key Concepts

What automation means

A task is automated when a script or system performs a sequence of steps that a person would otherwise do manually.

For example, imagine an operations analyst who every morning:

1. Downloads a CSV report
2. Removes incomplete rows
3. Calculates totals
4. Saves a cleaned file
5. Emails the summary to a manager

If those same steps are written in Python and run automatically, that process becomes an automation workflow.

Characteristics of a good automation candidate

Not every task should be automated. Good automation candidates are usually:

- Repetitive
- Rule-based
- Time-consuming
- Prone to human error
- Performed frequently
- Based on structured inputs and outputs

Examples:

- Daily policy renewal report generation
- Bulk file renaming
- Consolidating branch-level spreadsheets
- Checking claim records for missing fields

Poor candidates for automation often require constant human judgment, changing rules, or unstructured decision-making.

Benefits of automation

Automation creates value in several ways:

1. Saves time

A task that takes 30 minutes per day can consume many hours over a month. Automation can reduce that effort to a single script run.

2. Reduces errors

Humans can skip rows, mistype values, or forget steps. Scripts follow the same logic every time.

3. Improves consistency

Automation applies rules the same way across all files, records, or systems.

4. Increases scalability

A person may be able to process 10 files manually, but a script can often process hundreds with little extra effort.

5. Frees people for higher-value work

Instead of spending time on repetitive tasks, teams can focus on analysis, decisions, and customer service.

Limits and risks of automation

Automation is powerful, but it must be used carefully.

Common risks include:

- Automating a bad process without improving it first
- Running scripts on incorrect data
- Missing edge cases or exceptions
- Accidentally overwriting files or records
- Depending on fragile steps such as screen clicks or exact file names

A good rule is: **understand the process first, then automate it.**

Where Python fits into automation

Python can automate tasks at multiple levels:

- File automation: create, rename, move, and read files
- Data automation: process CSV, Excel, JSON, or database data

- Web automation: call APIs or interact with websites
- Desktop automation: automate mouse, keyboard, or GUI actions
- Scheduled automation: run scripts at specific times

This course builds those skills progressively.

Examples

Example 1: Manual task versus automated task

```
manual_steps = [ "Open report", "Remove empty rows", "Calculate total premium", "Save cleaned file", "Email summary" ] print("Manual workflow:") for step in manual_steps: print(f"- {step}") print("\nAutomated workflow:") print("- Run Python script to complete all steps")
```

Example 2: Estimate time saved

```
daily_minutes_saved = 25 working_days_per_month = 22 monthly_hours_saved = (daily_minutes_saved * working_days_per_month) / 60 print(f"Estimated time saved per month: {monthly_hours_saved:.2f} hours")
```

Example 3: Identify automation candidates

```
tasks = { "Generate daily report": True, "Review complex legal dispute": False, "Rename 500 uploaded files": True, "Approve claim based on human judgment": False, } for task, automatable in tasks.items(): label = "Good candidate" if automatable else "Needs human judgment" print(f"{task}: {label}")
```

Notes

- Start with small, low-risk automation tasks.
- Keep original files unchanged until you trust the script.
- Test with sample data before using real production data.
- Add logging or printed status messages so you can trace what the script did.
- Automation should support people, not remove necessary review or controls.

Summary

- Automation uses software to perform repeated, rule-based tasks.
 - Good automation targets are frequent, structured, and error-prone manual processes.
 - Python is widely used for automation because it is simple, flexible, and supported by many libraries.
-

Practice Exercises

1. List three tasks from your work or studies that are repetitive and rule-based.
 2. Explain two benefits and one risk of automating a daily reporting process.
 3. Write a short paragraph describing a manual workflow that could be improved with Python.
-

Further Reading

- [Python documentation](#)
- [Automate the Boring Stuff with Python](#)

Introduction to Python Libraries for Automation

Learning Objectives

By the end of this chapter, you will be able to:

- Describe what a Python library is
 - Identify common libraries used in automation projects
 - Match libraries to automation tasks such as files, web requests, spreadsheets, and databases
 - Install libraries using `pip` and use them responsibly
-

Introduction

A Python library is a collection of reusable code that helps you perform tasks without writing everything from scratch. Instead of building file readers, spreadsheet handlers, or HTTP clients yourself, you can use well-tested libraries.

Libraries are one of the main reasons Python is so effective for automation.

Key Concepts

Standard library vs third-party libraries

Python gives you two major sources of tools:

Standard library

These libraries come with Python and do not require separate installation.

Common examples:

- `os` for working with operating system paths and folders
 - `pathlib` for clean path handling
 - `shutil` for copying and moving files
 - `csv` for reading and writing CSV files
 - `json` for JSON data
-

- `sqlite3` for SQLite databases
- `datetime` for date and time logic
- `subprocess` for running system commands

Third-party libraries

These are installed separately, usually with `pip`.

Popular automation examples:

- `requests` for calling web APIs
- `pandas` for tabular data processing
- `openpyxl` for Excel files
- `pyautogui` for mouse and keyboard automation
- `beautifulsoup4` for parsing HTML
- `selenium` for browser automation
- `python-dotenv` for loading environment variables from a `.env` file

Choosing the right library

When selecting a library, consider:

- Does it solve the problem directly?
- Is it well known and actively maintained?
- Is it already used in your project?
- Does it introduce unnecessary complexity?

In many cases, the standard library is enough. Start there before adding external dependencies.

Common library categories for automation

1. File and folder automation

Useful modules:

- `pathlib`
- `os`
- `shutil`

- `glob`

Use cases:

- Rename files in bulk
- Move reports into dated folders
- Search for files matching a pattern

2. Data processing

Useful modules and libraries:

- `csv`
- `json`
- `pandas`

Use cases:

- Clean downloaded reports
- Transform rows and columns
- Merge data from multiple files

3. Web and API automation

Useful libraries:

- `requests`
- `beautifulsoup4`
- `selenium`

Use cases:

- Download data from an API
- Extract information from websites
- Automate browser-based systems when APIs are unavailable

4. Spreadsheet automation

Useful libraries:

- `openpyxl`
- `pandas`

Use cases:

- Read and update Excel workbooks
- Build summary sheets
- Format export files

5. Database automation

Useful modules and libraries:

- `sqlite3`
- database drivers such as `psycopg` or `mysql-connector-python`

Use cases:

- Load data into tables
- Query operational records
- Generate reports automatically

Installing libraries with pip

Use `pip` to install third-party packages:

```
pip install requests pandas openpyxl
```

In professional workflows, it is best to install packages inside a virtual environment so each project keeps its own dependencies.

Importing libraries in Python

```
import csv from pathlib import Path import requests
```

Once imported, you can call functions or classes provided by the library.

Examples

Example 1: Using the standard library for file paths

```
from pathlib import Path reports_folder = Path("reports") output_file =
reports_folder / "daily_summary.csv" print(output_file)
```

Example 2: Using `requests` to call an API

```
import requests response =
requests.get("https://jsonplaceholder.typicode.com/todos/1", timeout=10)
print(response.status_code) print(response.json())
```

Example 3: Using `pandas` for quick data analysis

```
import pandas as pd data = { "branch": ["North", "South", "North"], "premium": [1200,
900, 1500] } df = pd.DataFrame(data) print(df.groupby("branch")["premium"].sum())
```

Notes

- Prefer the Python standard library when it already solves the problem well.
 - Add third-party libraries only when they clearly save time or improve reliability.
 - Read package documentation before using a new library in production work.
 - Pin important dependencies in `requirements.txt` for reproducible environments.
 - Be careful with browser or GUI automation because those workflows can be more fragile than API or file-based automation.
-

Summary

- Python libraries provide reusable building blocks for automation.
 - The standard library covers many common automation tasks, including files, CSV, JSON, and SQLite.
 - Third-party libraries such as `requests`, `pandas`, and `openpyxl` expand Python's automation capabilities.
-

Practice Exercises

1. Name three standard library modules that are useful for automation and describe one use case for each.
 2. Compare `requests` and `selenium` in one or two sentences.
 3. Write a small Python script that imports `Path` from `pathlib` and prints a file path.
-

Further Reading

- [Python Standard Library](#)
- [PyPI](#)
- [Requests documentation](#)
- [pandas documentation](#)

Setting up the development environment

Learning Objectives

By the end of this chapter, you will be able to:

- Install Python and verify the installation
 - Create and activate a virtual environment
 - Install project dependencies with `pip`
 - Organize a basic folder structure for automation scripts
 - Use a code editor and terminal effectively for Python work
-

Introduction

Before building automation scripts, you need a reliable development environment. A good setup makes it easier to install packages, run scripts, debug errors, and keep project dependencies organized.

A typical Python automation environment includes:

- Python installed on your machine
 - A terminal or command prompt
 - A code editor such as Zed, VS Code, or PyCharm
 - A project folder
 - A virtual environment
 - A `requirements.txt` file for dependencies
-

Key Concepts

Step 1: Install Python

Download Python from the official website:

- <https://www.python.org/downloads/>

After installation, verify it in the terminal:

```
python --version
```

Some systems use:

```
python3 --version
```

If Python is installed correctly, you should see a version number such as `Python 3.12.0`.

Step 2: Create a project folder

A simple project structure helps keep automation scripts organized.

Example:

```
automation-project/  scripts/  data/  output/  logs/   
requirements.txt
```

This structure separates source scripts, input files, generated outputs, and logs.

Step 3: Create a virtual environment

A virtual environment isolates packages for one project so they do not conflict with other Python projects.

Create it with:

```
python -m venv venv
```

Or on some systems:

```
python3 -m venv venv
```

This creates a local folder named `venv`.

Step 4: Activate the virtual environment

On macOS or Linux

```
source venv/bin/activate
```

On Windows (Command Prompt)

```
venv\Scripts\activate
```

Once activated, package installations with `pip` will go into the virtual environment instead of the global Python installation.

Step 5: Install required libraries

For example:

```
pip install requests pandas openpyxl
```

Save the installed packages to a requirements file:

```
pip freeze > requirements.txt
```

This lets you recreate the environment later with:

```
pip install -r requirements.txt
```

Step 6: Choose a code editor

A good editor improves productivity with:

- Syntax highlighting
- Auto-completion
- File explorer
- Integrated terminal
- Search across files

Popular choices include Zed, VS Code, and PyCharm. For this course, any editor that can run Python files and open folders is enough.

Step 7: Run a test script

Create a file such as `scripts/hello_automation.py` and run:

```
print("Automation environment is ready!")
```

Then execute it:

```
python scripts/hello_automation.py
```

If the message appears, your environment is working.

Step 8: Basic setup best practices

- Use one virtual environment per project.
- Keep dependency lists in `requirements.txt`.
- Do not store secrets directly in scripts.

- Use clear folder names for data and outputs.
 - Test installation and script execution before starting larger work.
-

Examples

Example 1: Create useful project folders in Python

```
from pathlib import Path for folder_name in ["scripts", "data", "output", "logs"]:  
    Path(folder_name).mkdir(exist_ok=True) print("Project folders created")
```

Example 2: Simple dependency notes file

```
libraries = ["requests", "pandas", "openpyxl"] for library in libraries:  
    print(f"Install with: pip install {library}")
```

Example 3: Check current Python executable

```
import sys print("Python executable:", sys.executable)
```

This is useful when confirming that your script is running inside the expected virtual environment.

Notes

- If `python` does not work, try `python3`.
 - If `pip` fails, use `python -m pip install package_name`.
 - If a library installs globally by mistake, check whether the virtual environment is activated.
 - Avoid keeping large generated files in the same folder as source code.
 - When working in teams, share `requirements.txt`, not your `venv` folder.
-

Summary

- A good Python automation setup includes Python, a project folder, a virtual environment, and a code editor.
 - Virtual environments keep project dependencies isolated and manageable.
 - Verifying the setup early helps prevent package and execution issues later.
-

Practice Exercises

1. Install Python and verify the version from your terminal.
 2. Create a new folder for an automation project and add `scripts`, `data`, and `output` directories.
 3. Create a virtual environment, install one package, and save the dependencies to `requirements.txt`.
-

Further Reading

- [Python Packaging User Guide](#)
- [venv documentation](#)
- [pip documentation](#)

Module 7: File Handling and Manipulation

Working with files and directories

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the difference between files and directories
 - Navigate file paths in Python
 - Create, inspect, and organize folders programmatically
 - Use modern path handling practices for automation scripts
-

Introduction

Most automation tasks depend on files. A script may need to read reports, create logs, save outputs, or organize folders by date. To do that safely, you need to understand how Python works with files and directories.

A **file** stores data, such as a text document, CSV, Excel workbook, or image. A **directory** (or folder) contains files and sometimes other directories.

Key Concepts

Files, folders, and paths

A **path** tells Python where a file or folder is located.

Examples:

- `reports/daily.csv`
- `output/summary.txt`
- `logs/2026/may/run.log`

Paths may be:

- **Relative:** based on the current working directory
- **Absolute:** the full location on the machine

For automation projects, relative paths are often easier to manage inside one project folder.

Why `pathlib` is useful

Python's `pathlib` module provides a clean, readable way to work with paths.

Benefits:

- easier path joining
- clearer code
- better cross-platform support

Example:

```
from pathlib import Path report = Path("reports") / "daily.csv" print(report)
```

Common directory tasks

Automation scripts often need to:

- check whether a file exists
- create folders before writing output
- list files inside a directory
- separate input and output locations

Safe automation habits

When working with files and directories:

- avoid hardcoding machine-specific paths unless necessary
- create output folders if they do not exist
- keep source files separate from generated files
- test on sample files before using important data

Examples

Example 1: Create project folders

```
from pathlib import Path for folder in ["input", "output", "logs"]:  
    Path(folder).mkdir(exist_ok=True) print("Folders are ready")
```

Example 2: Check whether a file exists

```
from pathlib import Path file_path = Path("input") / "customers.csv" if  
file_path.exists(): print("File found") else: print("File not found")
```

Example 3: List items in a directory

```
from pathlib import Path for item in Path("input").iterdir(): print(item.name)
```

Notes

- Prefer `pathlib` for new Python code.
 - Keep file paths readable and consistent across your project.
 - Use separate folders such as `input`, `output`, and `logs`.
 - Check for missing paths before reading or writing files.
-

Summary

- Files store data, while directories organize files and subdirectories.
 - Python scripts often rely on path handling for real automation work.
 - `pathlib` makes file and directory operations easier and safer.
-

Practice Exercises

1. Create a Python script that makes `input`, `output`, and `archive` folders.
 2. Write code to check whether `input/report.txt` exists.
 3. List all files inside a folder and print their names.
-

Further Reading

- [pathlib documentation](#)
- [Python file and directory access](#)

Automating file operations

Learning Objectives

By the end of this chapter, you will be able to:

- Automate common file operations with Python
 - Copy, move, rename, and delete files safely
 - Organize files based on simple rules
 - Apply automation patterns to repetitive file workflows
-

Introduction

Many business and personal workflows involve repetitive file tasks: renaming uploads, moving reports into dated folders, archiving old logs, or copying templates. Python can automate these steps and reduce time spent on manual file handling.

Key Concepts

Common file automation tasks

Typical file automation includes:

- copying files
- moving files into folders
- renaming files in bulk
- deleting temporary files
- archiving old outputs

Useful tools

Python provides several useful modules:

- `pathlib` for path handling

- `shutil` for copying and moving
- `os` for some low-level filesystem work

Copy, move, and rename

Use `shutil.copy()` or `shutil.copy2()` to copy files. Use `shutil.move()` to move or rename files.

Delete with care

Deleting files is permanent in many automation contexts. Before deleting:

- confirm the path
- test on sample data
- avoid deleting source files unless required

Rule-based organization

A useful automation pattern is:

1. scan files
2. decide what to do based on a rule
3. perform the action
4. log or print the result

Examples

Example 1: Rename all `.txt` files

```
from pathlib import Path
folder = Path("input")
for file_path in folder.glob("*.txt"):
    new_name = file_path.with_name(f"processed_{file_path.name}")
    file_path.rename(new_name)
    print(f"Renamed {file_path.name} -> {new_name.name}")
```

Example 2: Move CSV files to an archive folder

```
from pathlib import Path
import shutil
source = Path("input")
archive = Path("archive")
archive.mkdir(exist_ok=True)
for file_path in source.glob("*.csv"):
    shutil.move(str(file_path), archive / file_path.name)
    print(f"Moved {file_path.name} to archive")
```

Example 3: Delete temporary files

```
from pathlib import Path for file_path in Path("output").glob("*.tmp"):
file_path.unlink() print(f"Deleted {file_path.name}")
```

Notes

- Prefer moving files into an archive instead of deleting them immediately.
 - Print or log each action during automation.
 - Test file automation on a small folder first.
 - Be careful with wildcard patterns so you do not affect the wrong files.
-

Summary

- Python can automate repetitive file operations quickly and reliably.
 - `pathlib` and `shutil` are useful tools for copy, move, rename, and delete workflows.
 - File automation should be rule-based, testable, and safe.
-

Practice Exercises

1. Write a script that renames all `.log` files by adding a date prefix.
 2. Move all `.csv` files from `input` to `archive`.
 3. Delete all `.tmp` files from an `output` folder after printing their names.
-

Further Reading

- [shutil documentation](#)
- [pathlib documentation](#)

File searching and filtering

Learning Objectives

By the end of this chapter, you will be able to:

- Search for files in directories and subdirectories
 - Filter files by extension, name pattern, or size
 - Use glob patterns in automation scripts
 - Build file-selection logic before processing data
-

Introduction

Automation scripts rarely process every file in a folder. Usually, they need only certain files, such as `.csv` reports, files with a date in the name, or files larger than a minimum size. Searching and filtering helps scripts target the right inputs.

Key Concepts

Searching for files

Python can search for files using:

- `pathlib.Path.glob()`
- `pathlib.Path.rglob()`
- `glob` from the standard library

`glob()` searches one directory level based on a pattern. `rglob()` searches recursively.

Common file filters

You can filter by:

- file extension
 - file name pattern
-

- file size
- last modified date
- whether the path is a file or directory

Glob patterns

Examples:

- `*.txt` → all text files
- `report_*.csv` → CSV files starting with `report_`
- `**/*.log` → all log files in nested folders

Why filtering matters

Filtering prevents mistakes and improves performance. It ensures your script processes only relevant files.

Examples

Example 1: Find all CSV files

```
from pathlib import Path
for file_path in Path("input").glob("*.csv"):
    print(file_path.name)
```

Example 2: Search recursively for log files

```
from pathlib import Path
for file_path in Path("logs").rglob("*.log"):
    print(file_path)
```

Example 3: Filter files by size

```
from pathlib import Path
for file_path in Path("input").iterdir():
    if file_path.is_file() and file_path.stat().st_size > 1024:
        print(f"Large file: {file_path.name}")
```

Notes

- Use `glob()` when you know the folder level.
- Use `rglob()` when files may be inside subfolders.

- Combine pattern matching with additional checks such as size or date.
 - Always confirm that a path is a file before processing it.
-

Summary

- File searching helps automation scripts locate relevant inputs.
 - Filtering reduces mistakes by limiting processing to matching files.
 - `glob()` and `rglob()` are practical tools for file selection.
-

Practice Exercises

1. Print all `.txt` files inside an `input` folder.
 2. Search recursively for `.csv` files inside a `reports` directory.
 3. Print only files larger than 5 KB from a folder of sample data.
-

Further Reading

- [glob documentation](#)
- [pathlib glob patterns](#)

Opening Files

Learning Objectives

By the end of this chapter, you will be able to:

- Open files using the correct file modes
 - Understand read, write, and append behavior
 - Use context managers for safe file handling
 - Avoid common mistakes when opening files
-

Introduction

Before Python can read from or write to a file, the file must be opened. File opening tells Python what file to access and how you plan to use it.

This is a basic but important skill because nearly every automation workflow reads input files or generates output files.

Key Concepts

The `open()` function

Python uses the built-in `open()` function:

```
open("example.txt", "r")
```

The first argument is the file path. The second argument is the file mode.

Common file modes

- `"r"` → read
 - `"w"` → write (creates or overwrites)
 - `"a"` → append
 - `"rb"` → read binary
 - `"wb"` → write binary
-

Why context managers matter

The safest way to open files is with `with`:

```
with open("example.txt", "r") as file: data = file.read()
```

This automatically closes the file after the block finishes, even if an error occurs.

Text encoding

For text files, specify encoding when appropriate:

```
with open("notes.txt", "r", encoding="utf-8") as file: content = file.read()
```

This helps avoid text decoding issues across systems.

Examples

Example 1: Open and read a text file

```
with open("input/report.txt", "r", encoding="utf-8") as file: content = file.read()
print(content)
```

Example 2: Open a file for writing

```
with open("output/result.txt", "w", encoding="utf-8") as file: file.write("Automation
completed successfully")
```

Example 3: Open a file for appending

```
with open("logs/run.log", "a", encoding="utf-8") as file: file.write("Script
finished\n")
```

Notes

- Use `with open(...)` instead of opening a file manually and forgetting to close it.
 - Be careful with `"w"` mode because it overwrites existing content.
 - Use `encoding="utf-8"` for most text automation tasks.
 - Check that the file path exists before opening in read mode.
-

Summary

- Files must be opened before Python can read or write them.
 - File modes control whether you read, write, or append.
 - Context managers are the safest way to handle files.
-

Practice Exercises

1. Open a text file and print its contents.
 2. Create a new file and write one line into it.
 3. Append a second line to the same file.
-

Further Reading

- [open\(\) documentation](#)
- [Python I/O tutorial](#)

The `os` and `os.path` modules

Learning Objectives

By the end of this chapter, you will be able to:

- Use `os` and `os.path` for file and directory operations
 - Inspect the current working directory and path properties
 - Build and validate paths programmatically
 - Recognize when `pathlib` may be a cleaner alternative
-

Introduction

Before `pathlib` became widely preferred, many Python scripts used `os` and `os.path` for filesystem tasks. These modules are still common in real codebases, so it is important to understand them.

Key Concepts

The `os` module

The `os` module provides functions for interacting with the operating system.

Common uses:

- get the current working directory
- list folder contents
- create directories
- rename or remove files

The `os.path` module

`os.path` helps with path operations such as:

- joining path parts
- checking whether a path exists

- checking whether a path is a file or folder
- extracting file names or extensions

Example operations

Useful functions include:

- `os.getcwd()`
- `os.listdir()`
- `os.mkdir()` or `os.makedirs()`
- `os.remove()`
- `os.path.join()`
- `os.path.exists()`
- `os.path.isfile()`
- `os.path.isdir()`

`os.path` VS `pathlib`

Both approaches are valid. In modern Python, `pathlib` is often easier to read. However, you will still see `os` and `os.path` in older scripts and in some production code.

Examples

Example 1: Print the current working directory

```
import os print(os.getcwd())
```

Example 2: Build and inspect a path

```
import os file_path = os.path.join("input", "report.csv") print(file_path)
print("Exists:", os.path.exists(file_path))
```

Example 3: List files in a folder

```
import os for name in os.listdir("input"): print(name)
```

Notes

- `os.path.join()` is safer than manually concatenating path strings.
 - Use `os.makedirs(path, exist_ok=True)` when creating nested folders.
 - Prefer `pathlib` for new code, but be comfortable reading `os`-based code.
 - Always check path existence before removing or opening files.
-

Summary

- `os` and `os.path` are core tools for filesystem automation.
 - They support path creation, folder listing, and existence checks.
 - Modern Python often favors `pathlib`, but `os` remains important to understand.
-

Practice Exercises

1. Print the current working directory with Python.
 2. Use `os.path.join()` to build a path to `output/result.txt`.
 3. Write a script that lists all items inside a folder using `os.listdir()`.
-

Further Reading

- [os documentation](#)
- [os.path documentation](#)

Reading files

Learning Objectives

By the end of this chapter, you will be able to:

- Read text files using different techniques
 - Process file content line by line
 - Understand when to use `read()`, `readline()`, and iteration
 - Apply file reading to automation workflows
-

Introduction

Reading file content is one of the most common automation tasks. Scripts often need to load configuration files, process reports, parse logs, or inspect downloaded text data.

Python provides several ways to read from files depending on the amount and structure of the content.

Key Concepts

Read the whole file

`read()` returns the entire file as one string.

Best for:

- small files
- quick inspection
- simple text processing

Read one line at a time

`readline()` reads a single line.

Best for:

- step-by-step processing
- situations where you want only the first few lines

Iterate through the file

Looping over the file object is memory-efficient and useful for larger files.

Best for:

- logs
- reports with many lines
- row-based processing

Strip newline characters

Use `.strip()` or `.rstrip()` when you want cleaner text output.

Examples

Example 1: Read all content

```
with open("input/report.txt", "r", encoding="utf-8") as file: content = file.read()
print(content)
```

Example 2: Read one line

```
with open("input/report.txt", "r", encoding="utf-8") as file: first_line =
file.readline() print(first_line.strip())
```

Example 3: Process a file line by line

```
with open("input/report.txt", "r", encoding="utf-8") as file: for line_number, line
in enumerate(file, start=1): print(f"{line_number}: {line.strip()}")
```

Notes

- Use `read()` for small files and quick scripts.
- Use line-by-line iteration for larger files.
- Clean line endings with `.strip()` when printing or comparing text.

- Always open text files with the correct encoding when possible.
-

Summary

- Python supports multiple file-reading patterns.
 - The right method depends on file size and processing needs.
 - Line-by-line reading is often best for automation scripts.
-

Practice Exercises

1. Read a text file and print the full contents.
 2. Read only the first line of a file.
 3. Print each line of a file with its line number.
-

Further Reading

- [Python file methods](#)

Writing into a file

Learning Objectives

By the end of this chapter, you will be able to:

- Write text data into files using Python
 - Understand how write mode affects existing content
 - Create output files for automation workflows
 - Format written content clearly and safely
-

Introduction

Automation scripts often produce output: reports, logs, summaries, transformed data, or status files. Writing to files allows a script to save results so they can be reviewed, shared, or used in later steps.

Key Concepts

Write mode

To write into a file, use `"w"` mode:

```
with open("output.txt", "w", encoding="utf-8") as file: file.write("Hello")
```

If the file does not exist, Python creates it. If it already exists, Python overwrites it.

Writing strings

The `write()` method writes a string exactly as provided. If you want a new line, include `\n`.

Structured output

It is often helpful to write files in a predictable format, such as:

- one item per line
 - comma-separated text
 - key-value lines
-

- timestamped log entries

When writing is useful in automation

Examples:

- save a cleaned version of a report
 - create an output summary for a manager
 - store run results in a text file
 - generate intermediate files for another script
-

Examples

Example 1: Write one message to a file

```
with open("output/status.txt", "w", encoding="utf-8") as file: file.write("Automation run completed")
```

Example 2: Write multiple lines

```
lines = ["North branch: 12 claims", "South branch: 8 claims", "Total: 20 claims"]
with open("output/summary.txt", "w", encoding="utf-8") as file: for line in lines:
file.write(line + "\n")
```

Example 3: Write generated values

```
total_premium = 24500 report_date = "2026-05-23" with open("output/report.txt", "w",
encoding="utf-8") as file: file.write(f"Report date: {report_date}\n")
file.write(f"Total premium: {total_premium}\n")
```

Notes

- "w" mode replaces existing content, so use it carefully.
 - Ensure the parent output directory exists before writing.
 - Add newline characters when writing multiple lines.
 - Keep output format consistent so other scripts can read it later.
-

Summary

- Writing to files allows automation scripts to store results.
 - "w" mode creates or overwrites a file.
 - Clear, structured output makes later automation easier.
-

Practice Exercises

1. Write a file named `status.txt` containing one success message.
 2. Save three lines of summary text into a new file.
 3. Generate a text report with two labeled values using f-strings.
-

Further Reading

- [Python file methods](#)

Appending data to a file

Learning Objectives

By the end of this chapter, you will be able to:

- Append new data to existing files without overwriting them
 - Use append mode for logs and running summaries
 - Understand how append behavior differs from write mode
 - Add simple audit-style output to automation scripts
-

Introduction

Sometimes a script should add new information to a file instead of replacing the old content. This is common for logs, history files, or daily status updates. In those cases, Python uses append mode.

Key Concepts

Append mode

Use "a" mode to add content to the end of a file:

```
with open("run.log", "a", encoding="utf-8") as file: file.write("Finished\n")
```

If the file does not exist, Python creates it. If it exists, the new content is added after the existing content.

Append vs write

- "w" overwrites the file
- "a" preserves the existing content and adds new data

Common automation use cases

Append mode is useful for:

- execution logs

- daily run history
- error tracking
- status reports that grow over time

Add timestamps or labels

Appending becomes more useful when each entry has context, such as a date, time, or process name.

Examples

Example 1: Append one log line

```
with open("logs/run.log", "a", encoding="utf-8") as file: file.write("Automation completed successfully\n")
```

Example 2: Append multiple values

```
entries = ["Loaded file: claims.csv", "Processed 120 rows", "Export completed"] with open("logs/process.log", "a", encoding="utf-8") as file: for entry in entries: file.write(entry + "\n")
```

Example 3: Append a timestamped message

```
from datetime import datetime with open("logs/run.log", "a", encoding="utf-8") as file: timestamp = datetime.now().isoformat(timespec="seconds") file.write(f"[{timestamp}] Script finished\n")
```

Notes

- Use append mode for logs and run history.
- Add newline characters so entries stay readable.
- Avoid appending unstructured text if another script will parse the file later.
- Consider timestamps for better traceability.

Summary

- Append mode adds content to an existing file without deleting earlier data.

- It is especially useful for logs, history, and recurring status output.
 - Good appended data should be readable, structured, and easy to trace.
-

Practice Exercises

1. Append a new line to a file named `run.log`.
 2. Write a script that appends three process messages to a log file.
 3. Add a timestamped status line to a file using `datetime`.
-

Further Reading

- [datetime documentation](#)
- [Python I/O tutorial](#)

Introduction to web scraping

Learning Objectives

By the end of this chapter, you will be able to:

- Define web scraping in simple terms
 - Understand when web scraping is appropriate
 - Recognize the difference between downloading and parsing web content
 - Identify common risks and ethical considerations
-

Introduction

Web scraping means collecting information from web pages using software. Instead of copying text manually from a website, a Python script can download page content and extract the specific information you need.

This is useful when data is visible on a webpage but not easily available in a downloadable file or API.

Key Concepts

What web scraping involves

A basic scraping workflow usually has two steps:

1. Download the page content
2. Parse the HTML and extract useful data

Web scraping vs APIs

If a website offers an API, that is often the better option because it is more structured and stable. Scraping is more fragile because websites can change their layout at any time.

Common web scraping use cases

- collect product or pricing information

- capture headlines or announcements
- extract tables from public pages
- monitor website changes

Risks and responsibilities

Before scraping a website, consider:

- whether the site allows automated access
- whether requests should be rate-limited
- whether the data is public and appropriate to collect
- whether an API already exists

Always scrape responsibly.

Examples

Example 1: A simple scraping workflow description

```
steps = [ "Send an HTTP request", "Receive HTML content", "Parse the HTML", "Extract target elements" ] for step in steps: print(step)
```

Example 2: Very basic HTML extraction idea

```
html = "<h1>Daily Report</h1>" print("Downloaded HTML:", html)
```

Example 3: List possible scraping targets

```
targets = ["titles", "links", "table rows", "prices"] print(", ".join(targets))
```

Notes

- Prefer APIs over scraping when possible.
 - Web pages can change without warning, which can break a scraper.
 - Add delays and avoid excessive requests.
 - Review legal, ethical, and usage terms before scraping a site.
-

Summary

- Web scraping is the process of downloading and extracting information from web pages.
 - It is useful when data is available on a site but not through a better interface.
 - Responsible scraping requires care, restraint, and awareness of site rules.
-

Practice Exercises

1. Describe the two main steps in web scraping.
 2. Explain one case where an API is better than scraping.
 3. List three types of data that could be extracted from a webpage.
-

Further Reading

- [Requests documentation](#)
- [Beautiful Soup documentation](#)

Using BeautifulSoup and Requests libraries for web scraping

Learning Objectives

By the end of this chapter, you will be able to:

- Use `requests` to download a web page
- Parse HTML with `BeautifulSoup`
- Extract simple elements such as titles, links, and text
- Understand basic scraper structure and error handling

Introduction

Two popular Python libraries make beginner-friendly web scraping much easier:

- `requests` downloads page content
- `BeautifulSoup` parses HTML and helps extract information

Together, they support many simple automation tasks involving public web pages.

Key Concepts

Step 1: Send an HTTP request

The `requests` library is used to fetch a page:

```
import requests response = requests.get("https://example.com", timeout=10)
```

If the request succeeds, the page HTML will be in `response.text`.

Step 2: Parse the HTML

Use `BeautifulSoup` to turn raw HTML into a searchable structure:

```
from bs4 import BeautifulSoup soup = BeautifulSoup(response.text, "html.parser")
```

Step 3: Find elements

Common methods include:

- `find()`
- `find_all()`
- `get_text()`
- attribute access such as `tag["href"]`

Handling failures

A scraper should handle common problems such as:

- network errors
- timeouts
- missing elements
- unexpected HTML changes

Examples

Example 1: Fetch a page title

```
import requests from bs4 import BeautifulSoup response =
requests.get("https://example.com", timeout=10) soup = BeautifulSoup(response.text,
"html.parser") print(soup.title.get_text())
```

Example 2: Extract all links

```
import requests from bs4 import BeautifulSoup response =
requests.get("https://example.com", timeout=10) soup = BeautifulSoup(response.text,
"html.parser") for link in soup.find_all("a"): href = link.get("href") text =
link.get_text(strip=True) print(text, href)
```

Example 3: Extract headings safely

```
import requests from bs4 import BeautifulSoup response =
requests.get("https://example.com", timeout=10) soup = BeautifulSoup(response.text,
"html.parser") heading = soup.find("h1") if heading:
print(heading.get_text(strip=True)) else: print("No h1 heading found")
```

Notes

- Install required libraries with `pip install requests beautifulsoup4`.
 - Always use a timeout when making HTTP requests.
 - Check whether elements exist before accessing them.
 - Prefer simple, stable selectors over overly fragile parsing logic.
-

Summary

- `requests` downloads web pages and `BeautifulSoup` parses them.
 - Together they can extract structured information from HTML.
 - Robust scrapers should handle missing data and request failures.
-

Practice Exercises

1. Use `requests` to download `https://example.com` and print the status code.
 2. Parse the page title with `BeautifulSoup`.
 3. Print all links from a web page using `find_all("a")`.
-

Further Reading

- [Requests documentation](#)
- [Beautiful Soup documentation](#)

Module 8: GUI Automation

Introduction to GUI automation

Learning Objectives

By the end of this chapter, you will be able to:

- Define GUI automation and describe how it works
 - Identify tasks that are suitable for desktop automation
 - Recognize the strengths and weaknesses of GUI-based workflows
 - Understand common safety concerns before running GUI scripts
-

Introduction

GUI automation means controlling software through its graphical user interface, such as buttons, menus, windows, mouse movements, and keyboard input. Instead of calling an API or editing a file directly, the automation behaves like a user interacting with the screen.

GUI automation is useful when an application does not provide a cleaner integration method.

Key Concepts

What GUI automation does

A GUI automation script may:

- move the mouse
 - click buttons
 - type text
 - press keyboard shortcuts
 - switch between windows
 - take screenshots or locate elements on screen
-

When GUI automation is useful

GUI automation is often used when:

- a legacy desktop application has no API
- work must be entered into a form repeatedly
- a user interface follows stable, predictable steps
- a human process is already well documented

Why GUI automation can be fragile

GUI scripts depend on visible screen state. They may fail if:

- the window moves
- screen resolution changes
- the button text or location changes
- the application responds slowly
- a popup interrupts the workflow

Prefer stronger integrations when possible

If an API, file export, or database connection is available, it is often more reliable than GUI automation. GUI automation is usually a fallback approach when direct integration is not possible.

Examples

Example 1: A GUI workflow as steps

```
steps = [ "Open the application", "Wait for the window to load", "Click the customer field", "Type the customer ID", "Press Enter" ] for step in steps: print(step)
```

Example 2: Why timing matters

```
import time print("Opening screen...") time.sleep(2) print("Ready for next action")
```

Example 3: Compare automation options

```
options = { "API": "Most reliable if available", "File import": "Often better than GUI", "GUI automation": "Useful when no direct integration exists" } for method, note in options.items(): print(f"{method}: {note}")
```

Notes

- GUI automation should be used carefully because it is sensitive to screen changes.
 - Start with small repeatable tasks before automating long workflows.
 - Add delays and fail-safe exits when testing.
 - Document the exact screen conditions your script expects.
-

Summary

- GUI automation controls applications through visible interface actions.
 - It is useful when there is no better integration path.
 - GUI workflows must be designed carefully because they are more fragile than API or file-based automation.
-

Practice Exercises

1. Describe one process that could be automated with GUI automation.
 2. List three reasons a GUI script might fail.
 3. Explain why an API is usually preferred over GUI automation.
-

Further Reading

- [PyAutoGUI documentation](#)

Automating mouse and keyboard actions

Learning Objectives

By the end of this chapter, you will be able to:

- Use Python to move the mouse and simulate clicks
 - Send keyboard input and shortcuts programmatically
 - Add delays to make GUI automation more reliable
 - Use fail-safe practices while testing automation scripts
-

Introduction

The most direct form of GUI automation is controlling the mouse and keyboard. Libraries such as `pyautogui` can simulate actions that a human user performs on screen.

This approach is powerful, but it should be tested carefully because incorrect clicks or keystrokes can affect the wrong window.

Key Concepts

Common mouse actions

A GUI automation script may:

- move to screen coordinates
- click or double-click
- drag items
- scroll

Common keyboard actions

A script may:

- type text

- press individual keys
- send keyboard shortcuts
- navigate between fields using `Tab` or `Enter`

Timing and pauses

Applications may need time to respond. Adding small delays improves stability.

Fail-safe behavior

Tools like `pyautogui` support a fail-safe feature so rapid mouse movement to a screen corner stops the script. This is important for safe testing.

Examples

Example 1: Move and click with `pyautogui`

```
import pyautogui
pyautogui.moveTo(500, 300, duration=0.5)
pyautogui.click()
```

Example 2: Type text and press Enter

```
import pyautogui
pyautogui.write("POLICY-10025", interval=0.05)
pyautogui.press("enter")
```

Example 3: Use delays between actions

```
import pyautogui
import time
pyautogui.click(400, 250)
time.sleep(1)
pyautogui.write("Daily report")
```

Notes

- Install the library with `pip install pyautogui` before running examples.
 - Test with non-critical applications first.
 - Keep the target window visible and stable.
 - Use small pauses and fail-safe settings to reduce mistakes.
-

Summary

- Mouse and keyboard automation is a core part of GUI scripting.
 - `pyautogui` can simulate many user actions quickly.
 - Safe timing and testing practices are essential for reliable execution.
-

Practice Exercises

1. Write a script that waits briefly, then types a short message.
 2. Move the mouse to a position and click once.
 3. Simulate typing text followed by pressing Enter.
-

Further Reading

- [PyAutoGUI mouse functions](#)
- [PyAutoGUI keyboard functions](#)

Interacting with desktop applications

Learning Objectives

By the end of this chapter, you will be able to:

- Structure automation steps for desktop applications
 - Use screen coordinates, windows, and application states carefully
 - Understand simple strategies for finding UI elements
 - Reduce failures caused by timing or interface changes
-

Introduction

Desktop application automation means using Python to complete tasks inside installed software such as internal tools, spreadsheets, or legacy business systems. The challenge is not just sending clicks and keystrokes, but doing so at the right place and time.

Key Concepts

Build a predictable workflow

A reliable desktop automation script usually follows a sequence like this:

1. open the application
2. wait for the window to load
3. focus the correct window or field
4. perform input steps
5. confirm the result

Coordinate-based interaction

Some GUI scripts interact with fixed screen coordinates. This is simple but fragile if the window moves or screen resolution changes.

Image-based interaction

Some tools can locate buttons or fields from screenshots. This is often more flexible than fixed coordinates, but it can still fail if themes or layouts change.

Confirm application state

Before typing or clicking, confirm the correct application is open and ready. This reduces the risk of entering data in the wrong place.

Examples

Example 1: Plan a desktop automation sequence

```
workflow = [ "Launch the application", "Wait for login screen", "Enter username and password", "Open the claims screen", "Type the claim number", "Save the result" ] for step in workflow: print(step)
```

Example 2: Pause before interacting

```
import time print("Waiting for the application window...") time.sleep(3) print("Now continue")
```

Example 3: Validate state with a screenshot search concept

```
status = "button found" if status == "button found": print("Safe to continue") else: print("Stop and review the screen")
```

Notes

- Keep workflows short and focused when starting out.
 - Add checks between important steps.
 - Avoid running desktop automation in the background while also using the same machine.
 - If available, use window controls or application APIs instead of raw screen coordinates.
-

Summary

- Desktop automation requires both user-like actions and careful state management.

- Reliable workflows depend on timing, focus, and predictable screen layout.
 - Short, validated steps are safer than long blind click sequences.
-

Practice Exercises

1. Write a step-by-step plan for automating a simple form in a desktop app.
 2. List two risks of coordinate-based interaction.
 3. Explain why confirming application state matters before typing.
-

Further Reading

- [PyAutoGUI screenshot functions](#)

Module 9: Database Automation

Introduction to database automation

Learning Objectives

By the end of this chapter, you will be able to:

- Define database automation in a Python context
 - Identify common tasks that can be automated with databases
 - Understand the role of scripts in reading and updating stored data
 - Recognize why safety and consistency matter in database work
-

Introduction

Many automation workflows depend on data stored in databases. Instead of entering or exporting records manually, Python can connect to a database, run queries, process results, and update records automatically.

Database automation is especially useful in reporting, reconciliation, batch updates, and operational checks.

Key Concepts

What database automation means

Database automation means using scripts to perform repeatable database tasks such as:

- reading records
 - generating reports
 - inserting new rows
 - updating statuses
 - checking for missing or incorrect data
-

Why databases matter in automation

Databases store structured information efficiently. Automation scripts often rely on them for:

- daily reporting
- operational dashboards
- data cleanup
- scheduled updates
- system integration

Common automation patterns

A Python database workflow often looks like this:

1. connect to the database
2. run a query
3. fetch rows
4. process the results
5. optionally update or insert data
6. close the connection

Safety matters

Database automation can affect important records. Scripts should be designed carefully to avoid incorrect updates or accidental data loss.

Examples

Example 1: Describe a database workflow

```
steps = [ "Connect to database", "Run a SELECT query", "Fetch the rows", "Generate a summary", "Close the connection" ] for step in steps: print(step)
```

Example 2: Identify automation tasks

```
tasks = [ "Generate a daily premium report", "Find policies with missing email addresses", "Update renewal reminders", "Insert imported claim rows" ] for task in tasks: print(task)
```

Example 3: Think in records and rules

```
pending_claims = 18 print(f"Claims to review: {pending_claims}")
```

Notes

- Use database automation for repeatable, structured tasks.
 - Start with read-only queries before automating updates.
 - Test with sample data whenever possible.
 - Keep a clear record of what your automation script changed.
-

Summary

- Database automation uses Python scripts to read and update structured data.
 - It supports reporting, validation, and repetitive operational tasks.
 - Safe design and careful testing are essential when scripts modify data.
-

Practice Exercises

1. List three business tasks that could use database automation.
 2. Describe the typical steps in a Python database workflow.
 3. Explain one risk of automating database updates.
-

Further Reading

- [sqlite3 documentation](#)

Connecting to databases using Python

Learning Objectives

By the end of this chapter, you will be able to:

- Connect Python to a database using built-in or external libraries
 - Understand the role of a connection and cursor
 - Run basic queries and fetch results
 - Close database resources properly after use
-

Introduction

Before a Python script can read or update database records, it must connect to the database. The exact library depends on the database type, but the overall pattern is similar across systems.

For beginners, SQLite is a useful starting point because it is included with Python through the `sqlite3` module.

Key Concepts

Database connection

A connection represents the active link between your Python script and the database.

Cursor

A cursor lets you send SQL commands and fetch results.

Typical workflow

A common connection workflow is:

1. open a connection
 2. create a cursor if needed
 3. execute SQL statements
-

4. fetch or commit results
5. close the connection

Common libraries

- `sqlite3` for SQLite
- `psycopg2` for PostgreSQL
- `mysql-connector-python` for MySQL

Even when the library changes, the concepts of connection, execute, fetch, commit, and close remain similar.

Examples

Example 1: Connect to SQLite

```
import sqlite3 connection = sqlite3.connect("automation.db") print("Connected")
connection.close()
```

Example 2: Create a table and query rows

```
import sqlite3 connection = sqlite3.connect("automation.db") cursor =
connection.cursor() cursor.execute("CREATE TABLE IF NOT EXISTS tasks (id INTEGER
PRIMARY KEY, name TEXT)") cursor.execute("SELECT name FROM sqlite_master WHERE
type='table'") print(cursor.fetchall()) connection.close()
```

Example 3: Use a parameterized query

```
import sqlite3 connection = sqlite3.connect("automation.db") cursor =
connection.cursor() cursor.execute("CREATE TABLE IF NOT EXISTS customers (id INTEGER
PRIMARY KEY, name TEXT)") cursor.execute("INSERT INTO customers (name) VALUES (?)",
("Asha",)) connection.commit() connection.close()
```

Notes

- Always close the connection when the script is finished.
- Use parameterized queries instead of string formatting for SQL values.
- Commit changes after inserts, updates, or deletes.
- Store credentials securely when working with external databases.

Summary

- Python connects to databases through library-specific connection objects.
 - A cursor is commonly used to run SQL statements and fetch results.
 - Safe connection handling and parameterized queries are important best practices.
-

Practice Exercises

1. Connect to a SQLite database file and print a success message.
 2. Create a simple table using Python.
 3. Insert one row with a parameterized query and commit the transaction.
-

Further Reading

- [sqlite3 tutorial](#)
 - [DB-API 2.0 specification](#)
-

Automating database queries and data manipulation tasks

Learning Objectives

By the end of this chapter, you will be able to:

- Run automated SELECT, INSERT, and UPDATE operations from Python
 - Process query results in loops and simple reports
 - Use transactions and parameters more safely
 - Build small repeatable database workflows
-

Introduction

After connecting to a database, the next step is automating actual work: reading rows, inserting data, updating statuses, and generating simple outputs. Python is especially useful when the same query or update must run repeatedly.

Key Concepts

Automating queries

A script can run the same SQL query every day or every hour and then process the results automatically.

Examples:

- find claims submitted today
- list customers with missing contact data
- count unpaid invoices by branch

Automating inserts and updates

Python can insert imported data, update status values, or mark records as processed.

Parameterized SQL

Values should be passed as parameters, not built directly into SQL strings. This improves correctness and security.

Transactions and commits

For data-changing operations, call `commit()` after successful work. If something fails, you may need to roll back instead of saving partial changes.

Examples

Example 1: Query and print matching rows

```
import sqlite3 connection = sqlite3.connect("automation.db") cursor =  
connection.cursor() cursor.execute("SELECT id, name FROM customers") for row in  
cursor.fetchall(): print(row) connection.close()
```

Example 2: Insert multiple rows

```
import sqlite3 customers = [("Asha",), ("Vikram",), ("Nina",)] connection =  
sqlite3.connect("automation.db") cursor = connection.cursor() cursor.execute("CREATE  
TABLE IF NOT EXISTS customers (id INTEGER PRIMARY KEY, name TEXT)")  
cursor.executemany("INSERT INTO customers (name) VALUES (?)", customers)  
connection.commit() connection.close()
```

Example 3: Update rows safely

```
import sqlite3 connection = sqlite3.connect("automation.db") cursor =  
connection.cursor() cursor.execute("CREATE TABLE IF NOT EXISTS claims (id INTEGER  
PRIMARY KEY, status TEXT)") cursor.execute("UPDATE claims SET status = ? WHERE status  
= ?", ("reviewed", "pending")) connection.commit() connection.close()
```

Notes

- Log how many rows were inserted or updated when possible.
- Test update queries on sample data first.
- Use transactions carefully so partial failures do not create confusion.
- Keep SQL readable and focused on one job at a time.

Summary

- Python can automate repeated database queries and updates efficiently.
 - Parameterized SQL and transaction handling improve safety.
 - Small, focused scripts are often the best way to automate database tasks.
-

Practice Exercises

1. Query all rows from a sample table and print them.
 2. Insert three new rows using `executemany()`.
 3. Update a status field for matching rows and commit the change.
-

Further Reading

- [sqlite3 how-to guides](#)

Module 10: Task Scheduling and Automation

Introduction to task scheduling

Learning Objectives

By the end of this chapter, you will be able to:

- Define task scheduling in an automation context
 - Explain why automation jobs are often scheduled instead of run manually
 - Identify common recurring workflows that benefit from scheduling
 - Understand the role of timing, frequency, and reliability
-

Introduction

Writing a useful automation script is only part of the solution. In many real workflows, the script must also run at the right time: every morning, every hour, once per week, or after a specific event. Task scheduling allows automation to happen without manual triggering.

Key Concepts

What task scheduling means

Task scheduling means configuring a script or command to run automatically at a chosen time or interval.

Examples:

- generate a daily report at 8:00 AM
- archive logs every night
- check a folder every 15 minutes
- send reminders every Monday

Why scheduling matters

Scheduling helps automation become dependable and routine. It reduces the need for human memory and ensures repeated work happens on time.

Common scheduling decisions

When planning a scheduled job, think about:

- how often it should run
- what inputs must exist before it starts
- where output should be stored
- how to know whether it succeeded or failed

Scheduled automation needs visibility

A scheduled script may run with no human watching it, so logs, output files, and clear error handling are important.

Examples

Example 1: Common scheduling ideas

```
jobs = [ "Daily branch report at 08:00", "Weekly backup on Sunday", "Folder check every 15 minutes" ] for job in jobs: print(job)
```

Example 2: Represent a simple schedule in code

```
schedule_info = { "task": "Send daily summary", "frequency": "daily", "time": "08:00" } print(schedule_info)
```

Example 3: Why logs matter

```
print("A scheduled task should leave evidence that it ran")
```

Notes

- Schedule only scripts that can run unattended.
- Make input and output paths predictable.
- Add logs or status files so success is visible.
- Start with simple schedules before building more complex chains.

Summary

- Task scheduling runs automation scripts at planned times or intervals.
 - Scheduling is essential for recurring operational workflows.
 - Reliable scheduled tasks need good timing, logging, and error handling.
-

Practice Exercises

1. List three tasks that are good candidates for scheduling.
 2. Explain why logs are important for scheduled automation.
 3. Describe one daily script that should run automatically in a business workflow.
-

Further Reading

- [cron on Wikipedia](#)
 - [Windows Task Scheduler overview](#)
-

Using cron jobs and task scheduler libraries

Learning Objectives

By the end of this chapter, you will be able to:

- Understand how cron jobs schedule commands on Unix-like systems
 - Recognize the role of Windows Task Scheduler
 - Use Python scheduler libraries for in-process timing
 - Choose an appropriate scheduling tool for a given task
-

Introduction

There are several ways to schedule automation tasks. Some schedules are managed by the operating system, while others are handled by Python libraries. Choosing the right tool depends on where the script runs and how the workflow should be managed.

Key Concepts

Cron jobs

cron is a scheduler commonly used on Linux and macOS. It runs commands at specified times using cron expressions.

Example cron entry:

```
0 8 * * * /usr/bin/python3 /path/to/daily_report.py
```

This runs the script every day at 8:00 AM.

Windows Task Scheduler

On Windows, Task Scheduler provides similar functionality through a graphical interface or command-line tools.

Python scheduler libraries

Some libraries schedule jobs inside a running Python process.

Examples:

- `schedule`
- `APScheduler`

These are useful when you want scheduling logic in Python code, but they still need a running process or service.

Choosing between them

- Use OS schedulers for stable system-level jobs.
- Use Python scheduler libraries when scheduling rules belong inside the application.
- Keep the solution as simple as possible.

Examples

Example 1: Cron expression

```
30 9 * * 1-5 /usr/bin/python3 /path/to/check_claims.py
```

This runs at 9:30 AM on weekdays.

Example 2: Use the `schedule` library

```
import schedule import time def job(): print("Running scheduled task")
schedule.every().day.at("08:00").do(job) while True: schedule.run_pending()
time.sleep(1)
```

Example 3: A simple scheduled function idea

```
def generate_report(): print("Report generated")
```

Notes

- Install the `schedule` library with `pip install schedule` if you want to use it.
- Cron and Task Scheduler are better for jobs that must run even when your script is not already running.
- Use full paths in scheduler configurations to avoid path-related failures.

- Test the command manually before scheduling it.
-

Summary

- Cron and Task Scheduler are operating-system tools for scheduled execution.
 - Python libraries such as `schedule` support in-code scheduling.
 - The right scheduler depends on environment, reliability needs, and simplicity.
-

Practice Exercises

1. Explain what the cron entry `0 6 * * * script.py` is intended to represent.
 2. Write a small `schedule` example that prints a message every minute.
 3. Compare one advantage of OS-level scheduling with one advantage of a Python scheduler library.
-

Further Reading

- [schedule documentation](#)
- [APScheduler documentation](#)

Automating recurring tasks and processes

Learning Objectives

By the end of this chapter, you will be able to:

- Design recurring automation workflows with clear inputs and outputs
 - Add logging and simple reliability checks to scheduled jobs
 - Identify common recurring tasks in operations and reporting
 - Think about monitoring and recovery for unattended scripts
-

Introduction

Recurring tasks are one of the biggest opportunities in automation. If a job happens every day, every week, or every month, it is often worth turning into a scheduled process. The goal is not just to run code on a schedule, but to create a workflow that consistently produces the expected result.

Key Concepts

Common recurring automation tasks

Examples include:

- daily report generation
- weekly data backups
- hourly folder monitoring
- monthly reconciliation summaries
- recurring email notifications

Think beyond the script

A recurring process should define:

- where the input comes from

- where the output goes
- what happens if input is missing
- how success or failure is recorded

Logging and monitoring

A scheduled workflow should produce a log, status message, or output file that confirms it ran.

Failure handling

Unattended tasks may fail because of missing files, network issues, permission problems, or code errors. Even simple checks can make automation more reliable.

Examples

Example 1: Basic recurring task idea

```
def daily_summary(): print("Generate summary, save file, write log")
```

Example 2: Simple logging pattern

```
from datetime import datetime with open("run.log", "a", encoding="utf-8") as file:  
file.write(f"[{datetime.now().isoformat(timespec='seconds')}] Task ran\n")
```

Example 3: Check for a required input file

```
from pathlib import Path input_file = Path("input/daily.csv") if input_file.exists():  
print("Process input file") else: print("Input missing; stop and log the issue")
```

Notes

- Recurring automation should be predictable and easy to verify.
 - Logs, timestamps, and output folders make troubleshooting easier.
 - Handle missing input gracefully instead of failing silently.
 - Keep scheduled workflows small and modular when possible.
-

Summary

- Recurring tasks are strong candidates for automation.
 - Good scheduled workflows define timing, inputs, outputs, and failure behavior.
 - Logging and simple validation make unattended jobs much easier to manage.
-

Practice Exercises

1. Design a daily automation workflow with one input, one output, and one log file.
 2. Add a file existence check before processing a recurring input.
 3. Explain two things you would monitor in a scheduled reporting job.
-

Further Reading

- [logging documentation](#)
- [pathlib documentation](#)

Module 11: Error Handling and Logging

Handling errors and exceptions in automation scripts

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what exceptions are in Python
 - Use `try` and `except` blocks to handle failures
 - Distinguish between recoverable and non-recoverable errors
 - Make automation scripts safer and easier to troubleshoot
-

Introduction

Automation scripts often interact with files, websites, databases, and external systems. Any of these steps can fail because of missing files, invalid data, permission issues, network problems, or unexpected input. Error handling helps a script respond clearly instead of crashing without explanation.

Key Concepts

What an exception is

An **exception** is a runtime error that interrupts normal program execution.

Common examples:

- `FileNotFoundError`
- `ValueError`
- `KeyError`
- `ZeroDivisionError`
- `TimeoutError`

Basic error handling structure

Python uses `try` and `except` to catch exceptions:

```
try: risky_code() except SomeError: handle_problem()
```

This allows the script to show a meaningful message, log the problem, or continue with fallback logic.

`else` and `finally`

- `else` runs if no exception occurs
- `finally` runs whether an error occurs or not

These blocks are useful for cleanup and success handling.

Catch specific exceptions

It is better to catch specific exceptions than a broad `except Exception` unless you are logging and re-raising carefully. Specific handling makes scripts easier to debug.

Error handling in automation

Good automation error handling should:

- explain what failed
- avoid silent failure
- prevent partial or unsafe work where possible
- keep logs or messages useful for later review

Examples

Example 1: Handle a missing file

```
try: with open("input/report.txt", "r", encoding="utf-8") as file: content = file.read() except FileNotFoundError: print("Input file was not found")
```

Example 2: Use `else` and `finally`

```
try: number = int("25") except ValueError: print("Invalid number") else: print("Converted successfully:", number) finally: print("Conversion step finished")
```

Example 3: Validate data safely

```
def calculate_average(total, count): try: return total / count except
ZeroDivisionError: print("Count cannot be zero") return 0
print(calculate_average(100, 4)) print(calculate_average(100, 0))
```

Notes

- Catch only the exceptions you expect and understand.
 - Use clear error messages that identify the failed step.
 - Do not hide serious failures unless you have a recovery plan.
 - Combine error handling with logging for better visibility.
-

Summary

- Exceptions are runtime errors that interrupt a script.
 - `try`, `except`, `else`, and `finally` help control failure behavior.
 - Good automation scripts handle errors clearly and safely.
-

Practice Exercises

1. Write a script that tries to open a missing file and prints a friendly error message.
 2. Convert a string to an integer and handle a possible `ValueError`.
 3. Create a function that catches `ZeroDivisionError` and returns a safe default value.
-

Further Reading

- [Python exceptions documentation](#)

Implementing logging for debugging and monitoring

Learning Objectives

By the end of this chapter, you will be able to:

- Explain why logging is important in automation scripts
 - Use Python's `logging` module for structured messages
 - Record useful information for debugging and monitoring
 - Choose appropriate log levels for different situations
-

Introduction

When an automation script runs unattended, printed output may not be enough. Logging provides a structured way to record what happened, when it happened, and whether the script succeeded or failed. This is useful for debugging, monitoring, and audits.

Key Concepts

Why logging matters

Logging helps answer questions such as:

- Did the script run?
- Which step failed?
- How many records were processed?
- Was the output created successfully?

The `logging` module

Python includes a built-in `logging` module that supports levels and output destinations such as the console or log files.

Common levels:

- `DEBUG`
- `INFO`
- `WARNING`
- `ERROR`
- `CRITICAL`

Typical logging practice

A script might:

- log start and finish times
- log major steps
- log warnings for unusual but non-fatal conditions
- log errors when failures occur

Logging vs `print()`

`print()` is useful for quick tests, but logging is better for real automation because it is more structured and configurable.

Examples

Example 1: Basic logging setup

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Script started")
logging.warning("Input file is empty")
logging.error("Failed to connect to server")
```

Example 2: Log to a file

```
import logging
logging.basicConfig(filename="automation.log", level=logging.INFO,
format="%(%asctime)s - %(levelname)s - %(message)s" )
logging.info("Daily report generated")
```

Example 3: Logging inside error handling

```
import logging
logging.basicConfig(level=logging.INFO)
try:
    value = 10 / 0
except ZeroDivisionError:
    logging.exception("Calculation failed")
```

Notes

- Use `INFO` for normal progress messages.
 - Use `WARNING` for suspicious conditions that do not stop the script.
 - Use `ERROR` or `exception()` when something fails.
 - Keep log messages clear and specific.
-

Summary

- Logging records what happens inside an automation script.
 - The `logging` module is more useful than `print()` for production-style automation.
 - Good logs improve debugging, monitoring, and operational visibility.
-

Practice Exercises

1. Configure logging and record a script start message.
 2. Write a log message to a file with a timestamp format.
 3. Catch an exception and log it using `logging.exception()`.
-

Further Reading

- [logging documentation](#)

Module 12: Parallel Execution in Python

Threading

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what threads are in Python
 - Identify I/O-bound tasks that benefit from threading
 - Create and start threads in simple scripts
 - Understand basic thread coordination concepts
-

Introduction

Threading allows a program to work on multiple tasks that can make progress during waiting periods, such as file access, network calls, or API requests. In automation, threading is often useful for I/O-bound tasks where the script spends time waiting on external operations.

Key Concepts

What a thread is

A thread is a unit of execution within a process. Multiple threads can exist in one Python program.

When threading helps

Threading is often helpful for:

- downloading multiple files
 - calling several APIs
 - waiting on network responses
 - reading from many slow sources
-

I/O-bound vs CPU-bound

Threading is usually more effective for **I/O-bound** work than for heavy **CPU-bound** computation in standard Python.

Basic thread lifecycle

Typical steps:

1. define a target function
2. create a thread
3. start the thread
4. join the thread if you need to wait for completion

Examples

Example 1: Start one thread

```
import threading
def say_hello(): print("Hello from thread")
thread = threading.Thread(target=say_hello)
thread.start()
thread.join()
```

Example 2: Start multiple threads

```
import threading
import time
def worker(name): print(f"Starting {name}")
time.sleep(1)
print(f"Finished {name}")
threads = []
for i in range(3):
    thread = threading.Thread(target=worker, args=(f"task-{i}",))
    thread.start()
    threads.append(thread)
for thread in threads:
    thread.join()
```

Example 3: Threading for simulated waiting

```
import threading
import time
def fetch_data(source): print(f"Fetching from {source}")
time.sleep(2)
print(f"Done with {source}")
for source in ["api1", "api2"]:
    threading.Thread(target=fetch_data, args=(source,)).start()
```

Notes

- Use threading mainly for tasks that wait on I/O.
- Always think about shared state when multiple threads run together.
- Use `join()` when the main program must wait for completion.

- Keep threaded tasks small and independent when possible.
-

Summary

- Threading allows multiple tasks to make progress during waiting periods.
 - It is especially useful for I/O-bound automation work.
 - Good threaded code keeps tasks simple and coordinates completion clearly.
-

Practice Exercises

1. Create a thread that prints a message after a short delay.
 2. Launch three worker threads and wait for all of them to finish.
 3. List two automation tasks that are good candidates for threading.
-

Further Reading

- [threading documentation](#)

Multiprocessing for parallel execution

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what multiprocessing is
 - Recognize CPU-bound tasks that benefit from separate processes
 - Create worker processes in Python
 - Understand the tradeoffs of process-based parallelism
-

Introduction

Multiprocessing runs work in separate processes instead of separate threads. This is useful for CPU-heavy tasks because each process has its own Python interpreter and memory space. In automation, multiprocessing can help when many independent calculations or transformations must be performed.

Key Concepts

What multiprocessing means

A process is an independent running program instance. Multiple processes can run in parallel on different CPU cores.

When multiprocessing helps

Multiprocessing is often a better choice for:

- image transformations
- large data calculations
- CPU-heavy parsing
- expensive batch processing tasks

Process pools

The `multiprocessing.Pool` class makes it easier to distribute work across multiple processes.

Costs and tradeoffs

Multiprocessing adds overhead:

- more memory use
- more startup cost
- more complex data sharing

It should be used when the performance benefit justifies that complexity.

Examples

Example 1: Start one process

```
from multiprocessing import Process
def worker():
    print("Running in a separate process")
process = Process(target=worker)
process.start()
process.join()
```

Example 2: Use a process pool

```
from multiprocessing import Pool
def square(number):
    return number * number
with Pool(processes=2) as pool:
    results = pool.map(square, [1, 2, 3, 4])
print(results)
```

Example 3: Compare independent tasks

```
tasks = ["resize image", "compress file", "analyze dataset"]
for task in tasks:
    print(f"Independent task: {task}")
```

Notes

- Multiprocessing is best for CPU-bound work.
 - Keep tasks independent to reduce data-sharing complexity.
 - Measure performance before assuming multiprocessing is necessary.
 - On some platforms, process startup behavior requires careful script structure.
-

Summary

- Multiprocessing uses separate processes to achieve true parallel execution for CPU-bound work.
 - It is useful when independent tasks need significant computation.
 - The benefits must be weighed against extra complexity and overhead.
-

Practice Exercises

1. Create a script that starts one worker process.
 2. Use a process pool to apply a function to a list of numbers.
 3. Name two automation tasks that are better suited to multiprocessing than threading.
-

Further Reading

- [multiprocessing documentation](#)

Implementing automation best practices

Learning Objectives

By the end of this chapter, you will be able to:

- Apply safe design principles to concurrent automation workflows
 - Reduce risks around shared resources and race conditions
 - Keep parallel automation scripts understandable and maintainable
 - Choose simple, reliable concurrency approaches when appropriate
-

Introduction

Parallel execution can improve performance, but it also increases complexity. Good automation design matters even more when multiple tasks run at the same time. Best practices help you avoid subtle bugs, duplicate work, and hard-to-debug failures.

Key Concepts

Keep tasks independent

Parallel tasks are easier to manage when each unit of work is separate and does not depend heavily on shared mutable state.

Be careful with shared resources

Common shared resources include:

- files
- databases
- network connections
- shared variables

If two workers try to modify the same resource at the same time, the result can be inconsistent.

Prefer simple coordination

Simple approaches are often safer:

- divide work clearly
- collect results after workers finish
- avoid unnecessary communication between workers

Add monitoring and error visibility

Parallel automation should still log progress, failures, and important counts so problems can be traced later.

Examples

Example 1: Independent units of work

```
files = ["a.csv", "b.csv", "c.csv"] for file_name in files: print(f"Each file can be processed separately: {file_name}")
```

Example 2: Avoid shared output conflicts

```
workers = ["worker1", "worker2"] for worker in workers: print(f"{worker} should write to its own temporary output")
```

Example 3: Collect results after parallel work

```
partial_results = [10, 20, 30] total = sum(partial_results) print("Combined result:", total)
```

Notes

- Start with sequential code, then parallelize only the bottleneck.
 - Minimize shared mutable state.
 - Keep logging useful even when many workers run.
 - Choose correctness first, then optimize.
-

Summary

- Parallel automation should be designed for independence, safety, and clarity.
 - Shared resources create risk and should be handled carefully.
 - Simple coordination patterns are often the most maintainable.
-

Practice Exercises

1. List three resources that can cause conflicts in parallel automation.
 2. Explain why independent work units are easier to parallelize.
 3. Describe one way to avoid multiple workers writing to the same file.
-

Further Reading

- [concurrent.futures documentation](#)

Performance optimization techniques

Learning Objectives

By the end of this chapter, you will be able to:

- Identify common performance bottlenecks in automation scripts
 - Apply simple optimization techniques before adding complexity
 - Measure script behavior to guide improvement decisions
 - Choose appropriate performance strategies for different workloads
-

Introduction

Not every slow script needs threads or processes. Many performance problems come from inefficient file handling, repeated computations, unnecessary loops, or slow external calls. Good optimization starts with measurement and simple improvements.

Key Concepts

Find the bottleneck first

Before optimizing, determine what is actually slow:

- file I/O
- network calls
- database queries
- repeated computations
- inefficient data transformations

Simple optimizations

Useful low-complexity improvements include:

- reducing repeated work

- caching reusable values
- processing data in batches
- avoiding unnecessary file reads or writes
- using efficient libraries for tabular data

Measure execution time

Use timing tools or simple timestamps to compare approaches.

Optimize responsibly

A faster script is not better if it becomes unreliable or impossible to understand.

Examples

Example 1: Measure execution time

```
import time start = time.time() time.sleep(1) end = time.time() print(f"Elapsed time: {end - start:.2f} seconds")
```

Example 2: Avoid repeated computation

```
numbers = [1, 2, 3, 4] squares = [n * n for n in numbers] print(squares)
```

Example 3: Process data in chunks conceptually

```
rows = list(range(1, 11)) chunk_size = 3 for i in range(0, len(rows), chunk_size): print(rows[i:i + chunk_size])
```

Notes

- Measure before and after optimization.
 - Prefer simple wins before advanced concurrency.
 - Reduce unnecessary I/O when possible.
 - Keep correctness and readability as priorities.
-

Summary

- Optimization should begin with understanding the bottleneck.
 - Many automation scripts can be improved with simple changes.
 - Performance work should be guided by measurement, not guesswork.
-

Practice Exercises

1. Measure the execution time of a short Python operation.
 2. Describe two non-concurrency ways to speed up an automation script.
 3. Explain why optimization without measurement is risky.
-

Further Reading

- [time module documentation](#)
- [timeit documentation](#)

Module 13: Excel Automation

Introduction to Excel automation

Learning Objectives

By the end of this chapter, you will be able to:

- Define Excel automation in a Python context
 - Identify common tasks that can be automated in spreadsheets
 - Explain why Excel is common in business operations
 - Recognize when Python is useful alongside Excel
-

Introduction

Excel is widely used for reports, reconciliations, data entry, summaries, and operational tracking. Because of that, many automation workflows need to read data from spreadsheets, update values, or generate new Excel outputs. Python helps automate these repetitive tasks efficiently and consistently.

Key Concepts

What Excel automation means

Excel automation means using Python to work with spreadsheet files programmatically instead of editing them manually.

Common tasks include:

- reading workbook data
- updating sheets
- creating summary files
- combining multiple workbooks
- validating spreadsheet contents

Why Excel remains important

Many teams still store operational data in Excel because:

- it is familiar
- it is easy to share
- it supports tabular data well
- business users often prefer spreadsheets for review

Python's role

Python is especially useful when Excel tasks are:

- repetitive
- rule-based
- large in volume
- time-sensitive

Common libraries

Popular tools include:

- `pandas`
- `openpyxl`
- `xlrd` or compatible readers for older formats

Examples

Example 1: Describe spreadsheet automation tasks

```
tasks = [ "Read monthly sales workbook", "Clean missing values", "Calculate totals",  
"Write summary sheet" ] for task in tasks: print(task)
```

Example 2: Think of Excel as structured data

```
columns = ["branch", "premium", "claims"] print("Excel worksheet columns:", columns)
```

Example 3: Why automate Excel work

```
minutes_per_report = 20 reports_per_week = 5 print("Weekly manual effort:",  
minutes_per_report * reports_per_week, "minutes")
```

Notes

- Excel is often the starting point for operational automation.
- Keep spreadsheet structure consistent where possible.
- Prefer automation for repeated transformations, not one-time editing.
- Validate important outputs before sharing them.

Summary

- Excel automation uses Python to read, update, and generate spreadsheet files.
- It is valuable because spreadsheets remain common in real business workflows.
- Python helps make Excel work faster, more consistent, and easier to scale.

Practice Exercises

1. List three Excel tasks that could be automated with Python.
2. Explain why Excel is still common in business workflows.
3. Describe one repeated spreadsheet task you would like to automate.

Further Reading

- [pandas Excel I/O documentation](#)
- [openpyxl documentation](#)

Reading and writing Excel files using pandas library

Learning Objectives

By the end of this chapter, you will be able to:

- Read Excel files into pandas DataFrames
 - Write DataFrames back to Excel files
 - Select sheets and basic output options
 - Use pandas for simple spreadsheet automation tasks
-

Introduction

`pandas` is one of the most popular Python libraries for working with tabular data. It can read Excel sheets into DataFrames for analysis and transformation, then export the results back to Excel.

Key Concepts

Reading Excel files

Use `pd.read_excel()` to load spreadsheet data.

Common options include:

- file path
- sheet name
- selected columns
- header row

Writing Excel files

Use `DataFrame.to_excel()` to create output files.

Working with sheets

You can read a specific sheet by name or index. For multi-sheet output, `ExcelWriter` is often used.

Why DataFrames help

Once Excel data is in a `DataFrame`, you can:

- filter rows
- rename columns
- group and summarize data
- clean missing values

Examples

Example 1: Read an Excel file

```
import pandas as pd
df = pd.read_excel("reports.xlsx", sheet_name="Sheet1")
print(df.head())
```

Example 2: Write a DataFrame to Excel

```
import pandas as pd
data = { "branch": ["North", "South"], "premium": [1200, 900] }
df = pd.DataFrame(data)
df.to_excel("summary.xlsx", index=False)
```

Example 3: Write multiple sheets

```
import pandas as pd
df1 = pd.DataFrame({"name": ["Asha", "Ravi"]})
df2 = pd.DataFrame({"status": ["active", "inactive"]})
with pd.ExcelWriter("output.xlsx") as writer:
    df1.to_excel(writer, sheet_name="Customers", index=False)
    df2.to_excel(writer, sheet_name="Status", index=False)
```

Notes

- Install required support libraries such as `openpyxl` when working with `.xlsx` files.
- Use `index=False` unless you explicitly want `DataFrame` index values in the sheet.
- Check sheet names carefully when reading from existing workbooks.
- Validate column names after import.

Summary

- `pandas` makes Excel reading and writing straightforward.
 - DataFrames provide a powerful structure for spreadsheet automation.
 - Sheet selection and clean export settings improve reliability.
-

Practice Exercises

1. Read an Excel sheet into a DataFrame and print the first five rows.
 2. Create a DataFrame and save it to a new `.xlsx` file.
 3. Write two DataFrames to separate sheets in one workbook.
-

Further Reading

- [pandas read excel](#)
- [pandas to excel](#)

Handling different Excel formats (e.g., .xls, .xlsx)

Learning Objectives

By the end of this chapter, you will be able to:

- Distinguish between common Excel file formats
- Recognize compatibility issues between `.xls` and `.xlsx`
- Choose suitable tools for different workbook types
- Plan automation workflows that handle mixed Excel inputs

Introduction

Excel files do not all use the same format. Older spreadsheets may use `.xls`, while newer workbooks commonly use `.xlsx`. Automation scripts need to recognize these differences because library support and features can vary by format.

Key Concepts

Common Excel formats

- `.xls` → older Excel binary format
- `.xlsx` → newer XML-based format
- `.csv` → plain text tabular data, not a full Excel workbook format

Why format differences matter

Different formats may affect:

- which libraries can read the file
- support for formulas and styles
- speed and compatibility
- whether multiple sheets are available

Tooling considerations

In modern workflows:

- `.xlsx` is commonly handled with `openpyxl`
- `.xls` may require older or specific reader support
- conversion may be useful when legacy files are involved

Mixed-format automation

If a folder contains multiple Excel formats, scripts should detect file types before processing.

Examples

Example 1: Check file extensions

```
from pathlib import Path for file_path in Path("input").glob("*"):
print(file_path.name, file_path.suffix)
```

Example 2: Filter only `.xlsx` files

```
from pathlib import Path for file_path in Path("input").glob("*.xlsx"):
print("Process:", file_path.name)
```

Example 3: Decide processing path by extension

```
file_name = "legacy_report.xls" if file_name.endswith(".xlsx"): print("Use modern
Excel reader") elif file_name.endswith(".xls"): print("Use legacy-compatible reader")
else: print("Unsupported format")
```

Notes

- Standardize on `.xlsx` when possible for simpler workflows.
- Check library support before promising compatibility with old files.
- Treat `.csv` differently from workbook formats.
- Validate input format early in the script.

Summary

- Excel automation often involves more than one file format.
 - `.xls` and `.xlsx` differ in structure and tooling support.
 - Early format detection helps make automation more reliable.
-

Practice Exercises

1. Write a script that prints file extensions for all files in a folder.
 2. Filter a folder so only `.xlsx` files are selected.
 3. Explain one reason old `.xls` files may require extra handling.
-

Further Reading

- [pandas Excel I/O guide](#)

Extracting data from Excel worksheets and ranges

Learning Objectives

By the end of this chapter, you will be able to:

- Read data from specific worksheets
 - Extract selected ranges or subsets of spreadsheet data
 - Target rows and columns relevant to an automation task
 - Prepare worksheet data for further processing
-

Introduction

Many Excel workbooks contain more data than an automation script actually needs. A script may need one sheet, a few columns, or a specific range of rows. Extracting only relevant data makes processing faster and more accurate.

Key Concepts

Worksheet selection

Excel workbooks may contain many sheets. Your script should know which one to read by sheet name or position.

Column and row selection

When working with `pandas`, you can limit the imported data to selected columns or skip unnecessary rows.

Range-like extraction

Excel libraries may let you work with precise cell ranges, while `pandas` is often best for structured tabular regions.

Why focused extraction matters

Extracting only the needed area helps:

- reduce noise
 - improve performance
 - simplify downstream logic
-

Examples

Example 1: Read a specific worksheet

```
import pandas as pd
df = pd.read_excel("report.xlsx", sheet_name="Claims")
print(df.head())
```

Example 2: Read selected columns

```
import pandas as pd
df = pd.read_excel("report.xlsx", sheet_name="Claims",
usecols=["claim_id", "status"])
print(df)
```

Example 3: Skip rows when importing

```
import pandas as pd
df = pd.read_excel("report.xlsx", sheet_name="Claims",
skiprows=2)
print(df.head())
```

Notes

- Use clear sheet names instead of relying only on sheet position when possible.
 - Restrict columns early to keep data handling simpler.
 - Check header placement when worksheets include title rows or notes.
 - Validate that the extracted range contains the expected data.
-

Summary

- Worksheet and range extraction help target the right spreadsheet data.
 - Focused imports reduce complexity and improve script reliability.
 - `pandas` offers practical options for selecting sheets, columns, and rows.
-

Practice Exercises

1. Read one named worksheet from an Excel file.
2. Import only two selected columns from a sheet.
3. Skip title rows at the top of a worksheet and load the table below them.

Further Reading

- [pandas read_excel parameters](#)

Automating data manipulation and analysis tasks with Excel files

Learning Objectives

By the end of this chapter, you will be able to:

- Clean and transform Excel data with Python
 - Perform simple summaries and aggregations
 - Automate repeated spreadsheet analysis tasks
 - Create reusable workflows for Excel-based reporting
-

Introduction

Reading an Excel file is only the beginning. In real automation work, scripts often need to clean the data, calculate metrics, filter records, and export a useful summary. Python is especially strong at turning raw spreadsheet data into structured outputs.

Key Concepts

Common transformations

Automation tasks often include:

- removing empty rows
- renaming columns
- converting data types
- filtering records
- grouping and summarizing values

Analysis with `pandas`

DataFrames make it easy to automate business analysis such as totals, counts, averages, and grouped summaries.

Reusable reporting workflows

A useful pattern is:

1. load workbook data
2. clean and validate it
3. apply business rules
4. export the result

Why consistency matters

Automated spreadsheet analysis is most reliable when input files follow a stable structure.

Examples

Example 1: Filter rows

```
import pandas as pd
df = pd.DataFrame({ "branch": ["North", "South", "North"],
                    "premium": [1200, 900, 1500] })
north_df = df[df["branch"] == "North"]
print(north_df)
```

Example 2: Group and summarize

```
import pandas as pd
df = pd.DataFrame({ "branch": ["North", "South", "North"],
                    "premium": [1200, 900, 1500] })
summary = df.groupby("branch")["premium"].sum()
print(summary)
```

Example 3: Clean missing values

```
import pandas as pd
df = pd.DataFrame({ "name": ["Asha", None, "Ravi"], "status":
                    ["active", "inactive", None] })
clean_df = df.fillna("unknown")
print(clean_df)
```

Notes

- Keep cleaning and transformation steps explicit.
- Validate important columns before running analysis.
- Save outputs with clear file names or dates.
- Test the workflow on representative sample workbooks.

Summary

- Python can automate spreadsheet cleaning, transformation, and summary generation.
 - `pandas` is especially effective for repeated Excel analysis tasks.
 - Reliable Excel automation depends on both code and stable input structure.
-

Practice Exercises

1. Filter an Excel dataset to keep only rows matching one branch.
 2. Group data by a category column and calculate a sum.
 3. Replace missing values in a DataFrame with a default label.
-

Further Reading

- [pandas user guide](#)

Module 14: PDF Automation

Reading and extracting data from PDF files using PyPDF2 or pdfplumber library

Learning Objectives

By the end of this chapter, you will be able to:

- Read PDF files in Python
 - Extract text from pages using common PDF libraries
 - Recognize the difference between readable text PDFs and scanned PDFs
 - Choose an appropriate extraction tool for simple automation tasks
-

Introduction

PDF files are widely used for invoices, statements, reports, and official documents. Automation often requires reading these files to extract text, check values, or prepare data for later processing. Python libraries such as `PyPDF2` and `pdfplumber` help with many text-based PDF extraction tasks.

Key Concepts

PDFs are not always easy to parse

A PDF is designed for display, not always for structured extraction. Text order, spacing, and layout can vary.

Text PDFs vs scanned PDFs

- **Text PDF:** contains selectable text that libraries can often extract
- **Scanned PDF:** contains images of pages and may require OCR instead of direct text extraction

Common libraries

- PyPDF2 for basic PDF reading and manipulation
- pdfplumber for more detailed text extraction and page analysis

Extraction expectations

PDF extraction works best when:

- the PDF contains actual text
 - page layout is relatively consistent
 - the script targets clear labels or patterns
-

Examples

Example 1: Read text with PyPDF2

```
from PyPDF2 import PdfReader reader = PdfReader("report.pdf") first_page =  
reader.pages[0] text = first_page.extract_text() print(text)
```

Example 2: Extract text with pdfplumber

```
import pdfplumber with pdfplumber.open("report.pdf") as pdf: first_page =  
pdf.pages[0] text = first_page.extract_text() print(text)
```

Example 3: Loop through pages

```
from PyPDF2 import PdfReader reader = PdfReader("report.pdf") for page_number, page  
in enumerate(reader.pages, start=1): print(f"Page {page_number}")  
print(page.extract_text())
```

Notes

- Install needed libraries before use, such as `pip install PyPDF2 pdfplumber`.
 - Test extraction on real sample PDFs because layout quality varies.
 - If the PDF is scanned, direct text extraction may fail and OCR may be required.
 - Use pattern matching after extraction to locate specific fields.
-

Summary

- Python can read and extract text from many PDF files.
 - `PyPDF2` and `pdfplumber` are useful tools for text-based PDFs.
 - PDF extraction quality depends heavily on the document structure.
-

Practice Exercises

1. Open a PDF file and extract the first page's text.
 2. Loop through all pages of a PDF and print their text.
 3. Explain why scanned PDFs may need OCR instead of direct extraction.
-

Further Reading

- [PyPDF2 documentation](#)
- [pdfplumber documentation](#)

Automating PDF generation and manipulation tasks

Learning Objectives

By the end of this chapter, you will be able to:

- Generate simple PDF outputs from Python
- Perform basic PDF manipulation tasks such as merge or split
- Understand common document automation use cases
- Design repeatable PDF workflows for reports and documents

Introduction

Automation does not only read PDFs; it can also generate and manipulate them. Common workflows include creating reports, merging documents, splitting large PDFs into smaller files, or assembling document packets for operations.

Key Concepts

Common PDF automation tasks

Examples include:

- generate a report PDF
- merge multiple PDFs
- split a multi-page PDF
- reorder pages
- add or extract selected pages

Libraries and tools

Python libraries can help with PDF creation and manipulation, depending on the task.

Examples:

- PyPDF2 for merge and split work
- document-generation libraries such as `reportlab` for creating PDFs

Reliable document workflows

A good PDF automation process should define:

- input files
 - output naming rules
 - page or section logic
 - validation checks
-

Examples

Example 1: Merge PDFs with `PyPDF2`

```
from PyPDF2 import PdfMerger merger = PdfMerger() merger.append("part1.pdf")
merger.append("part2.pdf") merger.write("merged.pdf") merger.close()
```

Example 2: Split pages from a PDF

```
from PyPDF2 import PdfReader, PdfWriter reader = PdfReader("report.pdf") writer = PdfWriter()
writer.add_page(reader.pages[0]) with open("first_page.pdf", "wb") as output_file: writer.write(output_file)
```

Example 3: Describe document generation steps

```
steps = ["Collect data", "Format content", "Create PDF", "Save output"] for step in steps: print(step)
```

Notes

- Use predictable output names for generated PDFs.
 - Validate page counts and output files after manipulation.
 - Test merge and split workflows on sample documents.
 - Keep source files unchanged when possible.
-

Summary

- Python can automate PDF generation and document manipulation tasks.
 - Merge, split, and structured output workflows are common use cases.
 - Clear inputs, outputs, and checks improve PDF automation reliability.
-

Practice Exercises

1. Merge two PDF files into one output PDF.
 2. Extract the first page of a PDF into a new file.
 3. Describe one business process that could benefit from automated PDF generation.
-

Further Reading

- [PyPDF2 user guide](#)
- [ReportLab documentation](#)

Converting PDF files to other formats (e.g., text, images)

Learning Objectives

By the end of this chapter, you will be able to:

- Explain why PDF conversion is useful in automation
 - Convert PDF content into text or images conceptually and programmatically
 - Recognize tools commonly used for PDF conversion workflows
 - Understand when conversion is needed before further analysis
-

Introduction

Sometimes a PDF is not the final format you need. An automation process may need plain text for parsing, images for visual review, or page images for OCR. Converting PDFs into other formats makes those next steps easier.

Key Concepts

Why convert PDFs

Conversion can help when you need to:

- parse text more easily
- archive page images
- prepare documents for OCR
- feed pages into image-processing workflows

Common conversion targets

- PDF to text
- PDF to images
- PDF to extracted page-level content

Tooling ideas

Depending on the task, conversion may involve:

- text extraction libraries
- PDF rendering tools
- OCR pipelines for scanned pages

Conversion limits

Conversion quality depends on:

- document structure
- image quality
- fonts and layout
- whether the PDF is text-based or scanned

Examples

Example 1: Save extracted PDF text to a file

```
from PyPDF2 import PdfReader reader = PdfReader("report.pdf") text =
"\n".join(page.extract_text() or "" for page in reader.pages) with open("report.txt",
"w", encoding="utf-8") as file: file.write(text)
```

Example 2: Conceptual page image conversion workflow

```
steps = [ "Open PDF", "Render each page as an image", "Save images for OCR or review"
] for step in steps: print(step)
```

Example 3: Decide next step after conversion

```
pdf_type = "scanned" if pdf_type == "scanned": print("Convert pages to images and run
OCR") else: print("Extract text directly")
```

Notes

- Text extraction is often simpler than full visual conversion when the PDF already contains text.
- Scanned PDFs often require conversion plus OCR.

- Test conversion output quality before building downstream automation.
 - Save converted outputs in organized folders.
-

Summary

- Converting PDFs to text or images supports many automation workflows.
 - The right conversion path depends on whether the PDF is text-based or scanned.
 - Conversion quality should be validated before further processing.
-

Practice Exercises

1. Extract text from a PDF and save it to a `.txt` file.
 2. Describe a workflow for converting scanned PDF pages to images for OCR.
 3. Explain one reason image conversion may be useful even when the final input is a PDF.
-

Further Reading

- [PyPDF2 documentation](#)
- [pytesseract project](#)

Module 15: Image Processing with Python

Automating image manipulation tasks (e.g., resizing, cropping, filtering)

Learning Objectives

By the end of this chapter, you will be able to:

- Automate common image transformations with Python
 - Resize, crop, and apply simple filters to images
 - Process multiple images using repeatable rules
 - Prepare image files for later automation steps
-

Introduction

Image files often need preprocessing before they can be shared, analyzed, or passed into later automation steps. For example, images may need to be resized for a website, cropped to remove unwanted borders, or filtered to improve readability. Python can automate these tasks efficiently.

Key Concepts

Common image-processing tasks

Typical automation tasks include:

- resizing images
- cropping images
- rotating images
- converting formats
- applying filters or enhancements

Why preprocessing matters

Preprocessing helps make later steps more consistent, especially if the images will be used for OCR, recognition, or reporting.

Useful libraries

A common choice is the `Pillow` library (`PIL`), which supports many basic image operations.

Batch processing pattern

A simple pattern is:

1. load input images
2. apply one or more transformations
3. save processed outputs

Examples

Example 1: Resize an image

```
from PIL import Image image = Image.open("photo.jpg") resized = image.resize((300, 200)) resized.save("photo_resized.jpg")
```

Example 2: Crop an image

```
from PIL import Image image = Image.open("photo.jpg") cropped = image.crop((50, 50, 250, 200)) cropped.save("photo_cropped.jpg")
```

Example 3: Process multiple images in a folder

```
from pathlib import Path from PIL import Image for image_path in Path("input_images").glob("*.jpg"): image = Image.open(image_path) resized = image.resize((200, 200)) output_path = Path("output_images") / image_path.name resized.save(output_path)
```

Notes

- Install Pillow with `pip install pillow` before running examples.
- Keep original image files unchanged during testing.

- Use consistent output dimensions when processing batches.
 - Validate visual quality after transformations.
-

Summary

- Python can automate many repetitive image-processing tasks.
 - Resizing, cropping, and batch transformation are common workflow steps.
 - Preprocessed images are often easier to analyze or share later.
-

Practice Exercises

1. Resize one image and save the result with a new file name.
 2. Crop an image to a selected rectangle.
 3. Process all `.jpg` files in a folder and save resized copies.
-

Further Reading

- [Pillow documentation](#)

Working with image recognition and OCR (Optical Character Recognition) for automation

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what OCR is and when it is useful
 - Recognize how image recognition supports automation
 - Use Python tools conceptually and practically for OCR workflows
 - Identify image-quality factors that affect recognition accuracy
-

Introduction

Some automation tasks require extracting text from images or scanned documents. OCR converts visible text in an image into machine-readable text. This is useful for invoices, forms, screenshots, scanned PDFs, and document images that cannot be read directly as plain text.

Key Concepts

What OCR does

OCR analyzes an image and attempts to detect and convert characters into text.

Common automation uses

OCR is useful for:

- reading scanned forms
- extracting invoice numbers
- capturing text from screenshots
- processing scanned reports or receipts

Image recognition vs OCR

- **OCR** focuses on reading text from images
- **Image recognition** may identify objects, shapes, logos, or template matches

Why image quality matters

OCR accuracy depends heavily on:

- resolution
 - contrast
 - skew or rotation
 - background noise
 - font clarity
-

Examples

Example 1: OCR workflow idea

```
steps = [ "Load image", "Preprocess image", "Run OCR engine", "Clean extracted text" ]  
for step in steps: print(step)
```

Example 2: Use `pytesseract` conceptually

```
from PIL import Image  
import pytesseract  
image = Image.open("invoice.png")  
text = pytesseract.image_to_string(image)  
print(text)
```

Example 3: Improve OCR readiness conceptually

```
quality_checks = ["high contrast", "clear text", "not blurry"]  
print("OCR works best when images are:", quality_checks)
```

Notes

- OCR usually works better after image preprocessing.
 - Install both the Python package and the OCR engine when using tools such as Tesseract.
 - Expect imperfect results and plan for validation where needed.
 - Use OCR only when direct text sources are unavailable.
-

Summary

- OCR converts text from images into machine-readable text.
 - It is useful for scanned documents, screenshots, and image-based records.
 - Good image quality and preprocessing strongly affect OCR accuracy.
-

Practice Exercises

1. Describe the steps in a simple OCR workflow.
 2. Explain two image-quality factors that affect OCR accuracy.
 3. Name one business use case where OCR would help automate data extraction.
-

Further Reading

- [pytesseract project](#)
- [Tesseract OCR](#)

Module 16: Pandas and DataFrames

Introduction to NumPy and Arrays

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the role of NumPy in Python data science and automation.
- Create 1D and 2D NumPy arrays from standard Python lists.
- Retrieve array attributes such as shape, dimensions, data type, and size.

Introduction

NumPy (Numerical Python) is the foundational package for scientific computing in Python. It provides a powerful N-dimensional array object, tools for integrating C/C++ code, and advanced mathematical functions. Pandas builds directly on top of NumPy, making it essential to understand how arrays behave before working with DataFrames.

Key Concepts

Why NumPy?

Standard Python lists are highly flexible but slow for numerical processing. NumPy arrays (`ndarrays`) store elements in contiguous blocks of memory, allowing vectorization and fast mathematical operations.

1D Arrays

A one-dimensional array is like a single row or column of numbers (a vector).

2D Arrays

A two-dimensional array is a grid of numbers containing rows and columns (a matrix).

Array Attributes

NumPy arrays expose key properties to inspect their structure:

- **shape**: A tuple representing the size of the array along each dimension (e.g., `(rows, columns)`).
- **ndim**: The number of dimensions or axes (e.g., `1` for a 1D vector, `2` for a 2D matrix).
- **dtype**: The data type of the array elements. Unlike Python

lists, NumPy arrays must have homogeneous types. - **size**: The total number of elements in the array.

Examples

Example 1: Create a 1D NumPy Array

```
import numpy as np # Create a 1D array from a Python list arr1 = np.array([1, 2, 3, 4, 5]) print("1D Array:") print(arr1)
```

Example 2: Create a 2D NumPy Array

```
import numpy as np # Create a 2D array (matrix) arr2 = np.array([ [1, 2, 3], [4, 5, 6] ]) print("2D Array:") print(arr2)
```

Example 3: Inspect Array Attributes

```
import numpy as np arr2 = np.array([ [1, 2, 3], [4, 5, 6] ]) print("Shape:", arr2.shape) # Output: (2, 3) print("Dimensions:", arr2.ndim) # Output: 2 print("Data Type:", arr2.dtype) # Output: int64 (or int32 depending on OS) print("Size:", arr2.size) # Output: 6
```

Notes

- NumPy is typically imported as `np` by convention.
 - All elements in a NumPy array must be of the same data type. If you mix floats and integers, NumPy will automatically "upcast" them (e.g. converting integers to floats).
 - Accessing attributes like `shape` does not require parentheses because they are attributes, not methods.
-

Summary

- NumPy provides high-performance homogeneous numerical arrays.
 - Create arrays from Python lists using `np.array()`.
 - Inspect structures using `.shape`, `.ndim`, `.dtype`, and `.size`.
-

Practice Exercises

1. Create a 1D NumPy array with the numbers 10, 20, 30, 40, 50 and print its shape.
2. Create a 2D NumPy array with 3 rows and 3 columns containing numbers 1 through 9. Print its dimensions (`ndim`) and total size.
3. Create an array containing both integers and floating-point numbers. Print its `dtype` to see how NumPy handles mixed types.

Further Reading

- [NumPy Array Creation Quickstart](#)
- [NumPy ndarray Attributes](#)

NumPy Operations and Math

Learning Objectives

By the end of this chapter, you will be able to:

- Perform element-wise mathematical operations on NumPy arrays.
- Calculate basic statistics (mean, median, sum, min, max) using NumPy functions.
- Reshape arrays between different dimensions.
- Use built-in helper functions to generate arrays automatically.

Introduction

NumPy arrays allow fast mathematical computations without writing `for` loops. This capability is called vectorization. In this chapter, we will learn how to perform element-wise arithmetic, compute key summary statistics, reshape arrays to change their layout, and generate synthetic data arrays.

Key Concepts

Element-Wise Mathematical Operations

When you apply standard arithmetic operators (+, -, *, /) to two arrays of the same shape, NumPy performs the operation on corresponding elements.

Reshaping Arrays

Use `.reshape(rows, cols)` to change the dimensions of an array without changing its data. The total number of elements must remain the exact same.

Automated Array Generation

NumPy provides functions to initialize arrays quickly:

- `np.zeros(shape)`: Creates an array filled with zeros.
- `np.ones(shape)`: Creates an array filled with ones.
- `np.arange(start, stop, step)`: Returns evenly spaced values within a given interval (similar to Python's `range()`).
- `np.linspace(start, stop, num)`: Returns evenly spaced numbers over a specified interval.

Statistical Functions

NumPy has fast built-in functions to compute statistics across a whole array or along specific axes:

- `np.mean()`: Arithmetic mean.
- `np.median()`: Median value.
- `np.max()` / `np.min()`: Maximum and

minimum values. - `np.sum()`: Sum of elements.

Examples

Example 1: Mathematical Operations

```
import numpy as np arr_a = np.array([10, 20, 30]) arr_b = np.array([1, 2, 3])
print("Addition:", arr_a + arr_b) # Output: [11, 22, 33] print("Subtraction:", arr_a
- arr_b) # Output: [9, 18, 27] print("Multiplication:", arr_a * arr_b) # Output: [10,
40, 90] print("Division:", arr_a / arr_b) # Output: [10.0, 10.0, 10.0]
```

Example 2: Statistical Functions

```
import numpy as np numbers = np.array([10, 20, 30, 40, 50]) print("Mean:",
np.mean(numbers)) # Output: 30.0 print("Median:", np.median(numbers)) # Output: 30.0
print("Maximum:", np.max(numbers)) # Output: 50 print("Minimum:", np.min(numbers)) #
Output: 10 print("Sum:", np.sum(numbers)) # Output: 150
```

Example 3: Reshaping Arrays

```
import numpy as np arr = np.array([1, 2, 3, 4, 5, 6]) reshaped = arr.reshape(2, 3)
print("Reshaped Array:") print(reshaped) # Output: # [[1 2 3] # [4 5 6]]
```

Example 4: Generating Arrays Automatically

```
import numpy as np # 2x2 array of zeros print("Zeros:\n", np.zeros((2, 2))) # 3x3
array of ones print("Ones:\n", np.ones((3, 3))) # Range from 1 to 9 (exclusive)
print("Arange:", np.arange(1, 10)) # 5 linear points between 1 and 100
print("Linspace:", np.linspace(1, 100, 5))
```

Notes

- When reshaping, the product of the new dimensions must equal the total size of the original array (e.g. an array of size 6 can be reshaped to (2, 3), (3, 2), (1, 6), or (6, 1)).
- NumPy statistical functions can be called as functions (e.g., `np.mean(arr)`) or as methods on the array object (e.g., `arr.mean()`).

Summary

- Basic operators (+, -, *, /) act element-wise on arrays of matching shape.

- Use `np.mean`, `np.median`, `np.sum`, `np.max`, and `np.min` for rapid calculations.
 - Reshape layouts using `.reshape()`.
 - Automatically initialize arrays using `zeros()`, `ones()`, `arange()`, or `linspace()`.
-

Practice Exercises

1. Create a 1D array of 6 elements from 10 to 60. Calculate its mean and sum.
 2. Reshape the 6-element array from Exercise 1 into a 2x3 2D array. Print the reshaped array.
 3. Automatically generate an array containing 10 numbers evenly spaced between 0 and 1.
 4. Multiply all elements of a 1D array `np.array([1, 2, 3, 4])` by 5 and print the result.
-

Further Reading

- [NumPy Mathematical Functions](#)
- [NumPy Array Reshaping](#)
- [NumPy Statistics](#)

Introduction to Pandas and DataFrames

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what Pandas is and how it represents tabular data.
- Create a DataFrame from standard Python dictionaries.
- Inspect a DataFrame's structure, shape, columns, and data types.
- Select single and multiple columns from a DataFrame.

Introduction

Pandas is the standard library for data manipulation and analysis in Python. It provides fast, flexible, and expressive data structures designed to make working with relational or labeled data both easy and intuitive. In this chapter, we will look at the primary Pandas structure—the **DataFrame**—and learn how to inspect and navigate its columns.

Key Concepts

What is a DataFrame?

A DataFrame is a 2-dimensional labeled data structure, similar to a database table or an Excel spreadsheet. It consists of:

- **Columns**: Variables or features of the data.
- **Index**: Unique row identifiers (numbers starting from 0 by default).
- **Cells**: Individual data values.

Creating DataFrames

We can easily convert raw Python data, such as a dictionary of lists, into a Pandas DataFrame using `pd.DataFrame(data)`.

Inspecting Data

When loading a new dataset, use these basic inspection functions to understand its shape and contents:

- **head(n)**: Returns the first n rows of the DataFrame (defaults to 5).
- **tail(n)**: Returns the last n rows of the DataFrame (defaults to 5).
- **columns**: Returns a list of the column headers.
- **dtypes**: Shows the data type of each column.
- **info()**: Prints a detailed summary of the dataset including columns, non-null counts, types, and memory usage.
- **describe()**: Computes summary statistics (count, mean, standard deviation, min, max, quartiles) for numeric columns.

Selecting Columns

- Select a single column using bracket syntax: `df["column_name"]` (returns a Pandas **Series**).
- Select multiple columns by passing a list of column names inside brackets: `df[["col1", "col2"]]` (returns a new **DataFrame**).

Examples

Example 1: Create a DataFrame

```
import pandas as pd
data = { "name": ["Amol", "Snehal", "Rahul", "Priya", "Amit"],
        "salary": [500000, 250000, 700000, 450000, 300000],
        "city": ["Pune", "Mumbai", "Delhi", "Pune", "Bangalore"],
        "experience": [5, 2, 7, 4, 3] }
df = pd.DataFrame(data)
print(df)
```

Example 2: Inspect the DataFrame

```
import pandas as pd
# Assume 'df' is defined as in Example 1
print("--- Head ---")
print(df.head(2))
print("\n--- Columns ---")
print(df.columns)
print("\n--- Data Types ---")
print(df.dtypes)
print("\n--- Info Summary ---")
df.info()
print("\n--- Summary Statistics ---")
print(df.describe())
```

Example 3: Selecting Columns

```
import pandas as pd
# Assume 'df' is defined as in Example 1
# Select single column
names = df["name"]
print("Single Column (Series):\n", names)
# Select multiple columns
subset = df[["name", "salary"]]
print("\nMultiple Columns (DataFrame):\n", subset)
```

Notes

- Selecting a single column like `df["name"]` returns a 1D Pandas **Series**, while passing a list like `df[["name"]]` returns a 2D **DataFrame**.
- DataFrames align data automatically based on their indexes.

Summary

- Pandas DataFrames store tabular data in rows and columns.
- Use `pd.DataFrame(dict)` to turn lists of dictionaries or dictionaries of lists into tables.
- Query datasets using `.head()`, `.tail()`, `.info()`, and `.describe()`.

- Slice columns out of a DataFrame using single brackets `df["col"]` or double brackets `df[["col1", "col2"]]`.

Practice Exercises

1. Create a DataFrame containing details of 3 products: `Product Name`, `Price` (numeric), and `Stock` (integer).
2. Print the first 2 rows of your product DataFrame using `head()`.
3. Select and print only the `Product Name` column.
4. Select and print both the `Product Name` and `Price` columns together.
5. Inspect the summary statistics of the numeric columns in your DataFrame using `describe()`.

Further Reading

- [Pandas Getting Started Tutorials](#)
- [Pandas DataFrame reference](#)

Data Selection and Filtering

Learning Objectives

By the end of this chapter, you will be able to:

- Filter DataFrame rows using single and multiple logical conditions.
- Select data cells, rows, and columns using label-based `.loc`.
- Select data cells, rows, and columns using integer-based `.iloc`.
- Distinguish between index-based and label-based indexing.

Introduction

Finding specific data in large datasets is a core task in automation. Pandas offers robust filtering features. In this chapter, we will learn how to write boolean filters to extract subsets of rows, and how to query rows and columns using labels (`.loc`) or integer indices (`.iloc`).

Key Concepts

Filtering Data (Boolean Masking)

To filter rows in a DataFrame, we pass a boolean condition inside the brackets: `df[condition]`. - For single conditions: `df[df["column"] > value]` - For multiple conditions: Combine them using bitwise logical operators like `&` (AND) or `|` (OR). You **must** wrap each condition in parentheses: `df[(cond1) & (cond2)]`.

Label-Based Indexing with `.loc`

`.loc` matches elements by row index labels and column names. - Syntax: `df.loc[row_label, column_label]` - Supports slicing, lists, and boolean criteria: - `df.loc[0]` gets the first row (if label is 0). - `df.loc[0, "name"]` gets the "name" cell in the row labeled 0. - `df.loc[:, ["name", "salary"]]` gets all rows for specified columns. - `df.loc[0:2]` gets rows labeled 0 through 2 **inclusive**.

Integer-Based Indexing with `.iloc`

`.iloc` matches elements by their physical integer position (0-indexed). - Syntax: `df.iloc[row_position, column_position]` - Matches by position rather than label: - `df.iloc[0]` gets the first row. - `df.iloc[0, 1]` gets the value in the first row, second column. - `df.iloc[0:3, 0:2]` gets first 3 rows (indices 0, 1, 2) and first 2 columns (indices 0, 1). Note that `.iloc` slices are **exclusive** of the stop index, matching standard Python slicing.

Examples

Example 1: Filtering Data on Single and Multiple Conditions

```
import pandas as pd
data = { "name": ["Amol", "Snehal", "Rahul", "Priya", "Amit"],
"salary": [500000, 250000, 700000, 450000, 300000], "city": ["Pune", "Mumbai",
"Delhi", "Pune", "Bangalore"], "experience": [5, 2, 7, 4, 3] }
df = pd.DataFrame(data)
# Single condition: salary > 400000
high_salary = df[df["salary"] > 400000]
print("High Salary Employees:\n", high_salary)
# Multiple conditions: salary > 300000 AND city is Pune
filtered = df[(df["salary"] > 300000) & (df["city"] == "Pune")]
print("\nFiltered (Salary > 300k & City is Pune):\n", filtered)
```

Example 2: Selecting with `.loc`

```
import pandas as pd
# Using the same df # 1. Access first row by label
print("First Row:\n", df.loc[0])
# 2. Specific cell value
print("\nFirst Row Name:", df.loc[0, "name"])
# 3. All rows, selected columns
print("\nNames and Salaries:\n", df.loc[:, ["name", "salary"]])
# 4. Multiple rows (inclusive slicing)
print("\nRows 0 to 2:\n", df.loc[0:2, ["name", "city"]])
```

Example 3: Selecting with `.iloc`

```
import pandas as pd
# Using the same df # 1. First row by position
print("First Row:\n", df.iloc[0])
# 2. First row, second column (salary)
print("\nFirst Row, Second Column:", df.iloc[0, 1])
# 3. Position-based slice: first 3 rows and first 2 columns (exclusive)
print("\nSlicing with iloc:\n", df.iloc[0:3, 0:2])
```

Notes

- **Crucial difference:** Slicing with `.loc` is **inclusive** (e.g. `0:2` returns rows 0, 1, and 2). Slicing with `.iloc` is **exclusive** (e.g. `0:3` returns rows 0, 1, and 2, but not 3).
- Always use parentheses for combined boolean operations: `df[(df["A"] > 1) & (df["B"] < 3)]` (using `and` or `or` directly will fail).

Summary

- Filter rows by passing boolean vectors into bracket indexing `df[...]`.
- Sift datasets by row/column labels using `.loc[row_label, col_label]`.
- Slice datasets by numerical positions using `.iloc[row_idx, col_idx]`.

Practice Exercises

1. Create a DataFrame representing 5 books with columns: `Title`, `Author`, `Price`, and `Genre`.
 2. Filter the books to find all books priced under \$20.
 3. Filter the books to find all books that are in the "Fiction" genre AND priced under \$15.
 4. Using `.loc`, print the `Title` and `Price` of the first book in the dataset (row index 0).
 5. Using `.iloc`, slice the DataFrame to get the first 2 rows and the first 2 columns.
-

Further Reading

- [Pandas Indexing and Selecting Data](#)
- [loc API reference](#)
- [iloc API reference](#)

Data Manipulation and Cleaning

Learning Objectives

By the end of this chapter, you will be able to:

- Update values inside specific cells in a DataFrame.
- Add new calculated columns and delete existing columns.
- Sort DataFrame rows by one or more columns.
- Group rows based on matching values and calculate aggregations.
- Check for and fill in missing (NaN) values.

Introduction

In typical automation tasks, raw data is rarely perfect. You will often need to modify values, add calculated fields (such as taxes or bonuses), delete unused columns, sort reports, and summarize data by category. In this chapter, we will master DataFrame updates, sorting, groupings, and data cleaning.

Key Concepts

Updating Data

You can modify specific cells by referencing their label indices using `.loc[row, col] = new_value`.

Adding and Deleting Columns

- **Add:** Assign a series of values or a calculated expression directly: `df["new_col"] = df["col1"] * 0.10`.
- **Delete:** Use `.drop("col_name", axis=1)`. The `axis=1` parameter indicates that you are dropping a column rather than a row. By default, `.drop()` returns a new DataFrame copy; to modify the original in-place, pass `inplace=True` or reassign the result.

Sorting Data

Use `df.sort_values(by="col_name", ascending=True)` to sort your rows. Set `ascending=False` for descending order.

Grouping and Aggregating (groupby)

Use `.groupby("col1")` to group data by matching categories in "col1", then chain an aggregation function (like `.mean()`, `.sum()`, `.count()`) on target columns. E.g. `df.groupby("city")["salary"].mean()` calculates the average salary for each city.

Handling Missing Values

In Pandas, missing data is represented as `NaN` (Not a Number) or `None`. - **Detect**: Use `df.isnull()` to return a boolean table where `True` marks missing cells. - **Fill**: Use `df.fillna(value)` to replace all missing values with a default choice (like 0 or the mean of the column).

Examples

Example 1: Updating Data and Adding/Deleting Columns

```
import pandas as pd
data = { "name": ["Amol", "Snehal", "Rahul", "Priya", "Amit"],
        "salary": [500000, 250000, 700000, 450000, 300000], "city": ["Pune", "Mumbai", "Delhi", "Pune", "Bangalore"], "experience": [5, 2, 7, 4, 3] }
df = pd.DataFrame(data)
# 1. Update salary of row 0
df.loc[0, "salary"] = 600000
print("Updated Salary for Amol:\n", df.head(1))
# 2. Add 'bonus' column (10% of salary)
df["bonus"] = df["salary"] * 0.10
print("\nWith Bonus Column:\n", df)
# 3. Delete 'bonus' column
new_df = df.drop("bonus", axis=1)
print("\nAfter Dropping Bonus:\n", new_df)
```

Example 2: Sorting and Grouping Data

```
import pandas as pd
# Using the df from Example 1 (after updating Amol's salary to 600k)
# 1. Sort by salary descending
sorted_df = df.sort_values(by="salary", ascending=False)
print("Sorted by Salary (Descending):\n", sorted_df)
# 2. Group by city and find average salary
grouped = df.groupby("city")["salary"].mean()
print("\nAverage Salary by City:\n", grouped)
```

Example 3: Handling Missing Values

```
import pandas as pd
import numpy as np
# Create a DataFrame with missing data
missing_data = { "name": ["Amol", "Snehal", None], "salary": [500000, None, 700000] }
missing_df = pd.DataFrame(missing_data)
print("Original Table with Missing Data:\n", missing_df)
# Check for missing values
print("\nIs Null Check:\n", missing_df.isnull())
# Fill missing values (replacing NaN with 0)
filled_df = missing_df.fillna(0)
print("\nFilled Table (NaN replaced with 0):\n", filled_df)
```

Notes

- Slicing methods like `drop()` and `sort_values()` return copies of the data by default. To modify the DataFrame in-place, pass `inplace=True`.

- Be careful with `fillna(0)` on numeric data. In statistics, replacing missing cells with 0 can distort mean and standard deviation calculations. Instead, consider filling with the column's mean: `df["salary"].fillna(df["salary"].mean())`.
-

Summary

- Modify cells with `.loc[row, col] = value`.
 - Add columns dynamically and drop columns using `.drop(..., axis=1)`.
 - Sort DataFrames using `.sort_values(by="column")`.
 - Split and aggregate tables by group using `.groupby("col")`.
 - Screen and resolve empty cells using `.isnull()` and `.fillna()`.
-

Practice Exercises

1. Create a DataFrame containing employee data: `Name`, `Department`, `Sales`.
 2. Add a new column `Commission` which is equal to 5% of `Sales`.
 3. Update the `Sales` of one employee to a higher value. Verify that their `Commission` updates if you recalculate.
 4. Group the DataFrame by `Department` and calculate the total `Sales` in each department.
 5. Create a copy of the DataFrame with some sales values set to `None`. Identify which rows have missing sales using `isnull()`, then fill those missing sales with the average `Sales` value of the company.
-

Further Reading

- [Pandas GroupBy: Split-Apply-Combine](#)
- [Working with Missing Data in Pandas](#)
- [sort_values API reference](#)

File I/O and Real-World Case Study

Learning Objectives

By the end of this chapter, you will be able to:

- Export Pandas DataFrames to CSV files.
- Load datasets from CSV files into DataFrames.
- Build complete end-to-end automation scripts that load, clean, analyze, and save structured tabular reports.

Introduction

Automating administrative processes usually requires interacting with file systems. CSV (Comma-Separated Values) is the most common format for transferring structured tabular records. In this chapter, we will learn how to write to and read from CSV files, then tie together all NumPy and Pandas skills in a complete real-world employee dataset analysis script.

Key Concepts

Exporting to CSV

Use `df.to_csv("filename.csv", index=False)` to save your DataFrame. The parameter `index=False` is highly recommended as it prevents Pandas from writing the default numeric row indices (0, 1, 2...) as a separate unnamed column in the file.

Reading from CSV

Use `pd.read_csv("filename.csv")` to load a CSV file directly back into a Pandas DataFrame.

Examples

Example 1: Exporting and Reading a CSV File

```
import pandas as pd # 1. Create a DataFrame df = pd.DataFrame({ "product": ["Apples",  
"Bananas"], "price": [1.2, 0.8] }) # 2. Export to CSV  
df.to_csv("products_output.csv", index=False) print("CSV File Exported Successfully")  
# 3. Read it back loaded_df = pd.read_csv("products_output.csv") print("\nLoaded  
DataFrame:\n", loaded_df)
```

Example 2: Complete End-to-End Analysis Case Study

This example demonstrates a complete data workflow: 1. Define a raw employee dictionary. 2. Initialize the DataFrame. 3. Compute a new `bonus` column. 4. Filter for high-salary staff. 5. Demonstrate `.loc` and `.iloc` lookups. 6. Sort values by salary. 7. Compute average salaries by city. 8. Extract summary statistics. 9. Export the resulting tables.

```
import pandas as pd import numpy as np # 1. Define raw data employee_data = { "name":
["Amol", "Snehal", "Rahul", "Priya", "Amit"], "salary": [500000, 250000, 700000,
450000, 300000], "city": ["Pune", "Mumbai", "Delhi", "Pune", "Bangalore"],
"experience": [5, 2, 7, 4, 3] } # 2. Convert to DataFrame employees =
pd.DataFrame(employee_data) print("Original Data:") print(employees) # 3. Add a 10%
bonus column employees["bonus"] = employees["salary"] * 0.10 print("\nWith Bonus:")
print(employees) # 4. Filter for employees with salary > 400,000 high_salary =
employees[employees["salary"] > 400000] print("\nHigh Salary Employees:")
print(high_salary) # 5. loc Example (get row 0 name) print("\nloc Example (Row 0
name):") print(employees.loc[0, "name"]) # 6. iloc Example (get row 1, column 2
'city') print("\niloc Example (Row 1, Column 2):") print(employees.iloc[1, 2]) # 7.
Sort by salary descending sorted_employees = employees.sort_values(by="salary",
ascending=False) print("\nSorted By Salary:") print(sorted_employees) # 8. Group by
city and get average salary avg_salary_city =
employees.groupby("city")["salary"].mean() print("\nAverage Salary By City:")
print(avg_salary_city) # 9. Get dataset statistics print("\nStatistics:")
print(employees.describe()) # 10. Export to CSV
employees.to_csv("final_employee_data.csv", index=False) print("\nFile Exported
Successfully")
```

Notes

- Make sure you have write permissions in your current directory when exporting files.
- If your CSV files use different delimiters (like tabs or semicolons), customize `read_csv` using the `separator` parameter: `pd.read_csv("file.txt", sep="\t")`.

Summary

- Use `df.to_csv()` and `pd.read_csv()` to serialize and deserialize DataFrames.
- Pass `index=False` to avoid writing row indexes to the file.
- Grouping, filtering, and sorting functions can be chained together to build complete, automated analytical workflows.

Practice Exercises

1. Create a script that defines a DataFrame of 5 movies with columns: `Movie Name`, `Release Year`, `Rating` (out of 10), and `Box Office Revenue` (in millions).
2. Save this movies DataFrame to `movies.csv` with `index=False`.
3. Load the data back from `movies.csv` into a new variable called `movies_df`.
4. Add a new column `Rating Out of 100` which multiplies `Rating` by 10.
5. Sort the movies by `Box Office Revenue` in descending order.
6. Group the movies by their release decade (or just print the mean rating) and write the sorted, modified DataFrame to `sorted_movies.csv`.

Further Reading

- [Pandas IO Tools \(Text, CSV, etc.\)](#)
- [read_csv API reference](#)
- [to_csv API reference](#)

Module 17: Plotly and Matplotlib

Introduction to Matplotlib and Line Charts

Learning Objectives

By the end of this chapter, you will be able to:

- Set up Matplotlib and understand its scripting interface (`pyplot`).
- Create simple line charts using `plt.plot()`.
- Customize chart aesthetics, including titles, axis labels, grids, markers, and line styles.
- Save charts to files on your system.

Introduction

Visualizing data is essential for communicating insights and understanding trends. **Matplotlib** is the oldest and most widely used library for creating static, animated, and interactive visualizations in Python. In this chapter, we will learn how to create a basic line chart, format its styling, and write it to disk.

Key Concepts

Matplotlib Pyplot

The `matplotlib.pyplot` module is a collection of functions that make Matplotlib work like MATLAB. It provides simple commands to build figures, draw lines, add labels, and format layouts.

Creating a Line Chart

The most fundamental plot is the line chart, created using `plt.plot(x, y)`.

Customizing Line Plots

We can customize lines and markers to distinguish dataset trends:

- **marker**: Defines data points (e.g. `'o'` for circles, `'s'` for squares, `'^'` for triangles).
- **linestyle**: Defines the line drawing type (e.g. `'-'` for solid, `'--'` for dashed, `'.'` for dotted).
- **color**: Sets line colors (e.g. `'blue'`, `'red'`, or hex codes).

Annotations and Layouts

- `plt.title()`: Adds a main chart title.
- `plt.xlabel()` and `plt.ylabel()`: Add axis labels.

- `plt.grid(True)`: Toggles coordinate grid lines.
- `plt.legend()`: Shows a key explaining each plotted line (requires setting the `label` parameter inside `plt.plot()`).

Saving Plots

Use `plt.savefig("filename.png")` to export your charts to disk.

Examples

Example 1: Your First Line Chart

```
import matplotlib.pyplot as plt # 1. Define data points x = [1, 2, 3, 4, 5] y = [10, 20, 30, 40, 50] # 2. Draw line plt.plot(x, y) # 3. Add titles and labels plt.title("Simple Line Chart") plt.xlabel("X Values") plt.ylabel("Y Values") # 4. Display chart plt.show()
```

Example 2: Customizing Line Charts

```
import matplotlib.pyplot as plt x = [1, 2, 3, 4, 5] y = [5, 10, 15, 20, 25] # Plot with circular markers and dashed line plt.plot(x, y, marker='o', linestyle='--', color='green') plt.title("Customized Line Chart") plt.xlabel("X Axis") plt.ylabel("Y Axis") plt.grid(True) plt.show()
```

Example 3: Saving Charts to Disk

```
import matplotlib.pyplot as plt x = [1, 2, 3] y = [10, 20, 30] plt.plot(x, y, marker='s') plt.title("Save Example") # Save as PNG image in current directory plt.savefig("line_chart.png") print("Chart Saved Successfully")
```

Notes

- `plt.show()` opens a window displaying your plot and pauses script execution until the window is closed. In non-interactive environments (like automated scripts), you will usually bypass `plt.show()` and use `plt.savefig()` instead.
- If you call `plt.plot()` multiple times without resetting, Matplotlib will draw them on the same figure automatically.

Summary

- Import the scripting interface using `import matplotlib.pyplot as plt`.
 - Draw line plots using `plt.plot(x, y)`.
 - Configure plots with `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, and `plt.grid()`.
 - Export static files using `plt.savefig()`.
-

Practice Exercises

1. Create a script that plots a line chart showing a company's profit over 5 years (Year 1 to Year 5). Add axis labels and a title.
 2. Modify the line chart from Exercise 1 to use red square markers and a dotted line. Enable the grid lines.
 3. Save the modified chart from Exercise 2 as `profit_report.png` and confirm that the image file is created in your workspace.
-

Further Reading

- [Matplotlib Pyplot tutorial](#)
- [matplotlib.pyplot.plot API](#)
- [Saving Figures in Matplotlib](#)

Matplotlib Basic Chart Types

Learning Objectives

By the end of this chapter, you will be able to:

- Construct vertical and horizontal bar charts in Matplotlib.
- Create pie charts with customized labels and percentage layouts.
- Generate histograms to represent frequency distributions.
- Create scatter plots to observe relationship patterns between variables.

Introduction

Line charts are ideal for displaying values over continuous intervals or time, but different datasets call for different visualizations. In this chapter, we will look at how to represent categories with bar and pie charts, frequencies with histograms, and individual data point coordinates with scatter plots.

Key Concepts

Bar Charts

Bar charts are used to compare values across different categories.

- Vertical bar chart: `plt.bar(categories, values)`
- Horizontal bar chart: `plt.barh(categories, values)`

Pie Charts

Pie charts display proportional shares of categories in a circular layout.

- Syntax: `plt.pie(values, labels=labels, autopct='%1.1f%%')`
- The `autopct` parameter formats the percentage label on each slice (e.g. `'%1.1f%%'` displays one decimal place).

Histograms

Histograms represent frequency distributions of numeric values by dividing data into intervals ("bins").

- Syntax: `plt.hist(data, bins=n)`

Scatter Plots

Scatter plots show individual points mapped on Cartesian coordinates, revealing correlation trends.

- Syntax: `plt.scatter(x, y)`

Examples

Example 1: Vertical Bar Chart

```
import matplotlib.pyplot as plt
students = ["Amol", "Snehal", "Rahul", "Priya"]
marks = [85, 92, 78, 88]
plt.bar(students, marks, color='skyblue')
plt.title("Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.show()
```

Example 2: Horizontal Bar Chart

```
import matplotlib.pyplot as plt
courses = ["Python", "Java", "SQL", "AI"]
students = [120, 90, 70, 150]
plt.barh(courses, students, color='salmon')
plt.title("Course Enrollments")
plt.xlabel("Students Enrolled")
plt.ylabel("Courses")
plt.show()
```

Example 3: Pie Chart

```
import matplotlib.pyplot as plt
labels = ["Python", "Java", "C++", "AI"]
values = [40, 25, 15, 20]
plt.pie(values, labels=labels, autopct='%1.1f%%', colors=['gold', 'lightblue', 'lightcoral', 'lightgreen'])
plt.title("Programming Language Usage")
plt.show()
```

Example 4: Histogram

```
import matplotlib.pyplot as plt
# Student marks dataset
marks = [55, 60, 65, 70, 75, 80, 85, 90, 95]
plt.hist(marks, bins=5, color='purple', edgecolor='black')
plt.title("Histogram Example")
plt.xlabel("Marks Ranges")
plt.ylabel("Frequency")
plt.show()
```

Example 5: Scatter Plot

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [10, 15, 20, 25, 30]
plt.scatter(x, y, color='red', marker='x')
plt.title("Scatter Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

Notes

- For bar charts, you can pass custom colors as a list to color each bar uniquely.
- Make sure to add `edgecolor='black'` to histograms to distinguish the boundaries of adjacent bins clearly.

Summary

- Use `plt.bar()` and `plt.barh()` for categorical comparisons.

- Draw fractional share layouts using `plt.pie(..., autopct)`.
 - Bin numeric arrays into distribution frequencies using `plt.hist()`.
 - Display point relationships using `plt.scatter()`.
-

Practice Exercises

1. Create a Matplotlib bar chart showing the population of 4 cities: Delhi (30M), Mumbai (20M), Bangalore (12M), Pune (7M). Label axes appropriately.
 2. Create a pie chart showing a student's daily time distribution: Study (40%), Sleep (30%), Entertainment (15%), Others (15%). Include labels and percentage annotations.
 3. Generate a scatter plot of study hours vs exam scores using the data:
 4. Study Hours: `[2, 4, 5, 7, 8]`
 5. Exam Scores: `[55, 70, 75, 90, 95]` Add labels and a title.
-

Further Reading

- [Matplotlib Bar Chart guide](#)
- [Matplotlib Pie Chart guide](#)
- [Matplotlib Histograms](#)

Matplotlib Subplots and Pandas Integration

Learning Objectives

By the end of this chapter, you will be able to:

- Draw multiple data traces on the same figure and use legends.
- Arrange grids of distinct plots using `plt.subplot()`.
- Extract data from Pandas DataFrames and plot it directly in Matplotlib.

Introduction

In sophisticated reporting systems, you will often need to overlay multiple data series on a single chart for comparison, or organize multiple charts side-by-side in a dashboard. In this chapter, we will learn how to plot multiple lines, create grids of charts, and directly visualize Pandas DataFrames.

Key Concepts

Multiple Lines & Legends

To plot multiple trends on the same grid, call `plt.plot()` repeatedly. Add `label="name"` inside each call, then call `plt.legend()` to display the key describing which line belongs to which series.

Creating Subplots

Use `plt.subplot(rows, columns, active_index)` to build a grid of charts in a single figure window.

- **rows**: Number of rows in the subplot grid.
- **columns**: Number of columns in the subplot grid.
- **active_index**: The grid position (1-indexed, starting from top-left) where the next plot commands will be drawn.
- Use `plt.tight_layout()` to automatically clean up margins and prevent title overlap.

Plotting with Pandas DataFrames

You can pass Pandas DataFrame columns (Series) directly into Matplotlib functions: `plt.plot(df["x_col"], df["y_col"])`.

Examples

Example 1: Multiple Lines with a Legend

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y1 = [1, 4, 9, 16, 25]
y2 = [1, 2, 3, 4, 5]
plt.plot(x, y1, label="Square", color='blue', marker='o')
plt.plot(x, y2, label="Linear", color='red', marker='s')
plt.title("Multiple Lines")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```

Example 2: Subplots Grid Layout

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y1 = [10, 20, 30, 40]
y2 = [40, 30, 20, 10]
# Setup a grid with 1 row, 2 columns, and select the first panel
plt.subplot(1, 2, 1)
plt.plot(x, y1, color='blue')
plt.title("First Plot")
# Select the second panel in the 1x2 grid
plt.subplot(1, 2, 2)
plt.plot(x, y2, color='red')
plt.title("Second Plot")
# Optimize layout spacing
plt.tight_layout()
plt.show()
```

Example 3: Plotting directly from a Pandas DataFrame

```
import pandas as pd
import matplotlib.pyplot as plt
sales_data = { "month": ["Jan", "Feb", "Mar", "Apr"], "sales": [1000, 1500, 1800, 2200] }
sales_df = pd.DataFrame(sales_data)
# Pass DataFrame columns directly to matplotlib
plt.plot(sales_df["month"], sales_df["sales"], marker='o', color='purple')
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales ($)")
plt.grid(True)
plt.show()
```

Notes

- Call `plt.subplot()` before applying any title, label, or plot commands, as those commands will only apply to the currently selected active panel.
- Slices in subplots start at index 1 (not 0). E.g. `plt.subplot(2, 2, 1)` to `plt.subplot(2, 2, 4)`.

Summary

- Plot multiple traces on a single grid by calling `plt.plot()` sequentially and invoking `plt.legend()`.
- Divide a figure into a grid using `plt.subplot(rows, cols, active_index)`.
- Pass Pandas columns directly to Matplotlib plotting commands.

Practice Exercises

1. Create a single plot containing three lines: $y = x$, $y = x^2$, and $y = x^3$ for $x = [1, 2, 3, 4]$. Give each line a label, a different color, and show the legend.
 2. Build a 2x1 vertical grid of subplots:
 3. Upper panel: Bar chart showing course enrollments (Python: 120, SQL: 70).
 4. Lower panel: Line plot of the same enrollment numbers. Use `tight_layout()` and add sub-titles to both.
 5. Load a dictionary containing weekly hours worked by two employees:
 6. Week: Week 1, Week 2, Week 3, Week 4
 7. Employee A: 40, 42, 38, 40
 8. Employee B: 35, 36, 40, 42 Convert it to a DataFrame and plot both lines on the same Matplotlib chart directly from the DataFrame.
-

Further Reading

- [Matplotlib Multiple Plots](#)
- [matplotlib.pyplot.subplot API](#)

Introduction to Plotly and Basic Charts

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the key differences and use cases for Plotly versus Matplotlib.
- Use Plotly Express (px) to create interactive line charts.
- Build interactive scatter plots, bar charts, and pie charts.
- Identify the components of a Plotly figures output.

Introduction

Static images (like those generated by Matplotlib) are great for PDF reports and books, but modern web applications require interactive visualizations. **Plotly** is a high-level graphing library that produces interactive, publication-quality charts. Users can zoom in, pan, hover over points to view tooltips, and click legends to toggle data traces on and off in their browsers.

Key Concepts

Matplotlib vs. Plotly

- **Matplotlib:** Static PNG/PDF plots. Fast, stable, excellent for printing.
- **Plotly:** Dynamic HTML plots. Written in Javascript, renders in browsers, excellent for dashboards.

Plotly Express

`plotly.express` (typically imported as `px`) is the easy-to-use, high-level API for Plotly. It takes a Pandas DataFrame as input and creates a fully configured figure in a single function call.

Basic Interactive Charts

- **Line Chart:** `px.line(df, x="x_col", y="y_col", title="title")`
 - **Scatter Plot:** `px.scatter(df, x="x_col", y="y_col", title="title")`
 - **Bar Chart:** `px.bar(df, x="x_col", y="y_col", title="title")`
 - **Pie Chart:** `px.pie(df, names="name_col", values="value_col", title="title")`
-

Examples

Example 1: Your First Plotly Line Chart

```
import plotly.express as px
import pandas as pd
sample_data = { "x": [1, 2, 3, 4, 5],
                "y": [10, 20, 30, 40, 50] }
sample_df = pd.DataFrame(sample_data)
# Create an interactive line chart
fig = px.line(sample_df, x="x", y="y", title="Plotly Line Chart")
# Open in browser
fig.show()
```

Example 2: Interactive Scatter Plot

```
import plotly.express as px
import pandas as pd
scatter_df = pd.DataFrame({ "hours": [1, 2, 3, 4, 5],
                             "marks": [50, 60, 70, 80, 90] })
fig = px.scatter(scatter_df, x="hours", y="marks", title="Study Hours vs Marks")
fig.show()
```

Example 3: Plotly Bar Chart

```
import plotly.express as px
import pandas as pd
bar_df = pd.DataFrame({ "course": ["Python", "Java", "SQL"],
                         "students": [120, 90, 70] })
fig = px.bar(bar_df, x="course", y="students", title="Course Students")
fig.show()
```

Example 4: Plotly Pie Chart

```
import plotly.express as px
import pandas as pd
pie_df = pd.DataFrame({ "language": ["Python", "Java", "AI", "SQL"],
                         "usage": [45, 25, 20, 10] })
fig = px.pie(pie_df, names="language", values="usage", title="Language Usage")
fig.show()
```

Notes

- Slicing methods in Plotly Express expect a Pandas DataFrame as their first argument, and column name strings as their x and y parameters.
 - Calling `.show()` on a Plotly figure will temporarily host the interactive chart locally and open your default web browser to display it.
-

Summary

- Plotly generates interactive, web-browser-ready visualizations.
- Use `plotly.express` (`px`) for rapid, single-line chart creation.
- Set up line, scatter, bar, and pie charts by passing a DataFrame and matching column header strings.

Practice Exercises

1. Create a script that builds a Plotly bar chart showing monthly product sales:
 2. Months: ["Jan", "Feb", "Mar"]
 3. Sales: [5000, 7000, 6500] Open the chart in your browser and check the hover information.
 4. Build an interactive scatter plot of height vs weight using the data:
 5. Height (cm): [150, 160, 170, 180]
 6. Weight (kg): [55, 62, 70, 85] Give it an interactive title.
 7. Build a Plotly pie chart representing a department's budget breakdown:
 8. Department: ["R&D", "Marketing", "HR", "Sales"]
 9. Budget: [40000, 25000, 10000, 30000]
-

Further Reading

- [Plotly Express introductory guide](#)
- [Plotly Line Charts](#)
- [Plotly Bar Charts](#)

Plotly Histograms, Box Plots, and 3D Charts

Learning Objectives

By the end of this chapter, you will be able to:

- Construct interactive histograms to observe dataset frequencies.
- Create box plots (`px.box`) to analyze data ranges, quartiles, and outliers.
- Render interactive 3D scatter plots to visualize relationships between three variables.

Introduction

Beyond line and bar charts, data analysts rely on distributions and multi-dimensional analysis. In this chapter, we will learn how to build interactive histograms and box plots to see how values are distributed, and how to rotate, zoom, and inspect data points in a 3D coordinate space.

Key Concepts

Histograms

Like Matplotlib's static histograms, Plotly's `px.histogram()` groups continuous numbers into bins, but adds interactive hover counts.

Box Plots

Box plots represent numerical distributions by highlighting the minimum, first quartile (Q1), median, third quartile (Q3), maximum, and any statistical outliers. - Syntax: `px.box(df, y="numeric_column")` - Hovering over a Plotly box plot reveals the exact quartile and median values.

3D Scatter Plots

To analyze three numeric variables simultaneously, we can map points in a 3-dimensional space. - Syntax: `px.scatter_3d(df, x="col1", y="col2", z="col3", title="title")` - Users can click and drag in the browser to rotate the 3D grid and observe points from different angles.

Examples

Example 1: Interactive Histogram


```
import plotly.express as px
import pandas as pd
hist_df = pd.DataFrame({ "marks": [55, 60, 70, 75, 80, 85, 90, 95] })
fig = px.histogram(hist_df, x="marks", nbins=5, title="Marks Distribution")
fig.show()
```

Example 2: Interactive Box Plot

```
import plotly.express as px
import pandas as pd
box_df = pd.DataFrame({ "salary": [25000, 30000, 35000, 45000, 50000, 60000] })
fig = px.box(box_df, y="salary", title="Salary Distribution")
fig.show()
```

Example 3: 3D Scatter Plot

```
import plotly.express as px
import pandas as pd
three_d_df = pd.DataFrame({ "x": [1, 2, 3, 4], "y": [10, 20, 30, 40], "z": [5, 15, 25, 35] })
fig = px.scatter_3d(three_d_df, x="x", y="y", z="z", title="3D Scatter Plot")
fig.show()
```

Notes

- 3D plots require a GPU-enabled web browser to render smoothly.
- Box plots are extremely useful in automated data pipelines for identifying data entry errors and anomalies (outliers that fall outside the maximum/minimum range).

Summary

- Display counts of range intervals interactively using `px.histogram()`.
- Unpack data quartiles, medians, and outliers using `px.box()`.
- Plot three coordinates in space using `px.scatter_3d()`, allowing complete browser rotation.

Practice Exercises

1. Create a Plotly histogram representing the distribution of exam scores of a large class: `[62, 65, 78, 82, 85, 88, 92, 95, 98]`. Set the number of bins to 4.
2. Build a box plot of employee age data: `[22, 25, 28, 30, 32, 45, 52, 61, 80]`. Observe what is flagged as an outlier (if anything).
3. Create a 3D scatter plot showcasing real estate properties:
4. x: Square Footage `[1000, 1500, 2000, 2500]`

5. y: Number of Bedrooms [2, 3, 3, 4]

6. z: Price in Thousands [150, 220, 310, 420]

Further Reading

- [Plotly Box Plots Guide](#)
- [Plotly Histograms Guide](#)
- [Plotly 3D Scatter Plots](#)

Plotly Graph Objects, Layouts, and Dashboards

Learning Objectives

By the end of this chapter, you will be able to:

- Use Plotly Graph Objects (go) for granular chart styling.
- Add multiple independent data traces to a single figure.
- Arrange multi-chart dashboard layouts using `make_subplots`.
- Export interactive Plotly figures to standalone HTML files.
- Compare and combine Matplotlib and Plotly in a real-world script.

Introduction

While Plotly Express is excellent for quick, single-line chart creation, complex data visualization pipelines sometimes require fine-grained control over layout, styling, and multi-chart layouts. In this chapter, we will learn about **Plotly Graph Objects** for adding custom data traces, building dashboard-style subplot layouts, and exporting interactive charts to standalone HTML files. We will conclude with a real-world case study comparing both Matplotlib and Plotly.

Key Concepts

Plotly Graph Objects (go)

Unlike `plotly.express`, which automatically interprets DataFrames, `plotly.graph_objects` (go) is a lower-level API that requires manual trace configuration. It is much more powerful for building highly customized visualizations.

- Create figure container: `fig = go.Figure()`
- Add line/scatter trace: `fig.add_trace(go.Scatter(x=..., y=..., mode='lines+markers', name='label'))`
- Add bar trace: `fig.add_trace(go.Bar(x=..., y=..., name='label'))`
- Update layouts: `fig.update_layout(title="title", xaxis_title="X")`

Dashboard Subplots

Use `make_subplots(rows=r, cols=c)` from `plotly.subplots` to build a grid of charts, and direct individual traces to specific cells using the `row` and `col` arguments in `add_trace()`.

Exporting Plotly Charts

Use `fig.write_html("dashboard.html")` to export interactive charts. This generates a standalone HTML document containing the JavaScript bundle, which can be viewed in any browser or embedded in email notifications.

Examples

Example 1: Multi-Trace Line Plot with Graph Objects

```
import plotly.graph_objects as go fig = go.Figure() # Add Trace A
fig.add_trace(go.Scatter( x=[1, 2, 3, 4], y=[10, 15, 20, 25], mode='lines+markers',
name='Product A' )) # Add Trace B fig.add_trace(go.Scatter( x=[1, 2, 3, 4], y=[5, 10,
15, 20], mode='lines', name='Product B' )) fig.update_layout( title="Sales Comparison
(Graph Objects)", xaxis_title="Time Period", yaxis_title="Units Sold" ) fig.show()
```

Example 2: Dashboard-Style Layout with Subplots

```
from plotly.subplots import make_subplots import plotly.graph_objects as go # Setup 1
row, 2 columns subplot grid fig = make_subplots(rows=1, cols=2) # Add line chart to
row 1, column 1 fig.add_trace( go.Scatter(x=[1, 2, 3], y=[10, 20, 30], name="Line
Track"), row=1, col=1 ) # Add bar chart to row 1, column 2 fig.add_trace(
go.Bar(x=["A", "B", "C"], y=[5, 7, 3], name="Category Count"), row=1, col=2 )
fig.update_layout(title_text="Multi-Plot Dashboard Layout") fig.show()
```

Example 3: Exporting Plotly Charts to HTML

```
import plotly.express as px fig = px.line(x=[1, 2, 3, 4], y=[10, 20, 30, 40],
title="Export Example") # Write to a standalone HTML file
fig.write_html("plotly_chart.html") print("Plotly Chart Exported to HTML
Successfully")
```

Example 4: Real-World Visualization Case Study

This example demonstrates a complete script that converts data into a Pandas DataFrame, creates a static Matplotlib bar chart, and generates an interactive Plotly scatter plot.

```
import pandas as pd import plotly.express as px import matplotlib.pyplot as plt # 1.
Prepare Employee Data employee_data = { "employee": ["Amol", "Snehal", "Rahul",
"Priya", "Amit"], "salary": [50000, 60000, 55000, 70000, 65000], "experience": [5, 7,
4, 9, 6] } employees = pd.DataFrame(employee_data) print(employees) # 2. Matplotlib
Static Chart plt.bar(employees["employee"], employees["salary"], color="darkblue")
plt.title("Employee Salaries (Static)") plt.xlabel("Employee") plt.ylabel("Salary
($)") plt.savefig("employee_salaries.png") plt.show() # 3. Plotly Interactive Scatter
Plot fig = px.scatter( employees, x="experience", y="salary", text="employee",
title="Experience vs Salary (Interactive)" )
fig.write_html("experience_salary_plot.html") fig.show()
```

Notes

- Standalone HTML pages generated by `write_html()` contain the full `plotly.js` library embedded within them (about 3MB in size). If you want to optimize size for web hosting, you can set `include_plotlyjs='cdn'` to fetch the Javascript library over the web.

- Plotly subplots grid indexes start at 1 (not 0), similar to Matplotlib's `plt.subplot()`.
-

Summary

- Import the lower-level API using `import plotly.graph_objects as go`.
 - Manually construct figures and register layers using `fig.add_trace()`.
 - Create grid subplots using `make_subplots(rows, cols)` and update visual elements with `fig.update_layout()`.
 - Save interactive visual pages using `fig.write_html()`.
-

Practice Exercises

1. Create a script that uses Plotly Graph Objects to draw a bar chart showing the stock level of 3 warehouse items: Item A (45), Item B (12), Item C (28). Customize the bar color to purple and set axis labels using `update_layout`.
 2. Build a Plotly subplot dashboard (1 row, 2 columns):
 3. Left cell: A pie chart showing company revenue sources.
 4. Right cell: A line chart showing quarterly profit.
 5. Export the dashboard from Exercise 2 into a file named `quarterly_report.html` and verify that the HTML file opens and displays interactively in your web browser.
-

Further Reading

- [Plotly Graph Objects Overview](#)
- [Plotly Subplots Documentation](#)
- [Exporting Figures to HTML and Images](#)

Module 18: Building Web Apps with Flask

Introduction to Flask and Setup

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what Flask is and why it is classified as a microframework.
- Create a virtual environment and install Flask using `pip`.
- Structure and write a minimal Flask application.
- Start the development server and run the application in debug mode.

Introduction

In modern automation, running local scripts or CLI tools is often not enough. You may need to expose your Python functions so that other systems, team members, or frontend web pages can interact with them.

Flask is a popular Python web framework that allows you to build web applications and API services with minimal boilerplate. It is known as a **microframework** because it does not require a specific database, form validation tool, or template engine by default. Instead, it provides the core routing and HTTP request-handling engine, letting you add external libraries (like Pandas or SQLite) as needed.

Key Concepts

What is a Web Framework?

A web framework is a collection of libraries and tools that handles standard web development tasks such as:

- Listening for network requests on specific ports.
- Mapping URLs (e.g., `/home`, `/api/users`) to Python functions.
- Formatting responses (HTML pages, file downloads, or JSON data).
- Parsing request details (cookies, form values, and files).

Why Flask?

Flask is lightweight, highly extensible, and easy to learn. It is perfect for automation engineers who want to quickly build a dashboard, wrap their automation scripts in web APIs, or create lightweight microservices.

Installation and Virtual Environments

To avoid cluttering your system's global Python installation, you should always run your web applications inside a **virtual environment**. A virtual environment isolates project-specific dependencies.

Commands to initialize and activate a virtual environment:

```
# Create the environment folder named 'venv' python -m venv venv # Activate on
macOS/Linux: source venv/bin/activate # Activate on Windows (Command Prompt):
venv\Scripts\activate # Activate on Windows (PowerShell): .\venv\Scripts\Activate.ps1
```

Once activated, you install Flask using `pip`:

```
pip install Flask
```

Examples

Example 1: Instantiating a Minimal Flask App

Create a file named `app.py`. Write the following code:

```
from flask import Flask # Initialize the Flask application # __name__ helps Flask
locate templates and static files app = Flask(__name__) # Define a route for the
homepage @app.route("/") def home(): return "Hello from Flask! Your setup is
successful." if __name__ == "__main__": # Run the application locally
app.run(debug=True)
```

Example 2: Running the Development Server

There are two primary ways to run your Flask application:

Directly running the Python script: `bash python app.py`

Using the Flask CLI command: `bash export FLASK_APP=app.py export FLASK_ENV=development # Deprecated in Flask 2.2+, use --debug instead flask run --debug` (On Windows Command Prompt, use `set` instead of `export`. On PowerShell, use `$env:`).

Once run, you will see output indicating the server is running, typically at `http://127.0.0.1:5000/`. Open this URL in your browser to see your message.

Notes

- **The debug=True Mode:** When debug mode is active, Flask enables a **hot reloader**, meaning the server will automatically restart whenever you save code changes. It also shows an **interactive debugger** in the browser if an unhandled exception occurs in your code.
 - **Security Warning:** Never run your application with `debug=True` in a production environment, as it allows arbitrary code execution via the browser debugger.
 - **Port Conflicts:** By default, Flask runs on port 5000. If port 5000 is occupied (e.g. by AirPlay on macOS), you can change it in code: `app.run(port=8080, debug=True)` or in the CLI: `flask run --port=8080`.
-

Summary

- Flask is a lightweight microframework used to build web applications and APIs.
 - Virtual environments (`venv`) isolate your Flask dependencies from the rest of the OS.
 - A Flask app uses decorators like `@app.route("/")` to bind a URL to a Python function.
 - Debug mode provides automatic code reloading and error reporting during development.
-

Practice Exercises

1. Create a Flask application that returns `"Welcome to the Python Automation Course!"` when visiting the homepage.
 2. Add a second route `/health` that returns `"Server is healthy and running!"`.
 3. Modify your Flask script to run on port 8000 with debug mode disabled. Confirm that the application loads on `http://127.0.0.1:8000/`.
-

Further Reading

- [Flask Official Documentation - Installation](#)
- [Flask Official Documentation - Quickstart](#)

Routing and Request Handling

Learning Objectives

By the end of this chapter, you will be able to:

- Define URL routes containing dynamic path variables.
- Use Flask's variable converters to restrict parameter types.
- Access and parse query parameters from the URL.
- Parse form data and JSON payloads from incoming HTTP requests.

Introduction

Web applications and APIs work by receiving requests and sending responses. To make your Flask app interactive, you need it to react differently depending on the URL path the user visits, and the parameters or data they submit. This is accomplished using **routing** and the global **request** object.

Key Concepts

Dynamic Routing and Converters

Routes are not limited to static strings. You can declare dynamic variables in your route path using angle brackets: `@app.route('/user/<username>')`.

By default, Flask treats path parameters as strings, but you can use **converters** to enforce specific types:

- `string`: Accepts any text without a slash (default).
- `int`: Accepts positive integers.
- `float`: Accepts positive floating-point values.
- `path`: Similar to string but accepts slashes (useful for file paths).

For example, `@app.route('/orders/<int:order_id>')` matches `/orders/123` but returns a 404 error for `/orders/abc`.

The request Object

To inspect the details of an incoming connection, Flask provides the `request` object, which is imported from the `flask` library.

The `request` object contains:

1. `request.args`: A dictionary-like object (`MultiDict`) containing query parameters passed in the URL (e.g. `?name=Amol&role=admin`).
2. `request.form`: A dictionary containing form data sent via an HTML form POST request.
3. `request.get_json()`: A method that parses and returns the JSON payload sent in the request body (typically for APIs).
4. `request.method`: A string showing the HTTP method used (e.g. `"GET"`, `"POST"`).
5. `request.headers`: A dictionary containing HTTP headers.

Examples

Example 1: Dynamic Route Variables

This example shows how to use type-restricted dynamic variables:

```
from flask import Flask app = Flask(__name__) # String dynamic path
@app.route("/greet/<name>") def greet(name): return f"Hello, {name}!" # Integer
dynamic path @app.route("/user/<int:user_id>") def show_user(user_id): # user_id is
automatically converted to an integer return f"Displaying profile for User ID:
{user_id}"
```

Example 2: Parsing Query Parameters

Query parameters are commonly used to filter or sort data (e.g. `http://127.0.0.1:5000/search?q=Python&page=2`).

```
from flask import Flask, request app = Flask(__name__) @app.route("/search") def
search(): # Use request.args.get() to safely handle missing keys query =
request.args.get("q", default="all") page = request.args.get("page", default=1,
type=int) return f"Searching for: '{query}' on page {page}"
```

Example 3: Parsing JSON Request Payloads

For APIs, client systems often send data formatted as JSON.

```
from flask import Flask, request app = Flask(__name__) @app.route("/api/users",
methods=["POST"]) def create_user(): # Retrieve JSON data from the body data =
request.get_json() # Safely extract parameters name = data.get("name") email =
data.get("email") if not name or not email: return "Missing name or email in request
body", 400 return f"Created user {name} with email {email} successfully!"
```

Notes

- **Using `get()` vs brackets:** When retrieving parameters from `request.args` or `request.form`, always prefer `.get("key")` over `["key"]`. If the key does not exist, `.get()` returns `None` (or a default value you specify), whereas `["key"]` raises a `KeyError` (returning a 400 Bad Request error to the user).
 - **Importing `request`:** The `request` object is a thread-safe local proxy. Do not define variables with the name `request` in your route functions to avoid shadowing the Flask object.
-

Summary

- Route paths can contain variables, optionally filtered using type converters (like `<int:id>`).
 - Flask parses incoming data and makes it accessible through properties on the `request` object.
 - Use `request.args` for query parameters and `request.get_json()` for API JSON payloads.
-

Practice Exercises

1. Create a route `/square/<int:number>` that calculates the mathematical square of the variable passed in the URL path and returns it.
 2. Create a route `/report` that reads a query parameter named `format`. If `format=pdf`, return "Generating PDF Report". If `format=csv`, return "Generating CSV Report". For any other value (or if missing), return "Generating HTML Report".
 3. Create a POST route `/login` that reads form data variables `username` and `password`. If `username` is "admin" and `password` is "secret", return "Access Granted". Otherwise, return "Access Denied".
-

Further Reading

- [Flask Route Rules](#)
- [The Flask Request Object](#)

Rendering Templates and Static Files

Learning Objectives

By the end of this chapter, you will be able to:

- Render separate HTML documents using the `render_template` function.
- Inject dynamic Python variables and objects into your web pages.
- Use Jinja2 control structures (loops, conditionals) inside HTML templates.
- Link and serve static assets (such as CSS stylesheets, JavaScript files, and images).

Introduction

Returning hardcoded HTML strings directly from your Python route functions quickly becomes messy and unmaintainable. To separate presentation from logic, Flask utilizes a template engine called **Jinja2**. This allows you to write standard HTML documents containing dynamic placeholders that Flask populates before sending the final page to the user's browser.

Key Concepts

Directory Structure

Flask expects a specific folder layout by default:

- **templates/**: This directory contains all your HTML files (e.g., `index.html`).
- **static/**: This directory contains your static client-side files, such as CSS, client-side JavaScript, and images.

```
my_flask_project/ ■■■■ app.py ■■■■ templates/ ■■■■ index.html ■■■■ static/ ■■■■
css/ ■■■■ style.css ■■■■ images/ ■■■■ logo.png
```

Jinja2 Syntax

Jinja2 templates use specific delimiters to distinguish normal HTML text from template commands:

1. **{{ expression }}**: Variable placeholders. Used to print variables, object properties, or dictionary values directly onto the page.
2. **{% block %}**: Control flow statements. Used to declare loops (`for`), conditional blocks (`if`), or inheritance blocks.
3. **{# comment #}**: Template comments. These comments are stripped out and are not sent to the client's browser.

Serving Static Files

To link a stylesheet or script, you should not hardcode the file path. Instead, use Flask's `url_for` utility function, which dynamically generates correct URLs for routes and static assets.

Example syntax in HTML:

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
```

Examples

Example 1: Basic Template Rendering

First, create a template file under `templates/welcome.html`:

```
<!DOCTYPE html> <html> <head> <title>Flask Template</title> </head> <body>
<h1>Welcome, {{ user_name }}!</h1> <p>Today is {{ day }}. Enjoy learning Flask!</p>
</body> </html>
```

Next, write the Python logic in `app.py`:

```
from flask import Flask, render_template app = Flask(__name__)
@app.route("/welcome/<name>") def welcome(name): # Pass variables as keyword
arguments to render_template return render_template("welcome.html", user_name=name,
day="Friday")
```

Example 2: Loops and Conditionals in Jinja2

Create `templates/products.html`:

```
<!DOCTYPE html> <html> <head> <title>Product List</title> </head> <body>
<h1>Available Products</h1> <ul> {% for product in product_list %} <li> <strong>{{
product.name }}</strong> - ${{ product.price }} {% if product.stock > 0 %} <span
style="color: green;">(In Stock: {{ product.stock }})</span> {% else %} <span
style="color: red;">(Out of Stock)</span> {% endif %} </li> {% endfor %} </ul>
</body> </html>
```

Python logic in `app.py`:

```
from flask import Flask, render_template app = Flask(__name__)
@app.route("/products") def list_products(): products = [ {"name": "Laptop", "price":
999.99, "stock": 5}, {"name": "Mouse", "price": 24.99, "stock": 0}, {"name":
"Keyboard", "price": 49.99, "stock": 12} ] return render_template("products.html",
product_list=products)
```

Example 3: Adding Static Stylesheet

Create a stylesheet under `static/style.css`:

```
body { font-family: Arial, sans-serif; background-color: #f4f6f9; padding: 20px; } h1
{ color: #1e3a8a; }
```

Reference it in your HTML page (`templates/products.html`):

```
<head> <title>Product List</title> <!-- Use url_for to dynamically build path to
static folder --> <link rel="stylesheet" href="{{ url_for('static',
filename='style.css') }}"> </head>
```

Notes

- **XSS Protection:** Jinja2 automatically escapes variables before displaying them (e.g. converting `<` to `<`). This prevents Cross-Site Scripting (XSS) attacks by default. If you need to render raw HTML code, you must explicitly mark it as safe using the `|safe` filter: `{{ my_html_string | safe }}`.
- **Auto-Reloading Templates:** By default, Flask caches templates. When running in debug mode (`debug=True`), the cache is bypassed, ensuring you see changes immediately after saving template files without needing to restart the server.

Summary

- Store HTML files in the `templates/` folder and assets in the `static/` folder.
- Use `render_template("filename.html", **context)` to return formatted pages.
- Jinja2 expressions (`{{ ... }}`) render data; statements (`{% ... %}`) execute control logic.
- Use `url_for('static', filename='...')` to reference assets safely.

Practice Exercises

1. Build a Flask route `/dashboard` that passes a dictionary of statistics (e.g., `{'users': 150, 'sales': 1200, 'errors': 3}`) to a template and displays them in an HTML table.
2. In the template from Exercise 1, use an `if` condition to apply a red class style to the error value if the number of errors is greater than 0.
3. Write a Flask route that passes a list of 5 computer server names. Display them inside an ordered list (`<ol>`) using a Jinja2 `for` loop, and load a custom logo image from the `static/` folder at the top of the page.

Further Reading

- [Flask Templates Guide](#)
- [Jinja Template Designer Documentation](#)

Introduction to REST APIs and HTTP Methods

Learning Objectives

By the end of this chapter, you will be able to:

- Define what a REST API is and identify its core constraints.
- Explain the role and behavior of the main HTTP methods (GET, POST, PUT, DELETE).
- Map HTTP methods directly to standard CRUD operations.
- Select and return appropriate HTTP status codes for various API outcomes.

Introduction

In software automation, system integration is key. Scripts need to fetch data from database servers, trigger workflows on build machines, or submit stats to reporting dashboards. The industry-standard way for systems to talk to each other over the network is through **REST APIs** (Representational State Transfer Application Programming Interfaces).

Understanding how REST APIs work is essential for writing code that integrates with web services, cloud providers, and internal backend platforms.

Key Concepts

What is REST?

REST is an architectural style for designing networked applications. It relies on a stateless, client-server protocol—almost always HTTP.

Core concepts of REST:

- **Resources**: Any entity or object you want to expose (e.g. a user, a product, a file). Resources are identified by logical URIs (Uniform Resource Identifiers).
- **Statelessness**: Every request from a client must contain all the information needed to understand and complete the request. The server does not store session memory about the client.
- **Representations**: The client requests a resource, and the server returns a representation of that resource (almost always formatted as JSON in modern APIs).

HTTP Methods (Verbs) and CRUD

REST uses standard HTTP methods to represent operations. They map directly to standard CRUD (Create, Read, Update, Delete) database operations:

HTTP Method	CRUD Operation	Description	Safe?	Idempotent?
GET	Read	Retrieve a resource or a collection of resources.	Yes	Yes
POST	Create	Create a new resource with data from the request body.	No	No
PUT	Update	Replace or fully update an existing resource.	No	Yes
DELETE	Delete	Remove a specified resource.	No	Yes

- **Safe:** An HTTP method is safe if it does not modify the server state (it only reads data).
- **Idempotent:** An HTTP method is idempotent if executing it multiple times yields the same state on the server as running it once.

HTTP Response Status Codes

Servers must notify clients about the result of their requests using standard three-digit HTTP status codes:

- **2xx (Success)**
- **200 OK:** Request succeeded.
- **201 Created:** Request succeeded and a new resource was created (commonly used for POST).
- **4xx (Client Errors)**

- **400 Bad Request:** The request had invalid syntax or missing required parameters.
- **401 Unauthorized:** Authentication is required or has failed.
- **403 Forbidden:** The client is authenticated but does not have permission for the resource.
- **404 Not Found:** The requested resource could not be found on the server.
- **405 Method Not Allowed:** The resource exists, but the request used an HTTP method that is not supported.
- **5xx (Server Errors)**
- **500 Internal Server Error:** The server encountered an unexpected error and could not complete the request.

Examples

Example 1: RESTful Endpoint Design

A properly designed REST API organizes endpoints around nouns (plural) representing resources, using the HTTP verb to describe the action:

- **GET /api/books** - Retrieve all books (Read list)
- **GET /api/books/45** - Retrieve book with ID 45 (Read item)
- **POST /api/books** - Add a new book (Create)
- **PUT /api/books/45** - Update book 45 (Update)
- **DELETE /api/books/45** - Remove book 45 (Delete)

Avoid endpoints like `/api/getBooks`, `/api/createBook`, or `/api/deleteBook/45` because they contain verbs in the URI, violating REST design standards.

Example 2: Implementing Route Methods in Flask

By default, Flask routes only respond to `GET` requests. To support other methods (like `POST` or `DELETE`), you must specify them in the `@app.route` decorator's `methods` list:

```
from flask import Flask, request
app = Flask(__name__) # This route supports both GET and POST requests
@app.route("/api/books", methods=["GET", "POST"])
def handle_books():
    if request.method == "GET":
        return "Returning list of books..."
    elif request.method == "POST":
        return "Book created successfully!", 201
```

Example 3: Returning Specific HTTP Status Codes

You can return status codes from route functions by returning a tuple: `(response_body, status_code)`:

```
from flask import Flask, request
app = Flask(__name__)

@app.route("/api/verify", methods=["POST"])
def verify():
    data = request.get_json()
    if not data or "token" not in data:
        # Return 400 Bad Request if token is missing
        return "Bad Request: 'token' is required", 400
    if data["token"] != "secret-123":
        # Return 401 Unauthorized if token is incorrect
        return "Unauthorized: Invalid token", 401
    return "Verification Successful!", 200
```

Notes

- **HTTP Status Code Defaults:** If you do not specify a status code in your return tuple (e.g. you return just a string), Flask defaults to 200 OK.
- **405 Handling:** If a user makes a request to a route using a method not defined in the `methods` list (e.g. sending a PUT to `/api/verify` from Example 3), Flask will automatically reply with a 405 Method Not Allowed response.

Summary

- REST APIs use resources identified by URIs and standard HTTP methods to represent actions.
- GET is safe and idempotent. POST is neither. PUT and DELETE are idempotent but not safe.
- Status codes communicate the result of an HTTP request (2xx for success, 4xx for client errors, 5xx for server errors).

Practice Exercises

1. Design the REST URI endpoints and HTTP methods required for a student grading system where you need to:
2. List all student grades.
3. Fetch the grade of a student named "SnehaL".
4. Submit a new grade.
5. Change an existing grade.
6. Delete a student grade profile.

7. Write a Flask route `/api/items` that handles GET, POST, and DELETE requests. Return a different string and appropriate status code for each method.
8. Write a Flask route `/api/divide` that parses two query parameters `a` and `b`. If either is missing, return a 400 status code. If `b` is zero, return a 400 status code. Otherwise, return the division result and a 200 status.

Further Reading

- [Architectural Styles and the Design of Network-based Software Architectures \(REST\)](#)
- [MDN Web Docs: HTTP Request Methods](#)
- [MDN Web Docs: HTTP Response Status Codes](#)

Integrating CSV Data with Pandas and SQLite

Learning Objectives

By the end of this chapter, you will be able to:

- Read tabular data from a CSV file into a Pandas DataFrame.
- Connect to a SQLite database from a Python script.
- Export a DataFrame structure directly into a SQLite table using `to_sql()`.
- Retrieve and verify database records using standard SQL queries.

Introduction

In real-world automation, data often originates in CSV files, spreadsheets, or log files. While Pandas is excellent for analyzing and cleaning this data, keeping data in CSV format is inefficient for web applications. CSV files are slow to query and do not support concurrent reads/writes well.

Instead, we can read a CSV using Pandas, clean it, and insert it into **SQLite**—a lightweight, serverless relational database engine. Once stored in SQLite, our Flask API can query and modify records efficiently.

Key Concepts

What is SQLite?

SQLite is a self-contained, serverless SQL database engine. Unlike PostgreSQL or MySQL, SQLite stores its entire database as a single file on disk (e.g., `app.db`). This makes it highly portable and ideal for desktop applications, testing, and microservices.

Pandas to SQLite Integration

Pandas integrates with database engines using a library called **SQLAlchemy**. SQLAlchemy is a SQL toolkit and Object Relational Mapper (ORM) for Python.

To write a Pandas DataFrame to a database:

1. Create a SQLAlchemy engine.
2. Load a CSV with `pd.read_csv()`.
3. Use the `df.to_sql()` method.

Key parameters of `df.to_sql()`:

- `name`: The name of the SQL table to create or write to.
- `con`: The database connection or SQLAlchemy engine object.
- `if_exists`: What to do if the table already exists:
 - `'fail'`: Raise an error (default).
 - `'replace'`: Drop the table and recreate it before inserting.
 - `'append'`: Keep existing structure and insert new rows.
- `index`: Boolean (True/False). Dictates

whether the DataFrame's index column should be stored as a database column.

Examples

Example 1: Ingesting a CSV and Storing in SQLite

Suppose we have a file named `employees.csv` in our project:

```
Name,Department,Salary
Amol,Engineering,95000
Snehal,Product,92000
Rahul,Marketing,65000
Priya,Engineering,105000
```

The following Python script reads this CSV and saves it to a SQLite database:

```
import pandas as pd from sqlalchemy import create_engine # 1. Load CSV data into a
Pandas DataFrame df = pd.read_csv("employees.csv") # 2. Create a SQLAlchemy
connection engine for a SQLite database named 'company.db' # 'sqlite:///...' uses 3
slashes for a relative path on your system engine =
create_engine("sqlite:///company.db") # 3. Save the DataFrame to a SQL table named
'employees' # We use if_exists='replace' so we can run this script repeatedly
df.to_sql("employees", con=engine, if_exists="replace", index=False) print("Data
imported from CSV to SQLite 'employees' table successfully!")
```

Example 2: Querying SQL Table Data with Pandas

Once data is in SQLite, you can query it back into a DataFrame using `pd.read_sql()`:

```
import pandas as pd from sqlalchemy import create_engine engine =
create_engine("sqlite:///company.db") # Query the entire table query = "SELECT * FROM
employees WHERE Department = 'Engineering'" df_engineering = pd.read_sql(query,
con=engine) print("Engineering Department Employees:") print(df_engineering)
```

Example 3: Connecting with built-in `sqlite3` for REST APIs

While Pandas is great for bulk operations (like loading and saving DataFrames), for thin API routes we can use Python's built-in `sqlite3` module to make quick, light queries.

```
import sqlite3 # Connect to the SQLite database file conn =
sqlite3.connect("company.db") # Create a cursor object to execute commands cursor =
conn.cursor() # Run a query cursor.execute("SELECT Name, Salary FROM employees") rows
= cursor.fetchall() # Print results for row in rows: print(f"Employee: {row[0]},
Salary: ${row[1]}") # Always close the connection when done conn.close()
```

Notes

- **Database File Creation:** If the SQLite file (e.g. `company.db`) does not exist when you connect, SQLite will automatically create it in the specified directory.
 - **SQL Injection Risk:** Never construct queries using string interpolation (`f"SELECT * FROM table WHERE id = {user_id}"`) if user input is involved. Instead, always use parameterized queries (`cursor.execute("SELECT * FROM table WHERE id = ?", (user_id,))`) to prevent SQL injection security issues.
-

Summary

- SQLite stores relational tables inside a single local file.
 - Use `pd.read_csv()` to load external CSV data.
 - Use `df.to_sql()` with a SQLAlchemy engine connection to write DataFrames directly to SQL tables.
 - Use Python's built-in `sqlite3` library for executing specific SQL queries in application logic.
-

Practice Exercises

1. Create a CSV file `products.csv` containing columns `ID`, `Name`, `Price`, and `Category`. Write a Python script to import this file into a SQLite database `inventory.db` under the table `products`.
 2. Write a Python function `get_cheap_products(max_price)` that connects to `inventory.db` using `sqlite3`, queries products costing less than `max_price`, and returns their names as a Python list.
 3. Modify your import script from Exercise 1 to append a timestamp column `imported_at` to the DataFrame before saving it to SQL. Ensure that running the script twice appends the new data instead of replacing it (use `if_exists='append'`).
-

Further Reading

- [Pandas to_sql Documentation](#)
 - [SQLAlchemy Core Tutorial](#)
 - [Python sqlite3 Module Documentation](#)
-

Building and Running REST APIs

Learning Objectives

By the end of this chapter, you will be able to:

- Establish and clean up SQLite database connections within a Flask application lifecycle.
- Implement REST API endpoints for all CRUD actions (Create, Read, Update, Delete) against database tables.
- Return structured JSON payloads using Flask's `jsonify` function.
- Enforce input validation and respond with appropriate HTTP status codes for success and failure states.

Introduction

Exposing data and functionality via a REST API requires writing route handlers that execute SQL queries, capture client input, perform validation, and return structured JSON responses.

In this chapter, we will build a complete, runnable product inventory REST API. The API will connect to a SQLite database and expose full CRUD endpoints.

Key Concepts

Database Lifecycles in Flask

When running a web server, opening and closing database connections for every query can be slow. However, keeping a single connection open indefinitely can lead to resource leaks and locking issues.

The best practice in Flask is to use a **helper function** to open a connection when a request arrives, and automatically close it when the request finishes. Flask provides the `g` object (a global namespace for temporary request-specific variables) and the `@app.teardown_appcontext` decorator to handle this cleanly.

Returning JSON

In Flask 2.0 and newer, returning a Python dictionary or list from a route automatically serializes it to JSON and sets the response Content-Type header to `application/json`. However, using `jsonify(data)` is still highly recommended because it is more explicit and allows you to build complex structure combinations.

Examples

Example 1: Full CRUD API Implementation (app.py)

Here is a complete, single-file Flask API demonstrating all CRUD operations on a SQLite database named `inventory.db`:

```
import sqlite3 from flask import Flask, jsonify, request, g app = Flask(__name__)
DATABASE = "inventory.db" # 1. Database Connection Helpers def get_db(): """Opens a
connection to the database if not already open for this request.""" db = getattr(g,
"_database", None) if db is None: db = g._database = sqlite3.connect(DATABASE) #
Configure SQLite to return rows as dictionaries instead of tuples db.row_factory =
sqlite3.Row return db @app.teardown_appcontext def close_connection(exception):
"""Automatically closes the database connection when the request finishes.""" db =
getattr(g, "_database", None) if db is not None: db.close() # 2. Database
Initialization def init_db(): """Helper to initialize the database table if it does
not exist.""" with app.app_context(): db = get_db() cursor = db.cursor()
cursor.execute(""" CREATE TABLE IF NOT EXISTS products ( id INTEGER PRIMARY KEY
AUTOINCREMENT, name TEXT NOT NULL, price REAL NOT NULL, stock INTEGER NOT NULL ) """)
db.commit() # --- API ROUTES --- # GET ALL / READ LIST @app.route("/api/products",
methods=["GET"]) def get_products(): db = get_db() cursor = db.cursor() # Check if
user filtered by category or name via query params (e.g. ?name=Pen) name_filter =
request.args.get("name") if name_filter: cursor.execute("SELECT * FROM products WHERE
name LIKE ?", (f"%{name_filter}%",)) else: cursor.execute("SELECT * FROM products")
rows = cursor.fetchall() # Convert sqlite3.Row objects to standard Python dicts
products = [dict(row) for row in rows] return jsonify(products) # GET ONE / READ ITEM
@app.route("/api/products/<int:product_id>", methods=["GET"]) def
get_product(product_id): db = get_db() cursor = db.cursor() cursor.execute("SELECT *
FROM products WHERE id = ?", (product_id,)) row = cursor.fetchone() if row is None:
return jsonify({"error": "Product not found"}), 404 return jsonify(dict(row)) # POST
/ CREATE @app.route("/api/products", methods=["POST"]) def create_product(): data =
request.get_json() if not data or "name" not in data or "price" not in data or
"stock" not in data: return jsonify({"error": "Missing required fields (name, price,
stock)"}), 400 name = data["name"] price = data["price"] stock = data["stock"] #
Simple input validation if price < 0 or stock < 0: return jsonify({"error": "Price
and stock must be non-negative"}), 400 db = get_db() cursor = db.cursor()
cursor.execute( "INSERT INTO products (name, price, stock) VALUES (?, ?, ?)", (name,
price, stock) ) db.commit() # Retrieve the newly inserted product ID new_id =
cursor.lastrowid return jsonify({"id": new_id, "name": name, "price": price, "stock":
stock}), 201 # PUT / UPDATE @app.route("/api/products/<int:product_id>",
methods=["PUT"]) def update_product(product_id): data = request.get_json() if not
data or "name" not in data or "price" not in data or "stock" not in data: return
jsonify({"error": "Missing required fields"}), 400 db = get_db() cursor = db.cursor()
# Verify product exists first cursor.execute("SELECT id FROM products WHERE id = ?",
(product_id,)) if cursor.fetchone() is None: return jsonify({"error": "Product not
found"}), 404 # Perform update query cursor.execute( "UPDATE products SET name = ?,
price = ?, stock = ? WHERE id = ?", (data["name"], data["price"], data["stock"],
product_id) ) db.commit() return jsonify({"id": product_id, "name": data["name"],
"price": data["price"], "stock": data["stock"]}) # DELETE
@app.route("/api/products/<int:product_id>", methods=["DELETE"]) def
delete_product(product_id): db = get_db() cursor = db.cursor() # Verify product
```

```

exists cursor.execute("SELECT id FROM products WHERE id = ?", (product_id,)) if
cursor.fetchone() is None: return jsonify({"error": "Product not found"}), 404
cursor.execute("DELETE FROM products WHERE id = ?", (product_id,)) db.commit() return
jsonify({"message": f"Product {product_id} deleted successfully"}) if __name__ ==
"__main__": # Ensure tables exist before starting the server init_db()
app.run(debug=True)

```

Notes

- **row_factory = sqlite3.Row:** By default, SQLite returns tuples (e.g. ("Laptop", 999.99)). By setting `row_factory`, we can reference columns by name (e.g. `row["name"]`). Converting this with `dict(row)` yields a dictionary, which makes JSON serialization clean and dynamic.
 - **Teardown App Context:** The `@app.teardown_appcontext` decorator ensures that the database connection is closed safely even if an error occurred during route processing.
-

Summary

- Manage database connections inside Flask's request lifecycle using `g` and `@app.teardown_appcontext`.
 - Return errors using standard HTTP status codes (like 400 for bad data and 404 for missing resources).
 - Convert SQLite query row dictionaries into JSON arrays or objects using `jsonify()`.
-

Practice Exercises

1. Write a script `sample_inserter.py` that connects to `inventory.db` and inserts three test items into the database so you have initial records to query.
 2. Modify the `create_product()` POST route from Example 1 to make sure that the name column is unique. If a product with the same name already exists in the database, return a 409 `Conflict` status code and a JSON error message `{"error": "Product name already exists"}`.
 3. Write a new endpoint `GET /api/products/stats` that uses SQLite aggregation queries (`AVG`, `SUM`, `COUNT`) to return a JSON summary of your inventory: `json { "total_products": 14, "average_price": 54.30, "total_value": 760.20 }`
-

Further Reading

- [Flask Application Context](#)
- [SQLite Row Objects Documentation](#)

Testing APIs with Postman

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what Postman is and how it helps verify API functionality.
- Import a pre-configured JSON collection file into Postman.
- Manage and apply Postman environments and variables.
- Run complete HTTP request testing sequences (GET, POST, PUT, DELETE) on your local Flask server.

Introduction

Once you have written your Flask API, you need a way to test it. While writing a Python script using the `requests` library is a great approach for automation, it can be slow for manual testing and prototyping.

Postman is a popular, free API client interface that allows developers to construct, save, send, and test HTTP requests. Rather than typing complex `curl` commands in the terminal, Postman provides a visual interface for entering request bodies, setting headers, inspecting status codes, and verifying JSON responses.

Key Concepts

What is a Postman Collection?

A **Collection** in Postman is a group of pre-configured HTTP requests that are saved together. Instead of rebuilding requests (setting headers, query params, and body data) every time, you can group them by feature area or project. You can save, export, and share collections as JSON files.

Variables and Environments

In production, your API might run on `https://api.mycompany.com`. In development, it runs on `http://127.0.0.1:5000`.

To avoid updating the URL inside dozens of saved requests, Postman allows you to define an **Environment** containing variables. For example, you can create a variable named `base_url` and write your request paths as: `{{base_url}}/api/products`

When you switch your active environment in Postman, `{{base_url}}` automatically resolves to the correct host.

Examples

Example 1: Importing a Collection

To import the collection file provided in this module (`postman-collection.json`): 1. Open the Postman application. 2. In the top-left corner, click the **Import** button. 3. Drag and drop the `postman-collection.json` file into the upload box. 4. Click **Import** to confirm. A new collection folder named `Module 18: Flask Inventory API` will appear in your sidebar.

Example 2: Configuring Environment Variables

To set up a local development environment: 1. Click on the **Environments** tab on the left sidebar in Postman. 2. Click the **+** (Create Environment) button and name it `Flask Local`. 3. Add a new variable: - **Variable:** `base_url` - **Type:** `default` - **Initial Value:** `http://127.0.0.1:5000` - **Current Value:** `http://127.0.0.1:5000` 4. Click **Save** in the top-right corner. 5. Set the active environment in the top-right dropdown selector to `Flask Local`.

Now, any request referencing `{{base_url}}` will target your local Flask server.

Example 3: Running the CRUD Verification Sequence

With your Chapter 6 Flask app running (`python app.py`), execute the following requests in order:

1. **GET All Products:**
2. Send `GET {{base_url}}/api/products`.
3. Verify it returns a `200 OK` status and an empty JSON array `[]` (if the database is new).
4. **POST Create Product:**
5. Select the `Create Product` request. Notice the headers include `Content-Type: application/json`.
6. The body contains: `json { "name": "Wireless Mouse", "price": 29.99, "stock": 50 }`
7. Send the request. Verify it returns `201 Created` with the product dictionary including `"id": 1`.
8. **GET Single Product:**
9. Send `GET {{base_url}}/api/products/1`.
10. Verify it returns `200 OK` and details for the "Wireless Mouse".
11. **PUT Update Product:**
12. Select `Update Product` targeting `{{base_url}}/api/products/1`.
13. Update the body to change stock to 45: `json { "name": "Wireless Mouse", "price": 29.99, "stock": 45 }`

14. Send request. Verify it returns the updated object with stock 45.
15. **DELETE Product:**
16. Send `DELETE {{base_url}}/api/products/1`.
17. Verify it returns `200 OK` and a success message.
18. **GET Deleted Product (Verification):**
19. Send `GET {{base_url}}/api/products/1` again.
20. Verify the server returns a `404 Not Found` status code and `{"error": "Product not found"}` JSON.

Notes

- **Payload Format:** If your POST or PUT requests return a `400 Bad Request` or fail to parse variables on the Flask side, check that your Postman request body type is set to **raw** and the format dropdown is set to **JSON** (which adds the critical `Content-Type: application/json` header).
- **Postman Tests Tab:** You can write JavaScript assertions inside the **Tests** tab of any request. For example: `javascript pm.test("Status code is 201", function () { pm.response.to.have.status(201); });` This is highly useful for automating API validation.

Summary

- Postman helps build, organize, and test HTTP request endpoints in a visual dashboard.
- Collections organize related requests and can be imported/exported as JSON files.
- Environment variables (`{{variable_name}}`) allow you to change API targets dynamically.
- Verifying an API involves running sequence assertions (Create -> Read -> Update -> Delete -> Verify 404).

Practice Exercises

1. Create a new environment in Postman named `Flask Staging` and set `base_url` to `http://127.0.0.1:8080`. Toggle between `Flask Local` and `Flask Staging` to see the request URLs change dynamically.

2. In the GET All Products request, add a query parameter name with value Mouse. Send the request and check the query console to verify it translates to `{{base_url}}/api/products?name=Mouse`.
3. Open the Create Product request, click on the **Tests** tab, and add a test to verify that the returned JSON contains the property `id`. Run the request to confirm the test passes.

Further Reading

- [Postman Learning Center: Introduction to Collections](#)
- [Postman Learning Center: Using Variables](#)
- [Postman Learning Center: Test Scripts](#)

Module 19: Building Web Apps with Django

Introduction to Django and Setup

Learning Objectives

By the end of this chapter, you will be able to:

- Explain Django's "batteries-included" philosophy and MVT architecture.
- Initialize a Django project and create individual application components.
- Outline the purpose of key configuration files (`settings.py`, `urls.py`, `manage.py`).
- Run initial database migrations and start the Django development server.

Introduction

In the previous module, you learned how to use Flask, a microframework that provides minimal structure, letting you choose your own tools. While this is great for simple APIs, larger enterprise applications often benefit from a **batteries-included** framework.

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It comes pre-packaged with built-in tools for database management, an admin dashboard, user authentication, security controls, and URL routing. This makes Django highly productive for larger automation systems that require robust user management, reporting, and database storage.

Key Concepts

MVT (Model-View-Template) Architecture

Unlike the classic MVC (Model-View-Controller) design, Django uses the **MVT** pattern:

1. **Model (M)**: The data access layer. It defines your database structure using Python classes (ORM) and handles database transactions.
2. **View (V)**: The business logic layer. It receives HTTP requests, interacts with models to fetch/save data, and decides what response to return (e.g. rendering a template or returning JSON).
3. **Template (T)**: The presentation layer. It contains HTML and Django Template Language (DTL) tags to generate dynamic web pages.

Django Project vs. Django App

A key design aspect of Django is separating code into projects and apps:

- **Project**: The overall web configuration and collection of apps. A project contains settings, database connection details, and global

URL routings. - **App (Application)**: A self-contained web package that performs a specific function (e.g., a blog, a user auth system, a CSV parser). A single Django project can contain multiple apps.

Examples

Example 1: Installing and Initializing a Project

To set up Django, first activate your virtual environment, then run the following commands:

```
# Install Django via pip pip install django # Create a project folder named
'myproject' django-admin startproject myproject # Move inside the project folder cd
myproject # Create a self-contained app named 'reports' python manage.py startapp
reports
```

Example 2: Project Layout

After running the commands, Django builds the following directory structure:

```
myproject/ ■■■■ manage.py # Command-line utility to interact with the project ■
■■■ myproject/ # Project configuration directory ■■■■ __init__.py ■■■■
settings.py # Project settings and configurations ■■■■ urls.py # Main URL routing
declarations ■■■■ wsgi.py / asgi.py # Deployment entry points ■■■■ reports/ #
Your custom application folder ■■■■ __init__.py ■■■■ admin.py # Admin panel
configurations ■■■■ apps.py # App config metadata ■■■■ models.py # Database tables
defined as Python classes ■■■■ tests.py # Testing scripts ■■■■ views.py # Route
controller logic
```

Example 3: Configuring Settings and Running migrations

Before running Django, you must register your app in `myproject/settings.py` so Django knows it exists.

Open `myproject/settings.py` and add `'reports'` to the `INSTALLED_APPS` list:

```
# myproject/settings.py INSTALLED_APPS = [ 'django.contrib.admin',
'django.contrib.auth', 'django.contrib.contenttypes', 'django.contrib.sessions',
'django.contrib.messages', 'django.contrib.staticfiles', # Register your custom app
here 'reports', ]
```

Next, run the initial database migrations to configure Django's built-in tables (such as admin accounts and user authentication), and start the server:

```
# 1. Apply initial database structures to SQLite database (db.sqlite3) python
manage.py migrate # 2. Start the development server python manage.py runserver
```

Open `http://127.0.0.1:8000/` in your browser. You will see Django's default welcome page confirming the installation succeeded!

Notes

- **Default Database:** By default, Django is pre-configured to use SQLite (a local file named `db.sqlite3` created inside your project root). You can easily change this to PostgreSQL, MySQL, or Oracle by modifying the `DATABASES` dictionary in `settings.py`.
 - **Port adjustments:** If port 8000 is occupied, you can start the server on a different port by appending it to the run command: `python manage.py runserver 8080`.
-

Summary

- Django is a full-featured web framework using Model-View-Template (MVT) architecture.
 - Projects contain global settings and URLs; Apps contain modular, feature-specific code.
 - Add your app to `INSTALLED_APPS` inside `settings.py` to register it.
 - Run migrations using `python manage.py migrate` and start the server with `python manage.py runserver`.
-

Practice Exercises

1. Initialize a Django project named `company_portal` and create an application within it named `inventory`.
 2. Add your `inventory` app to the list of `INSTALLED_APPS` inside the `settings.py` of `company_portal`.
 3. Apply the initial migrations to the SQLite database and start the development server on port 9000.
-

Further Reading

- [Django Official Documentation: First Steps](#)
 - [Django Settings Reference](#)
-

Designing Views, Routing, and Templates

Learning Objectives

By the end of this chapter, you will be able to:

- Write function-based views to process requests and return HTTP responses.
- Set up a clean, modular routing architecture using project and app-level `urls.py` files.
- Create and render HTML templates using the Django Template Language (DTL).
- Configure static directories and load client-side stylesheets in your templates.

Introduction

In Flask, routing is defined directly above functions using decorators. Django, however, enforces a clean separation of concerns: URLs are declared in dedicated routing modules (`urls.py`), logic is written in view modules (`views.py`), and presentation is stored in template files.

In this chapter, you will learn how to connect these pieces together to serve dynamic HTML pages styled with static CSS files.

Key Concepts

Function-Based Views (FBVs)

A Django view is a Python function that takes an `HttpRequest` object as its first argument and returns an `HttpResponse` object. The response can be raw text, a redirect, a JSON payload, or rendered HTML.

Modular Routing with `urls.py`

Django maps URLs using the `path()` function. For clean organization, you should split your routing:

1. **Project-level `urls.py`:** Handles global paths and forwards specific sub-paths to their respective apps using the `include()` helper.
2. **App-level `urls.py`:** Manages the paths specific to that application.

Templates and Namespace Isolation

Django templates are HTML files that utilize DTL (Django Template Language).

By default, Django scans the `templates/` folder of all registered apps. To prevent multiple apps from having templates with the same name (e.g. `index.html`), Django best practice is to place templates inside an app-specific subfolder: `reports/templates/reports/dashboard.html`

Django Template Language (DTL)

Like Jinja2, DTL uses double braces and percent brackets: - `{{ user_name }}` - Variable placeholder. - `{% for sale in sales_list %}` - Control blocks. - `{% load static %}` - Special Django tag used to enable loading files from static folders.

Examples

Example 1: Creating a View and App-level URLs

First, create a view in `reports/views.py`:

```
# reports/views.py from django.http import HttpResponse def hello_report(request):
return HttpResponse("Welcome to the Reports App!")
```

Next, create a routing file `reports/urls.py` inside the app folder:

```
# reports/urls.py from django.urls import path from . import views urlpatterns = [ #
Match empty path (relative to the app namespace) path("", views.hello_report,
name="hello_report"), ]
```

Example 2: Forwarding from Project URLs

To connect the app routing to the main project, edit `myproject/urls.py`:

```
# myproject/urls.py from django.contrib import admin from django.urls import path,
include urlpatterns = [ path('admin/', admin.site.urls), # Include all paths from the
reports app under the root URL path('', include('reports.urls')), ]
```

Run `python manage.py runserver`. When you visit `http://127.0.0.1:8000/`, your view will run and show "Welcome to the Reports App!".

Example 3: Rendering a Template and Serving Stylesheets

Let's render a page. Create a template file under: `reports/templates/reports/home.html`

```
{% load static %} <!DOCTYPE html> <html> <head> <title>Reports Dashboard</title> <!--
Use static template tag to generate the stylesheet path --> <link rel="stylesheet"
href="{% static 'reports/style.css' %}"> </head> <body> <h1>Report Overview</h1>
<p>Logged in as: <strong>{{ analyst_name }}</strong></p> </body> </html>
```

Create a stylesheet under `reports/static/reports/style.css`:

```
body { font-family: 'Segoe UI', sans-serif; background-color: #f8fafc; padding: 30px;
} h1 { color: #1e293b; border-bottom: 2px solid #3b82f6; padding-bottom: 8px; }
```

Update your view in `reports/views.py` to render the template:

```
# reports/views.py from django.shortcuts import render def home_view(request):
context = {"analyst_name": "Amol Chawathe"} # render() automatically looks inside
'templates/' folders return render(request, "reports/home.html", context)
```

Finally, bind this view to a path in `reports/urls.py`:

```
# reports/urls.py path("home/", views.home_view, name="home"),
```

Visit `http://127.0.0.1:8000/home/` to see your styled page.

Notes

- **URL Trailing Slashes:** By default, Django enforces trailing slashes on URL patterns (e.g. `path('home/', ...)`). If a user visits `/home`, Django will automatically issue a 301 Redirect to `/home/`.
- **Static files configuration:** During local development (`DEBUG = True`), Django automatically serves files inside any app's `static/` folder. For production, Django uses a command called `python manage.py collectstatic` to gather all static files into a single folder for high-performance serving by web servers (like Nginx).

Summary

- Django views receive an `HttpRequest` and return an `HttpResponse`.
- Use `include()` in project-level `urls.py` to route requests to app-level `urls.py` files.
- Place HTML files inside `templates/<app_name>/` and reference them using `render(request, "app_name/file.html", context)`.
- Serve assets from `static/<app_name>/` and link them using `{% load static %}` and `{% static 'path' %}`.

Practice Exercises

1. Add a dynamic route to `reports/urls.py` that captures a string parameter `category`:
`path('category/<str:category>/', views.category_view, name='category')`.
2. Write the view `category_view` in `reports/views.py` to output: `"Displaying records for category: [category_name]"`.

3. Create a template `reports/detail.html` that renders a dictionary representing a report (containing `title`, `created_date`, and `row_count`). Style this template using a dedicated CSS stylesheet inside your static directory.
-

Further Reading

- [Django Official Documentation: URL dispatcher](#)
- [Django Official Documentation: Views](#)
- [Django Templates Introduction](#)

CSV Import and Database CRUD

Learning Objectives

By the end of this chapter, you will be able to:

- Define database structures by writing Django Model classes.
- Generate and apply SQL database migrations.
- Parse CSV files with Pandas and insert them into the database using Django's ORM.
- Perform standard Create, Read, Update, and Delete (CRUD) operations via the Django ORM.

Introduction

Django features a built-in **Object-Relational Mapper (ORM)**. The ORM allows you to define database tables as standard Python classes (called **Models**) and run SQL queries using Python syntax, without writing raw SQL.

In this chapter, you will learn how to define a model, run migrations, and write a Python automation script that loads a CSV file into Pandas, cleans it, and bulk-inserts the rows directly into your Django database using the ORM.

Key Concepts

Defining Django Models

A model represents a single table in your database. Each property in the class represents a column. Models are declared in `models.py` and inherit from `django.db.models.Model`.

Common Django field types:

- `CharField`: For short text fields (requires `max_length`).
- `IntegerField` / `FloatField`: For numerical values.
- `DateField` / `DateTimeField`: For date/time values.
- `TextField`: For large, multi-line text blocks.

The Migration Lifecycle

Whenever you create or edit a model class, you must synchronize the database structure:

1. `python manage.py makemigrations`: Scans `models.py` and creates a recipe file (in python) detailing the database changes.
2. `python manage.py migrate`: Executes those recipe files, running SQL commands (`CREATE TABLE`, `ALTER TABLE`) against the SQLite database.

Bulk Data Ingestion

While you can insert rows one-by-one using `Model.objects.create()`, this is slow for large datasets because it makes a database network roundtrip for every row.

Instead, you can load your CSV using Pandas, build a list of Django Model instances, and insert them in a single query using `bulk_create()`.

Examples

Example 1: Defining a Model

Open `reports/models.py` and write:

```
# reports/models.py from django.db import models class Sale(models.Model):
product_name = models.CharField(max_length=100) quantity = models.IntegerField()
price = models.FloatField() sale_date = models.DateField() category =
models.CharField(max_length=50) def __str__(self): return f"{self.product_name}
({self.quantity} sold)"
```

Create and run the database tables:

```
python manage.py makemigrations python manage.py migrate
```

Example 2: Importing CSV into the Model

Create a sample CSV `sales_data.csv` in your project folder:

```
Product,Qty,Price,Date,Category Laptop,2,999.99,2026-06-01,Electronics
Mouse,5,24.99,2026-06-02,Electronics Chair,1,149.99,2026-06-03,Furniture
Keyboard,3,49.99,2026-06-04,Electronics
```

To run a script that imports this data, we can write a custom Django management script. Create a file named `import_sales.py` in your project folder:

```
# import_sales.py import os import django import pandas as pd # 1. Setup Django
environment settings os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"myproject.settings") django.setup() # Import the model AFTER setting up Django from
reports.models import Sale def run_import(): csv_file = "sales_data.csv" if not
os.path.exists(csv_file): print("CSV file not found!") return # 2. Read CSV with
Pandas df = pd.read_csv(csv_file) # Clean up column names and fill missing data if
any df['Qty'] = df['Qty'].fillna(0).astype(int) df['Price'] =
df['Price'].fillna(0.0).astype(float) sales_to_create = [] # 3. Iterate through
DataFrame rows for index, row in df.iterrows(): # Build Sale model instances sale =
Sale( product_name=row['Product'], quantity=row['Qty'], price=row['Price'],
sale_date=row['Date'], category=row['Category'] ) sales_to_create.append(sale) # 4.
Bulk insert into the database Sale.objects.bulk_create(sales_to_create)
print(f"Successfully imported {len(sales_to_create)} sales records into SQLite!") if
__name__ == "__main__": run_import()
```

Run the script from your terminal:

```
python import_sales.py
```

Example 3: Querying the Database (CRUD)

Once data is imported, you can perform query operations:

```
import os
import django
os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"myproject.settings")
django.setup()
from reports.models import Sale

# 1. Create a single sale
new_sale = Sale.objects.create(
    product_name="USB Hub",
    quantity=10,
    price=19.99,
    sale_date="2026-06-05",
    category="Electronics"
)

# 2. Read (Querying) all sales
all_sales = Sale.objects.all()

# Filter for electronics
electronics = Sale.objects.filter(
    category="Electronics"
)

# Filter for expensive items
expensive = Sale.objects.filter(
    price__gt=100.0
)

# 3. Update sale_item
sale_item = Sale.objects.get(id=1)
sale_item.price = 949.99
sale_item.save()

# 4. Delete old_sale
old_sale = Sale.objects.get(
    product_name="Chair"
)
old_sale.delete()
```

Notes

- **django.setup():** If you run a standalone Python script outside of `manage.py`, you must call `django.setup()` and define the environment variable `DJANGO_SETTINGS_MODULE` first. Otherwise, Django will raise an `AppRegistryNotReady` error because it does not know where your configuration files are.
- **Double Underscore Lookups:** Django ORM uses double underscores (`__`) for filtering conditions (e.g. `price__gt=100` means price greater than, `product_name__contains="Mouse"` means containing substring).

Summary

- Django Models map Python classes directly to database tables.
- Generate migration scripts with `makemigrations` and execute them with `migrate`.
- Use Pandas to read and clean CSV data, then use `bulk_create` to insert data efficiently.
- Perform database tasks using the ORM: `create()`, `all()`, `filter()`, `get()`, `save()`, and `delete()`.

Practice Exercises

1. Create a model in `reports/models.py` named `Analyst` containing fields `name` (Char), `department` (Char), and `active` (Boolean). Run the migrations.
 2. Write a Python script to insert three Analysts into the database.
 3. Write an ORM query to retrieve and print the names of all active Analysts.
-

Further Reading

- [Django Official Documentation: Models](#)
- [Django ORM Making Queries](#)
- [Pandas DataFrame Iteration Reference](#)

Building APIs with DRF and APIView

Learning Objectives

By the end of this chapter, you will be able to:

- Install and configure Django Rest Framework (DRF) inside a project.
- Write Serializer classes to handle validation and model-to-JSON serialization.
- Implement REST API endpoints for GET, POST, PUT, and DELETE actions using `APIView`.
- Set up URL patterns for class-based views.

Introduction

While Flask uses the simple `jsonify()` helper to convert dictionary objects into JSON, Django has a more structured approach. To construct professional REST APIs, Django developers use the **Django Rest Framework (DRF)**.

DRF provides powerful tools, including:

1. **Serializers**: Classes that validate incoming input data and convert complex Django model objects into standard JSON arrays/objects.
2. **APIView**: A class-based view system that handles different HTTP verbs (GET, POST, etc.) inside structured class methods.

Key Concepts

Installing DRF

Install `django-rest-framework` inside your virtual environment:

```
pip install djangorestframework
```

Add `'rest_framework'` to the list of `INSTALLED_APPS` inside `settings.py`:

```
# myproject/settings.py INSTALLED_APPS = [ ... 'rest_framework', 'reports', ]
```

Serializers

Serializers are similar to Django models, but instead of mapping to database tables, they map tables to JSON structures. They handle both:

- **Serialization**: Converting a database query or model instance to a Python dictionary that can easily be sent as JSON.
- **Deserialization**: Validating incoming JSON data (e.g. from a POST request) and saving it back to a database model.

APIView

Unlike function-based views where you use `if request.method == 'POST'`, `APIView` lets you write clean Python classes with methods named after HTTP verbs:

```
from rest_framework.views import APIView from rest_framework.response import Response
class MyAPI(APIView): def get(self, request): return Response({"message": "GET
request received"}) def post(self, request): return Response({"message": "POST
request received"})
```

Examples

Example 1: Creating a Serializer

Create a new file `reports/serializers.py`:

```
# reports/serializers.py from rest_framework import serializers from .models import
Sale class SaleSerializer(serializers.ModelSerializer): class Meta: model = Sale #
List the fields you want to expose in your JSON API fields = ['id', 'product_name',
'quantity', 'price', 'sale_date', 'category']
```

Example 2: Implementing APIView CRUD

Open `reports/views.py` and implement the class-based views:

```
# reports/views.py from rest_framework.views import APIView from
rest_framework.response import Response from rest_framework import status from
.models import Sale from .serializers import SaleSerializer # Handler for /api/sales/
(Get list, Create items) class SaleListAPIView(APIView): def get(self, request):
sales = Sale.objects.all() # many=True tells the serializer we are passing a list of
objects serializer = SaleSerializer(sales, many=True) return
Response(serializer.data, status=status.HTTP_200_OK) def post(self, request): # Pass
request body data to the serializer serializer = SaleSerializer(data=request.data) if
serializer.is_valid(): serializer.save() # Inserts the record into the database
return Response(serializer.data, status=status.HTTP_201_CREATED) return
Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST) # Handler for
/api/sales/<int:pk>/ (Get single item, Update item, Delete item) class
SaleDetailAPIView(APIView): def get_object(self, pk): try: return
Sale.objects.get(pk=pk) except Sale.DoesNotExist: return None def get(self, request,
pk): sale = self.get_object(pk) if not sale: return Response({"error": "Sale not
found"}, status=status.HTTP_404_NOT_FOUND) serializer = SaleSerializer(sale) return
Response(serializer.data, status=status.HTTP_200_OK) def put(self, request, pk): sale
= self.get_object(pk) if not sale: return Response({"error": "Sale not found"},
status=status.HTTP_404_NOT_FOUND) # Pass instance AND data to update it serializer =
SaleSerializer(sale, data=request.data) if serializer.is_valid(): serializer.save()
return Response(serializer.data, status=status.HTTP_200_OK) return
Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST) def delete(self,
request, pk): sale = self.get_object(pk) if not sale: return Response({"error": "Sale
not found"}, status=status.HTTP_404_NOT_FOUND) sale.delete() return
Response({"message": "Sale deleted successfully"}, status=status.HTTP_200_OK)
```

Example 3: Routing Class-Based Views

To register class-based views in `reports/urls.py`, you must call the `.as_view()` method:

```
# reports/urls.py
from django.urls import path
from .views import SaleListAPIView, SaleDetailAPIView

urlpatterns = [
    # Class-based views must use .as_view()
    path("api/sales/", SaleListAPIView.as_view(), name="sales-list"),
    path("api/sales/<int:pk>/", SaleDetailAPIView.as_view(), name="sales-detail"),
]
```

Notes

- **Primary Key (pk):** Django uses `pk` (primary key) by convention in URL routing and object retrievals to refer to the primary key column (which is typically `id`).
- **DRF Request & Response:** DRF request objects are different from Django's default `HttpRequest`. You access JSON request data directly via `request.data` (which handles parsing JSON automatically), instead of calling `request.get_json()`.

Summary

- DRF is a powerful toolkit for building Web APIs in Django.
- `ModelSerializers` automatically create validation and mapping configurations based on your `Model` fields.
- `APIView` organizes CRUD endpoints using class methods named after HTTP verbs (`get`, `post`, `put`, `delete`).
- Access parsed JSON request bodies using `request.data` and return responses with the DRF `Response` object.

Practice Exercises

1. Create a `Serializer` class for the `Analyst` model.
2. Build an `AnalystListAPIView` class that supports listing and creating `Analysts`. Ensure that it returns correct status codes on success and validation failure.
3. Configure your app-level URLs to routing `/api/analysts/` to your `AnalystListAPIView` endpoint.

Further Reading

- [Django Rest Framework: Serializers](#)
- [Django Rest Framework: Class-based Views](#)
- [DRF Status Codes Reference](#)

Pagination, Filtering, and Ordering

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the performance benefits of paginating API responses.
- Implement DRF's `PageNumberPagination` inside class-based views.
- Sort database querysets using Django's `.order_by()` ORM methods.
- Retrieve URL query parameters to apply dynamic filtering and sorting rules.

Introduction

As your automation database grows, querying records becomes slower. Returning thousands of rows in a single HTTP response wastes server memory and network bandwidth.

To build professional, high-performance APIs, you must implement:

1. **Pagination**: Breaking a large list of results down into smaller, page-sized chunks (e.g., 20 results per page).
2. **Ordering**: Letting the client sort the results by specific fields (e.g. sorting sales by price or newest date).
3. **Filtering**: Allowing clients to restrict results based on parameters (e.g., category or name).

In this chapter, you will learn how to add these capabilities to your DRF views.

Key Concepts

Database Pagination in DRF

DRF provides built-in pagination helper classes. The most common is `PageNumberPagination`, which uses a `?page=X` query parameter.

To use pagination inside a basic `APIView`, you must manually initialize the pagination class, paginate the queryset, and return a paginated response wrapper using the helper method `get_paginated_response()`.

Queryset Ordering

In Django's ORM, you sort records using the `.order_by()` method:

- `Sale.objects.all().order_by('sale_date')` - Sorts by sale date in **ascending** order.
- `Sale.objects.all().order_by('-sale_date')` - Sorts by sale date in **descending** order (prefixed with `-`).
- `Sale.objects.all().order_by('-price', 'product_name')` - Sorts by price descending, then alphabetically by name.

Dynamic Filtering and Parameter Retrieval

You can inspect URL query parameters using `request.query_params` (which is DRF's equivalent of Flask's `request.args`).

For example, if a client calls `/api/sales/?category=Electronics&ordering=-price:` -
`request.query_params.get('category')` resolves to `"Electronics"`. -
`request.query_params.get('ordering')` resolves to `"-price"`.

Examples

Example 1: Creating a Custom Pagination Class

Create a custom class to define the page size rules. Open `reports/pagination.py`:

```
# reports/pagination.py from rest_framework.pagination import PageNumberPagination
class StandardResultsSetPagination(PageNumberPagination):
    default_limit = 10 # Default number of items per page
    page_size = 5 # Default page size
    page_size_query_param = 'page_size' # Let client specify page size (e.g.
    #page_size=20)
    max_page_size = 100 # Max page limit a client can request
```

Example 2: Integrating Pagination, Ordering, and Filtering in APIView

Update `SaleListAPIView` inside `reports/views.py` to support filtering, sorting, and pagination:

```
# reports/views.py from rest_framework.views import APIView from
rest_framework.response import Response from rest_framework import status from
.models import Sale from .serializers import SaleSerializer from .pagination import
StandardResultsSetPagination
class SaleListAPIView(APIView):
    def get(self, request):
        # 1. Start with base queryset
        queryset = Sale.objects.all()
        # 2. Apply Filtering based on query parameter (?category=Electronics)
        category_filter = request.query_params.get("category")
        if category_filter:
            queryset = queryset.filter(category__iexact=category_filter)
        # 3. Apply Ordering based on parameter (?ordering=-price)
        # Default to sorting by sale_date descending if parameter is missing
        ordering = request.query_params.get("ordering", "-sale_date")
        # Enforce allowed fields for ordering to prevent SQL errors
        allowed_ordering_fields = ["price", "-price", "sale_date", "-sale_date", "quantity", "-quantity"]
        if ordering in allowed_ordering_fields:
            queryset = queryset.order_by(ordering)
        else:
            queryset = queryset.order_by("-sale_date")
        # 4. Apply Pagination
        paginator = StandardResultsSetPagination()
        page = paginator.paginate_queryset(queryset, request, view=self)
        if page is not None:
            serializer = SaleSerializer(page, many=True)
            # Returns a wrapper dict containing 'count', 'next', 'previous', and 'results'
            return paginator.get_paginated_response(serializer.data)
        # Fallback if pagination is disabled or fails
        serializer = SaleSerializer(queryset, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)
    def post(self, request):
        serializer = SaleSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Example 3: Verifying the API Output

If you start your server and call `GET http://127.0.0.1:8000/api/sales/?category=Electronics&ordering=-price&page=1`, the response JSON is automatically structured as follows:

```
{ "count": 3, "next": null, "previous": null, "results": [ { "id": 1, "product_name": "Laptop", "quantity": 2, "price": 999.99, "sale_date": "2026-06-01", "category": "Electronics" }, { "id": 4, "product_name": "Keyboard", "quantity": 3, "price": 49.99, "sale_date": "2026-06-04", "category": "Electronics" }, { "id": 2, "product_name": "Mouse", "quantity": 5, "price": 24.99, "sale_date": "2026-06-02", "category": "Electronics" } ] }
```

Notes

- **UnorderedObjectListWarning:** If you apply pagination to a queryset without calling `.order_by()`, Django will issue a warning in the terminal. Without sorting, database rows can load in arbitrary order, causing duplicate or skipped items across page queries.
- **Lazy Evaluation:** Remember that Django querysets are lazy. Writing `queryset = queryset.filter(...)` or `queryset = queryset.order_by(...)` does not query the database immediately. The SQL query is only built and executed when the paginator evaluates the list.

Summary

- Use pagination to split data payloads, improving server latency and response size.
- Implement DRF's `PageNumberPagination` by paginating the queryset and returning `get_paginated_response()`.
- Use `.order_by()` to sort records. Prefix fields with `-` for descending order.
- Inspect incoming query parameters via `request.query_params` to build dynamic SQL filters.

Practice Exercises

1. Update the ordering rules in Example 2 to allow sorting alphabetically by product name (`product_name` and `-product_name`).
2. Write a custom pagination class named `TinyPagination` that sets a default page size of 2.

3. Apply `TinyPagination` to your active endpoints. Visit `/api/sales/?page=2` to verify that you receive records 3 and 4 of your inventory list.

Further Reading

- [Django Rest Framework: Pagination](#)
- [Django QuerySet Reference: order_by](#)
- [Django QuerySet Reference: filter](#)

Displaying Graphs in Templates

Learning Objectives

By the end of this chapter, you will be able to: - Run database aggregation and grouping queries using Django's ORM. - Format database results into arrays compatible with JavaScript charts. - Render dynamic graphs (such as Bar and Line charts) in templates using **Chart.js**. - Pass data lists safely to the frontend without character escaping issues.

Introduction

Data ingestion and API construction are major parts of automation. However, managers and operators often need a clean visual representation of the metrics. Rather than logging stats or exporting raw Excel sheets, you can build a reporting web page.

In this chapter, you will learn how to write Django queries that calculate aggregates (such as summing total sales by category), pass these numbers to a Django template, and render beautiful, interactive canvas charts using Chart.js.

Key Concepts

ORM Aggregation and Grouping

To calculate summary statistics in Django, you import aggregate functions (`Sum`, `Count`, `Avg`, `Max`) from `django.db.models`.

To group data (similar to a SQL `GROUP BY`), you combine the `.values()` and `.annotate()` methods:

```
from django.db.models import Sum from reports.models import Sale # Calculate sum of
'price * quantity' (or sum of quantity) grouped by category stats =
Sale.objects.values('category').annotate(total_qty=Sum('quantity'))
```

This returns a list of dictionaries, such as: `[{'category': 'Electronics', 'total_qty': 10}, {'category': 'Furniture', 'total_qty': 1}]`

Rendering Charts with Chart.js

Chart.js is a lightweight, open-source JavaScript library that renders charts inside HTML5 `<canvas>` tags. You do not need to install it via pip or npm; you can load it directly into your HTML page from a Content Delivery Network (CDN) script link:

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

Passing Python Lists to JavaScript

When passing lists of strings (e.g. ['Electronics', 'Furniture']) from a Django view into JavaScript, the Django template engine will automatically escape characters like single quotes into HTML entities (e.g. ').

To prevent this escaping and allow JavaScript to parse the array correctly, use Django's `|safe` filter:

```
const categories = {{ categories_list|safe }};
```

Examples

Example 1: The Aggregation View

Open `reports/views.py` and write the dashboard logic:

```
# reports/views.py from django.shortcuts import render from django.db.models import
Sum from .models import Sale import json def dashboard_view(request): # 1. Query sum
of quantities sold, grouped by category category_data =
Sale.objects.values('category').annotate(total_sold=Sum('quantity')) # 2. Separate
query results into two parallel lists for Chart.js categories = [] quantities = []
for entry in category_data: categories.append(entry['category'])
quantities.append(entry['total_sold']) # Convert lists to JSON strings to ensure safe
formatting in JS context = { "categories_js": json.dumps(categories),
"quantities_js": json.dumps(quantities), "total_sales_count": Sale.objects.count() }
return render(request, "reports/dashboard.html", context)
```

Bind this view inside `reports/urls.py`:

```
# reports/urls.py path("dashboard/", views.dashboard_view, name="dashboard"),
```

Example 2: The HTML Template with Chart.js

Create a new file `reports/templates/reports/dashboard.html`:

```
<!DOCTYPE html> <html> <head> <title>Reports Dashboard</title> <!-- 1. Load Chart.js
CDN --> <script src="https://cdn.jsdelivr.net/npm/chart.js"></script> <style> body {
font-family: Arial, sans-serif; background-color: #f1f5f9; margin: 30px; } .container
{ max-width: 800px; background-color: white; padding: 20px; border-radius: 8px;
box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1); } h1 { color: #1e3a8a; } .chart-box {
margin-top: 20px; position: relative; height: 400px; } </style> </head> <body> <div
class="container"> <h1>Sales Analytics</h1> <p>Total transaction items: <strong>{{
total_sales_count }}</strong></p> <!-- 2. Create canvas element for the chart -->
<div class="chart-box"> <canvas id="salesChart"></canvas> </div> <script> //
3. Retrieve variables passed from Django view (using |safe filter) const chartLabels
= {{ categories_js|safe }}; const chartDataValues = {{ quantities_js|safe }}; // 4.
```

```

Configure and render Chart.js
const ctx =
document.getElementById('salesChart').getContext('2d');
new Chart(ctx, { type: 'bar',
data: { labels: chartLabels, datasets: [{ label: 'Total Quantity Sold', data:
chartDataValues, backgroundColor: [ 'rgba(59, 130, 246, 0.7)', // blue 'rgba(245,
158, 11, 0.7)', // amber 'rgba(16, 185, 129, 0.7)', // emerald 'rgba(239, 68, 68,
0.7)' // red ], borderColor: [ 'rgb(59, 130, 246)', 'rgb(245, 158, 11)', 'rgb(16,
185, 129)', 'rgb(239, 68, 68)' ], borderWidth: 1 }] }, options: { responsive: true,
maintainAspectRatio: false, scales: { y: { beginAtZero: true } } } });
</script>
</body> </html>

```

Start the Django server (python manage.py runserver) and open `http://127.0.0.1:8000/dashboard/`. You will see a beautiful bar chart displaying your aggregated sales statistics!

Notes

- **json.dumps()**: Converting lists to JSON strings in Python using `json.dumps()` is safer than sending a raw list. It ensures brackets, quotes, and strings are correctly encoded for Javascript parser compatibility.
- **Canvas Size**: Always wrap the Chart.js `<canvas>` element in a parent `<div>` with specified width and height. Chart.js requires this wrapper container to correctly calculate scales when performing responsive screen resizing.

Summary

- Import database aggregates (like Sum or Count) from `django.db.models` to summarize record numbers.
- Group rows using `.values('group_field').annotate(summary_name=Sum('sum_field'))`.
- Use a Chart.js script CDN link to build frontend canvas charts without packages.
- Apply the `|safe` filter to variables passed to Javascript to avoid string formatting encoding issues.

Practice Exercises

1. Modify the `dashboard_view` in `reports/views.py` to calculate the total sales income (quantity * price) per category. Expose this list to Javascript.

2. Add a second chart canvas container in your dashboard template and render a Pie Chart showing the distribution of total sales value.
 3. Write a query to get sales quantities grouped by date, and plot this as a Line Chart to see the timeline progression of sales volumes.
-

Further Reading

- [Django DB Aggregation Documentation](#)
- [Chart.js Official Getting Started Guide](#)

